



FOM Hochschule für Oekonomie & Management

Hochschulzentrum Düsseldorf

Bachelor-Thesis

im Studiengang Wirtschaftsinformatik

zur Erlangung des Grades eines

Bachelor of Science (B.Sc.)

über das Thema

**Evaluierung der Prädiktion von Wort-Vektoren bezogen auf Online
Bewertungen eines Versandhändlers anhand differentieller Textquellen
mittels Machine Learning**

von

Nils Horn

Erstgutachter Prof. Dr. Rüdiger Buchkremer

Matrikelnummer 384420

Abgabedatum 2019-04-05

Inhaltsverzeichnis

I.	Abbildungsverzeichnis.....	IV
II.	Abkürzungsverzeichnis.....	V
III.	Formelverzeichnis.....	VI
IV.	Tabellenverzeichnis	VII
1.	Einführung in die Thematik	1
1.1	Motivation.....	1
1.2	Zielsetzung der Thesis	2
1.3	Aufbau der Arbeit und Methodik.....	3
2.	Theoretische Grundlagen.....	5
2.1	Machine Learning	5
2.2	Deep Learning und neuronale Netze.....	9
2.3	Natural Language Processing und Word Embedding	12
3.	Verfahren des Word Embeddings	16
3.1	Word2Vec.....	16
3.1.1	Definition und Parameter	16
3.1.2	Neuronale Architekturen.....	19
3.1.3	Lernalgorithmen.....	21
3.2	GloVe.....	23
3.2.1	Definition des Ansatzes sowie Besonderheiten.....	23
3.2.2	Co-Occurrence Matrix	25
4.	Methodische Vorgehensweise.....	28
4.1	Datenbeschaffung	28
4.1.1	Beschaffung der Evaluierungsdaten.....	28
4.1.2	Beschaffung der Trainingsdaten	29
4.2	Datenaufbereitung.....	30
4.2.1	Aufbereitung der Evaluierungsdaten.....	30
4.2.2	Aufbereitung der Trainingsdaten	31
4.3	Training von Word Embeddings.....	33
4.3.1	Auswahl der Trainingsmethoden	33
4.3.2	Prototypisierung der Trainingsmethoden.....	34
4.3.3	Trainingsdurchführung.....	38
4.4	Visualisierung der erstellten Vektoren.....	40
4.5	Evaluierung der erstellten Word Embeddings	42

5.	Anwendung von Word Embedding Verfahren	46
6.	Empirische Untersuchung	50
6.1	Präsentation der Ergebnisse	50
6.2	Statistische Ergebnisevaluation.....	57
6.2.1	Durchführung einer Varianzanalyse zur Hypothesenuntersuchung.....	57
6.2.2	Vergleich der Datenrelationen mittels Post-Hoc Test	60
6.2.3	Anwendung eines t-Tests zum Verfahrensvergleich.....	62
6.3	Validierung der Thesen.....	64
7.	Diskussion der Resultate	68
7.1	Wissenschaftliche Betrachtung.....	68
7.2	Implikationen für die Praxis.....	69
7.3	Reflexion des Vorgehens	70
7.4	Beschränkungen der Arbeit.....	72
8.	Abschließende Betrachtung	74
8.1	Schlussfolgerung mit Fazit.....	74
8.2	Forschungsausblick.....	75
V.	Literatur- und Quellenverzeichnis	VIII
VI.	Anhang.....	XXIV

I. Abbildungsverzeichnis

Abbildung 1: Datenclustering im Machine Learning.....	8
Abbildung 2: Aufbau eines neuronalen Netzes	10
Abbildung 3: semantische Beziehungen von Wörtern durch Word Embedding	13
Abbildung 4: Aufbau der CBOW Architektur	20
Abbildung 5: Aufbau der Skip-Gram Architektur.....	21
Abbildung 6: Hierarchical Softmax mit Huffmann Tree.....	22
Abbildung 7: Matrix Faktorisierung bei GloVe.....	24
Abbildung 8: Co-Occurrence Matrix im GloVe Verfahren	26
Abbildung 9: Selektion von ASIN Nummern aus HTML Quelltexten	28
Abbildung 11: Frequenz verwendeter Programmiersprachen	37
Abbildung 13: Verlustreduzierung per Iteration beim Training von Wortvektoren	39
Abbildung 14: Darstellung von Wortvektoren mit t-SNE.....	41
Abbildung 15: Oberfläche der Evaluierungsplattform.....	42
Abbildung 16: H1a - Prädiktionsmessung von GloVe mit 150 Megabyte	51
Abbildung 17: H1a - Prädiktionsmessung von GloVe mit 300 Megabyte	51
Abbildung 18: H1a - Prädiktionsmessung von GloVe mit 500 Megabyte	52
Abbildung 19: H1b - Prädiktionsmessung von Word2Vec mit 150 Megabyte	52
Abbildung 20: H1b - Prädiktionsmessung von Word2Vec mit 300 Megabyte	53
Abbildung 21: H1b - Prädiktionsmessung von Word2Vec mit 500 Megabyte	53
Abbildung 22: H2a -Prädiktionsmessung von GloVe mit 15 Iterationen	54
Abbildung 23: H2a - Prädiktionsmessung von GloVe mit 50 Iterationen	55
Abbildung 24: H2a - Prädiktionsmessung von GloVe mit 100 Iterationen	55
Abbildung 25: H2b - Prädiktionsmessung von Word2Vec mit 15 Iterationen.....	56
Abbildung 26: H2b - Prädiktionsmessung von Word2Vec mit 50 Iterationen.....	56
Abbildung 27: H2b - Prädiktionsmessung von Word2Vec mit 100 Iterationen.....	57
Abbildung 28: H1a - Varianzanalyse für GloVe	58
Abbildung 29: H1b - Varianzanalyse für Word2Vec.....	59
Abbildung 30: H2a - Varianzanalyse für GloVe	59
Abbildung 31: H2b - Varianzanalyse für Word2Vec.....	60
Abbildung 32: H1 - Post-Hoc Test für Word2Vec.....	61
Abbildung 33: H2 - Post-Hoc Test für GloVe	62
Abbildung 34: L1 - t-Test mit unbereinigten Daten	63
Abbildung 35: L1 - t-Test mit bereinigten Daten	64

II. Abkürzungsverzeichnis

Adam = Adaptive Moment Estimation

Amazon.com = Amazon

ANOVA = Analysis of Variance

API = Application Programming Interface

ASIN = Amazon Standard Identification Number

CBOW = Continuous Bag of Words

DL = Deep Learning

HS = Hierarchical Softmax

HSD = Honestly Significant Difference

LR = Learning Rate

LSTM = Long Short-Term Memory

ML = maschinelles Lernen

n. = nicht

NLP = Natural Language Processing

NS = Negative Sampling

RegEx = Regular Expressions

RNN = Rekurrentes neuronales Netz

SG = Skip-Gram

TF = TensorFlow

t-SNE = t-distributed Stochastic Neighbor

UML = Unified Modeling Language

W2V = Word2Vec

WE = Word Embedding

III. Formelverzeichnis

Formel 1: Distanzberechnung zur Ermittlung ähnlicher Wörter	14
---	----

IV. Tabellenverzeichnis

Tabelle 1: Logische Anwendung einer Co-Occurrence Matrix.....	26
Tabelle 3: Parametrisierung der Evaluierungsplattform.....	43
Tabelle 4: Parametrisierung des Optimizers der Evaluierungsplattform	44
Tabelle 5: Konfigurationen der Trainingssysteme	46
Tabelle 6: Parametrisierung der Verfahren	48
Tabelle 7: Validierung der Hypothesen.....	65

1. Einführung in die Thematik

1.1 Motivation

Während im Jahr 2018 schätzungsweise bereits 33 Zettabyte an neuen Daten erzeugt wurden, prognostiziert Statista (2019) eine Datengenerierung von 175 Zettabyte bereits im Jahr 2025¹, was einem Datenzuwachs von ca. 530 Prozent innerhalb von sieben Jahren entspräche. Innerhalb dieser Daten befindet sich ein großer Anteil an User-Generated-Content, der überwiegend aus Texten sowie Bildern besteht. Wie eine Statistik zeigt, wurden in einer gemessenen Minute im Juni 2018 bereits knapp 13 Millionen Textnachrichten versendet². Innerhalb dieser Daten stecken zahlreiche ungenutzte Daten, die beispielsweise betriebswirtschaftlich genutzt werden könnten.

Um die Flut an Daten interpretieren zu können, vertrauen immer mehr Unternehmen dem maschinellen Lernen. Eine Statistik von Statista (2019) zeigt auf, dass einer der wesentlichen Gründe für die Einführung von Machine Learning im Bereich Business Analytics liegt³. Ein wesentliches Anwendungsgebiet innerhalb des Bereiches ist das Text Mining. Durch unterschiedliche Methodiken wie beispielsweise Datenclustering oder durch das Bilden neuronaler Netze kann eine Identifikation von zusammenhängenden Informationen durch die automatische Erkennung von Mustern erfolgen⁴. Die Grundlage der automatischen Textanalyse sowie für die Erkennung von Kontexten zwischen Daten besteht in einem für den Computer interpretierbaren Datenformat, das Wörter in Beziehung zueinander setzt und mathematisch darstellt. Für die maschinelle Verarbeitung hat sich die Darstellung in Form von Vektoren durchgesetzt⁵.

Bereits im Jahr 2002 zeigt Sebastiani (2002) die Notwendigkeit auf, Wörter im Vektorformat darzustellen, um so Textanalysen mittels Machine Learning zu realisieren⁶. Als Basiskomponente des Natural Language Processing (NLP) bilden Wortvektoren beziehungsweise Word Embeddings die Grundlage für zahlreiche Anwendungen. Maas

¹ Vgl. (Statista, 2019a), Seite 1.

² Vgl. (Statista, 2018), Seite 1.

³ Vgl. (Statista, 2019b), Seite 1.

⁴ Vgl. (Sukanya & Biruntha, 2012), Seite 269-271.

⁵ Vgl. (Levy & Goldberg, 2014), Seite 302.

⁶ Vgl. (Sebastiani & Fabrizio, 2002), Seite 11.

et al. (2011) verwenden die Vektoren zur Analyse von semantischen Verbindungen in Onlinerezensionen⁷. Dabei verwenden sie einen unüberwachten Lernalgorithmus, der auf Deep Learning (DL), einer Teildisziplin des Machine Learning, aufbaut. Den Gedanken, Word Embeddings durch unüberwachte Lernverfahren zu trainieren, griffen Mikolov et al. (2013) im Jahr 2013 auf, indem sie den Ansatz Word2Vec implementierten⁸. Ein Jahr nach der erfolgreichen Entwicklung von Word2Vec präsentierten Pennington et al. (2014) GloVe als weiteren unüberwachten Lernalgorithmus zur Bildung von Word Embeddings⁹. Beide Verfahren erzielen bestmögliche Ergebnisse und haben sich als Standard zur Bildung von Wortvektoren etabliert.

Heute bilden beide Ansätze die Basis für zahlreiche Veröffentlichungen im Bereich der automatischen Textanalyse. Neueste Publikationen wie die Studien von Manna und Nakai (2019) befassen sich beispielsweise mit der Klassifizierung von Twitter-Daten¹⁰.

Es kann zusammengefasst werden, dass gerade aufgrund der hohen zu erwartenden Menge an neuen Daten der Fokus weiter auf automatischer Textverarbeitung liegen wird. Zur Gewährleistung bestmöglicher Analysen ist es daher elementar, dass die Basiskomponenten optimale Resultate liefern, um so eine Ergebnisverzerrung zu minimieren. Die Bedeutsamkeit von Wortvektoren, als Grundlage für zahlreiche Analysen, gilt somit als allgegenwärtig und wird in den kommenden Jahren weiter an Priorität zunehmen. Eine optimale Parametrisierung sowie die Entwicklung einfacher Benutzerschnittstellen ist derzeit für die Empirie und Betriebswirtschaft von großer Wichtigkeit und trifft somit den ‚Nerv der Zeit‘.

1.2 Zielsetzung der Thesis

Das Ziel der vorliegenden Arbeit ist es, verschiedene Word Embedding-Verfahren zu implementieren und diese über einfache Benutzerschnittstellen zur Verfügung zu stellen. Des Weiteren soll der Einfluss verschiedener Hyperparameter hinsichtlich der Prädiktion untersucht werden, um so eine Aussage über eine optimale Parametrisierung treffen zu können. Abschließend sollen die beiden gängigen Ansätze Word2Vec und GloVe

⁷ Vgl. (Maas et al., 2011), Seite 142.

⁸ Vgl. (Mikolov, Sutskever, Chen, Corrado, & Dean, 2013), Seite 3111.

⁹ Vgl. (Pennington et al., 2014), Seite 1532.

¹⁰ Vgl. (Manna & Nakai, 2019), Seite 158.

miteinander verglichen werden, um eine Nutzungsempfehlung für die Analyse von Onlinebewertungen aussprechen zu können.

1.3 Aufbau der Arbeit und Methodik

Die vorliegende Bachelorarbeit ist in mehrere Bestandteile gegliedert. Im ersten Teil der Arbeit werden theoretische Grundlagen erläutert. Diese dienen dem Zweck des tieferen Verständnisses der Erzeugung von Wortvektoren sowie zur theoretischen Einführung in die Gesamthematik.

Aufbauend auf den theoretischen Grundlagen soll durch das Kapitel ‚Verfahren des Word Embedding‘ (Kapitel 3.) ein detaillierter Überblick über zwei Algorithmen für die Erzeugung von Word Vektoren gegeben werden. Innerhalb des Kapitels werden insbesondere für das Training elementare Parameter sowie die Kernarchitekturen der beiden Algorithmen näher betrachtet.

Im Kapitel ‚Methodische Vorgehensweise‘ (Kapitel 4.) werden die für die Erzeugung und Evaluierung von Wortvektoren notwendigen Schritte sowie Implementierungen umfangreich dargestellt. Hier soll insbesondere auf die im Rahmen der vorliegenden Arbeit durchgeführten Programmierungen sowie Verfahren eingegangen werden. Innerhalb des Kapitels sollen sowohl die Forschungsfrage als auch die zu untersuchenden Hypothesen ermittelt werden. Beides, Forschungsfrage sowie Hypothesen, werden anhand bestehender empirischer Fragestellungen sowie Erkenntnissen abgeleitet.

Anschließend wird die Anwendung der ausgewählten Trainingsalgorithmen beschrieben. Darüber hinaus sollen die erzielten Resultate empirisch betrachtet werden, um so die generierten Ergebnisse untersuchen zu können. Zudem erfolgt eine Validierung der zentralen Forschungsfrage sowie der aufgestellten Hypothesen.

Darauffolgend werden die erzielten Ergebnisse mit der gegenwärtigen Forschung und deren Resultaten verglichen sowie die eigene Vorgehensweise kritisch betrachtet. Die Arbeit schließt mit einem Fazit, das auch einen Ausblick auf mögliche weitere Forschungsansätze beinhaltet.

Grundsätzlich wird für diese Thesis eine systematische Literaturanalyse nach Vom Brocke et al. (2015) als Methode der qualitativen Forschung durchgeführt¹¹. Der Fokus

¹¹ Vgl. (Vom Brocke, Simons, Riemer, & Niehaves, 2015), Seite 205-224.

der Recherche liegt dabei auf peer-reviewed Konferenzschriften sowie wissenschaftlichen Fachartikeln.

Zur Untersuchung der Prädiktion von Wortvektoren sollen unterschiedliche Anwendungen implementiert werden. Vor der Umsetzung soll jeweils ein Unified Modelling Language(UML)-Aktivitätsdiagramm nach Stevens und Pooley (2006) erstellt werden¹². Das jeweilige Diagramm dient zur Planung des Programmablaufes sowie als Grundlage einer strukturierten Entwicklung.

Für die Beantwortung der Forschungsfrage sowie der Hypothesen werden Methoden der quantitativen Forschung eingesetzt. Zur Untersuchung der Forschungsfrage soll ein t-Test durchgeführt werden. Zhang et al. (2017) verwenden diese statistische Vergleichsmethodik ebenfalls im Bereich Machine Learning und Word Embedding, um die gestellte Forschungsfrage zu beurteilen¹³. Zur empirischen Bewertung der in der vorliegenden Arbeit aufgestellten Hypothesen, wird jeweils eine Varianzanalyse, auch bekannt als Analysis of Variance (ANOVA), durchgeführt. Pou-Prom und Rudzicz (2018) sowie Fast et al. (2017) benutzen in ihren Forschungen bezüglich Word Embeddings, eine Varianzanalyse, um die Signifikanz erzielter Resultate zu untersuchen und entsprechend zu interpretieren^{14 15}.

Anderson et al. (2017) verwenden in ihren Studien einen Post-Hoc-Test nach Tukey, um Unterschiede zwischen verschiedenen Methodiken aufzuweisen¹⁶. Diese Methodik wird für die vorliegende Arbeit adaptiert, um basierend auf den Daten der ANOVA Unterschiede innerhalb der Testdaten zu analysieren. Sowohl der t-Test, die Varianzanalyse als auch der Post-Hoc Test sollen mit der Programmiersprache R realisiert werden¹⁷.

¹² Vgl. (Stevens & Pooley, 2006), Seite 132-145.

¹³ Vgl. (Yong Zhang, Er, Zhao, & Pratama, 2017), Seite 3228.

¹⁴ Vgl. (Pou-Prom & Rudzicz, 2018), Seite 2813 ff.

¹⁵ Vgl. (Fast, Chen, & Bernstein, 2017), Seite 4838.

¹⁶ Vgl. (Anderson, Kiela, Clark, & Poesio, 2017), Seite 24.

¹⁷ Vgl. (Verzani, 2014), Seite 1-494.

2. Theoretische Grundlagen

2.1 Machine Learning

Durch die fortschreitende Digitalisierung können immer mehr Datensätze in großen Mengen erzeugt und gespeichert werden. So werden beispielsweise auf einer Onlineverkaufsplattform große Mengen an Rezensionen durch Kunden erzeugt und im Anschluss massenhaft gespeichert. Die Rezensionen können wichtige Informationen über die Zufriedenheit der Kunden zum jeweiligen bewerteten Artikel enthalten. Durch gezielte Datenanalysen kann ein großer Informationsgewinn erzielt werden, was unter anderem dabei hilft, die Qualität einzelner Produkte nachhaltig zu verbessern. Um die Analyse von Massendaten effizient zu gestalten, kann maschinelles Lernen (ML) eingesetzt werden¹⁸.

In der Empirie umfasst die Begrifflichkeit ML verschiedene Bedeutungen und findet Anwendung in unterschiedlichen Bereichen, so zum Beispiel in der Medizin, Informatik, wie beispielsweise in optischen Netzwerken, in der Biologie, Werbeindustrie oder auch im Wertpapierhandel¹⁹.

Eine erste Definition des Begriffs gab Arthur Samuel bereits im Jahr 1959 mit „Machine Learning: Field of study that gives computers the ability to learn without being explicitly programmed“²⁰. Samuel definierte ML als eine Methodik, mittels derer Computer in der Lage sind, eigenständig und ohne explizite Programmierung zu lernen. Aufbauend auf der Definition von Samuel konnte Mitchell im Jahr 1997 mit der Frage, wie ein Computerprogramm zu erstellen ist, das eigenständig Probleme lösen und daraus lernen kann, eine zentrale Problemstellung innerhalb des maschinellen Lernens beschreiben²¹. Basierend auf dieser Fragestellung konnte Mitchell ML wie folgt definieren: „A computer program is said to learn from experience E with respect to some task T and some performance measure P, if its performance on T, as measured by P, improves with experience E“²². Um die Definition etwas zu vereinfachen soll diese an dem nachfolgenden selbstgewählten Beispiel erläutert werden: Ein entwickeltes System hat die Aufgabe, T Wörter aus Kundenrezensionen einer Onlineverkaufsplattform gemäß

¹⁸ Vgl. (Sze, Chen, Emer, Suleiman, & Zhang, 2017), Seite 1.

¹⁹ Vgl. (Musumeci et al., 2018), Seite 2.

²⁰ (Samuel, 1959), Seite 211.

²¹ Vgl. (T. M. Mitchell, 1997), Seite 2-3.

²² (T. M. Mitchell, 1997), Seite 2.

vergebener Sterne zu kategorisieren. Die Leistungskennzahl P ist die Anzahl der Wörter in den jeweiligen Kategorien (1-3 Sterne = negativ; 4-5 Sterne = positiv). Das System erlernt anhand der Leistungskennzahl P , ob das jeweilige Wort negativ oder positiv zu klassifizieren ist. Die Erfahrung E baut sich dabei aus den bereits verarbeiteten Kundenrezessionen auf. Sobald eine Kundenrezession verarbeitet ist und die Wörter entsprechend klassifiziert sind, ist bekannt, ob das jeweilige Wort positiv oder negativ ist. Die Erkenntnis wird an das System als Erfahrung weitergegeben und ermöglicht so einen Lerneffekt.

Weitere Definitionen beschreiben ML als Verfahren, um Maschinen so zu trainieren, dass große Mengen an Daten so effizient wie möglich behandelt werden²³. Die Algorithmen sollen dabei so implementiert werden, dass sowohl ein kontinuierliches Lernen ermöglicht wird als auch Vorhersagen getroffen werden können²⁴. Auf Basis dieser Vorhersagen können dann beispielsweise Informationen oder unentdeckte Muster in den Datenbeständen entdeckt werden²⁵.

Grundsätzlich kann ML mit unterschiedlichen Algorithmen praktiziert werden. In der Literatur finden sich insbesondere die Verfahren Supervised Learning, Unsupervised Learning und Reinforcement Learning²⁶. Nachfolgend sollen die verschiedenen Ansätze des maschinellen Lernens näher betrachtet werden:

Supervised Learning

Supervised Learning, beziehungsweise im Deutschen ‚überwachtes Lernen‘, bezeichnet eine Methodik innerhalb des maschinellen Lernens, die auf Basis von eingegebenen Daten eine Vorhersage treffen soll²⁷. Um eine Vorhersage treffen zu können, benötigen die Algorithmen präparierte Daten, die dann für eine Anlernphase verwendet werden sollen. Diese für die Anlernphase präparierten Trainingsdaten bestehen grundsätzlich aus einem Datenpaar. Dieses setzt sich aus Ein- sowie Ausgangsdaten zusammen, wobei die Ausgangsdaten zuvor markiert (labelled) werden²⁸, um die Ergebnisse bereits vorher

²³ Vgl. (Dey & Learning, 2016), Seite 1174.

²⁴ Vgl. (Xue & Zhu, 2009), Seite 272-274.

²⁵ Vgl. (Kim, Hong, & Han, 2018), Seite 127.

²⁶ Vgl. (Hua, 2008), Seite 192.

²⁷ Vgl. (Jordan & Mitchell, 2015), Seite 257.

²⁸ Vgl. (LeCun, Bengio, & Hinton, 2015), Seite 436.

aufzuzeigen. Ein Trainingsdatensatz beinhaltet somit bereits die erwarteten Ergebnisse. Mittels des supervised Learning Algorithmus wird versucht, letztendlich die Korrektheit der Daten auf Basis der bereits vorhandenen Ergebnisse sicherzustellen. Da die Struktur der Zieldaten bekannt ist, kann die Maschine die Ergebnisse nach dem Training auf ihre Richtigkeit überprüfen und ist dementsprechend in der Lage die Qualität des Zieldatensatzes sicherzustellen.

Grundsätzlich ist die Anlernung eines auf supervised Learning basierenden Modells sehr zeitintensiv. Um die Korrektheit der Zieldaten zu gewährleisten, ist es darüber hinaus erforderlich, dass die Eingabedatensätze eine ausbalancierte Anzahl von positiven sowie negativen Datenpaaren beinhalten. Des Weiteren ist dies eine Voraussetzung, um die Trainingsperformance des supervised Learning zu garantieren²⁹. Sofern der Algorithmus einem erfolgreichen Training unterzogen wurde, kann dieser für die Analyse neuer sowie unbekannter Datenbestände angewandt werden. Die Datenbestände können bei einem korrekt angelernten Algorithmus sinngemäß klassifiziert werden.

Unsupervised Learning

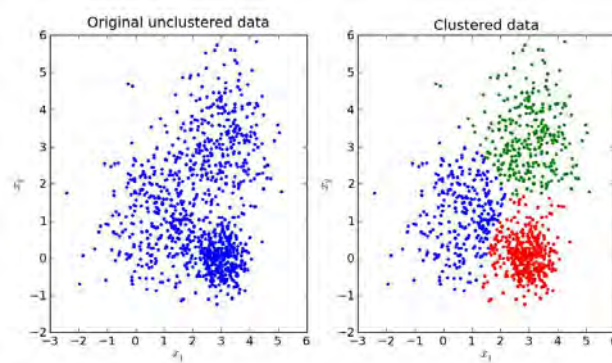
Beim Ansatz des unsupervised Learning, zu Deutsch des ‚unüberwachten Lernens‘, stehen dem Algorithmus bei der Verarbeitung nur die Eingabedaten zur Verfügung. Es ist somit im Vorfeld kein erwartetes Ergebnis definiert. Als Ziel des unüberwachten Lernens wird das maschinelle Erkennen von Mustern und Strukturen in den Daten deklariert³⁰. Basierend auf den in den Eingabedaten enthaltenen Strukturen sowie erkannten Strukturen nimmt die Maschine ein Clustering vor, das dem Bereich der Dichteschätzung innerhalb der Statistik zugeordnet werden kann³¹. Eine Clusterung (Gruppierung) der Daten beziehungsweise Objekte, wie in der nachfolgenden Abbildung dargestellt, kann beispielsweise anhand ähnlicher Zugehörigkeit durchgeführt werden.

²⁹ Vgl. (Parveen & Thuraisingham, 2012), Seite 141.

³⁰ Vgl. (Berral-Garcia, 2016), Seite 2.

³¹ Vgl. (H. Wang, 2009), Seite 10.

Abbildung 1: Datenclustering im Machine Learning



Quelle: (Kurama, 2018), Seite 1

So ist es möglich, dass beispielsweise für Textbestandteile, wie Adjektive oder Nomen, eine kontextbezogene Zugehörigkeit festgestellt werden kann. Die Objekte sind in solchen Fällen ähnlicher Art und es wird eine weniger distanzierte Beziehung innerhalb einer Gruppe erkannt als mit weiteren Objekten³².

Im Gegensatz zum supervised Learning findet beim unsupervised Learning keine Datenklassifizierung statt. Die Daten werden somit nicht beschriftet, sprich eine Markierung mit Data-Labels wird nicht durchgeführt³³. Aufgrund der nicht markierten Daten hat der Algorithmus keine Entscheidungsgrundlage, auf der eine Überprüfung der Ergebnisse vollzogen werden kann.

Oftmals findet der Algorithmus des unsupervised Learning bei der Verarbeitung sowie Analyse von großen Datenbeständen Anwendung³⁴. Die Methodik eignet sich insbesondere für den Bereich Data Mining und verfolgt dabei das Ziel, Beziehungen sowie Abhängigkeiten in den Massendaten zu entdecken und diese gezielt zusammenzufassen³⁵.

Reinforcement Learning

Das Reinforcement Learning ist ein Algorithmus innerhalb des ML, der durch eigenes Verhalten lernt³⁶, der dabei einen ‚Trial-and-Error‘-Ansatz verfolgt. Durch diese

³² Vgl. (Parveen & Thuraisingham, 2012), Seite 142.

³³ Vgl. (Ertel, 2013), Seite 230.

³⁴ Vgl. (Hsu, Zhang, & Glass, 2017), Seite 1.

³⁵ Vgl. (Görz, Schneeberger, & Schmid, 2014), Seite 407.

³⁶ Vgl. (C. Chen, Takahashi, Nakagawa, Inoue, & Kusumi, 2015), Seite 248.

Methodik ist eine dynamische Interaktion des Algorithmus mit der Umwelt möglich³⁷. Ein praxisnahes Beispiel für die dynamische Interaktion mit der Umwelt könnten Park-beziehungsweise Notbremsensensoren im Auto sein, die als externe Sensoren Daten abgeben und für das autonome Fahren eine entscheidende Rolle spielen.

2.2 Deep Learning und neuronale Netze

Computer haben bewiesen, dass sie logische Aufgaben, wie beispielsweise die Multiplikation von komplexen Zahlen oder auch Schachspielen, effizienter bewältigen können als der Mensch. In der Regel sind dies Aufgaben, bei denen eine klare, logische Anweisung durchgeführt werden kann. Ist die Logik einmal implementiert, so kann der Computer diese anwenden. Nichtsdestotrotz ist es schwierig, Aufgaben, die für den Menschen einfach erscheinen, wie beispielsweise Sehen oder Hören, mit dem Computer zu bewältigen. Um die Komplexität der Bearbeitung solcher Aufgaben zu reduzieren, soll Deep Learning eingesetzt werden, das in Zusammenarbeit mit neuronalen Netzen menschliches Denken nachempfinden soll³⁸.

Deep Learning oder auch tiefergehendes Lernen bezeichnet eine Unterkategorie des maschinellen Lernens, bei der der Lernansatz mithilfe von neuronalen Netzwerken realisiert werden soll³⁹. Bereits im Jahr 1965 wurden von Ivakhnenko und Lapa Algorithmen entwickelt, die den Ansatz des DL verfolgten⁴⁰. Obgleich der stetigen Weiterentwicklung von DL konnte das Verfahren zunächst, aufgrund der benötigten Systemressourcen und der damit verbundenen Applikationsgeschwindigkeit⁴¹, keinen Einsatz finden. Da die zur Verfügung stehende Rechenleistung mittlerweile mächtig genug ist, um DL mit ausreichender Anwendungsgeschwindigkeit zu praktizieren⁴², findet sich mit DL ein Ansatz, der sich als besonders effektiv für die Verarbeitung großer Datenmengen erweist⁴³. DL hat umfangreiche Einsatzgebiete, so beispielsweise in der Mustererkennung, Signalverarbeitung oder auch in der Text- und Bildanalyse⁴⁴. Für

³⁷ Vgl. (Sause, 2013), Seite 227.

³⁸ Vgl. (Nishani & Cico, 2017), Seite 1.

³⁹ Vgl. (Schmidhuber, 2015), Seite 4.

⁴⁰ Vgl. (Schmidhuber, 2015), Seite 10.

⁴¹ Vgl. (Rampasek & Goldenberg, 2016), Seite 12.

⁴² Vgl. (Dube & Sura, 2018), Seite 125.

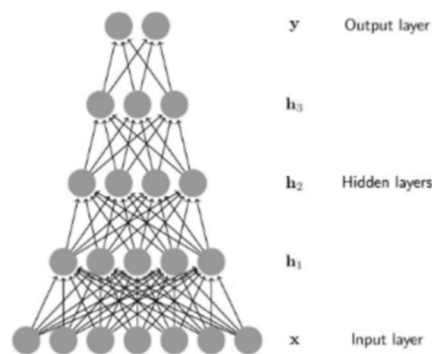
⁴³ Vgl. (M. Li, Yu, Nian, & Li, 2015), Seite 294.

⁴⁴ Vgl. (Wan et al., 2014), Seite 158.

möglichst effiziente Ergebnisse greift Deep Learning auf die Verwendung von neuronalen Netzen zurück⁴⁵.

Ein neuronales Netzwerk besteht grundsätzlich aus mehreren Neuronen, die beliebig untereinander verknüpft werden können. Grundsätzlich kann ein Neuron akzidentell viele Ein- und Ausgaben haben. Jede Verknüpfung zwischen einzelnen Neuronen enthält individuelle Gewichtungen, um die Intensität des jeweiligen Informationsflusses zu definieren. Wie in der unten aufgeführten Abbildung zu erkennen ist, unterteilt sich ein neuronales Netzwerk in verschiedene Schichten: in die Eingangsschicht (Input Layer), die dazu verwendet wird, um die Ursprungsdaten zu verarbeiten, und die Ausgangsschicht (Output Layer), die das Ergebnis zurückliefern soll⁴⁶. Zwischen der Eingangsschicht und der Ausgangsschicht können beliebig viele versteckte Schichten (Hidden Layer)⁴⁷ angelegt werden. Beim DL kommen in der Regel mehrere versteckte Zwischenschichten zum Einsatz. Die Anzahl der verborgenen Schichten bestimmt die tiefe des neuronalen Netzes⁴⁸.

Abbildung 2: Aufbau eines neuronalen Netzes



Quelle: (Min, Lee, & Yoon, 2016), Seite 855

Damit Aufgaben, wie beispielsweise das Erkennen von zusammengehörenden Wörtern in großen Datenbeständen erfolgreich, durch ein neuronales Netz bearbeitet werden können, ist es zunächst notwendig das Netzwerk zu trainieren⁴⁹.

⁴⁵ Vgl. (Setiowati, Zulfanahri, Franita, & Ardiyanto, 2017), Seite 4.

⁴⁶ Vgl. (Svozil, Kvasnieka, & Pospichal, 1997), Seite 45.

⁴⁷ Vgl. (Mu, Zeng, & Han, 2018), Seite 3.

⁴⁸ Vgl. (Kruse et al., 2015), Seite 38.

⁴⁹ Vgl. (Takhavova & Kosjukhina, 2018), Seite 2.

Das Training eines neuronalen Netzwerks kann sich vielseitig gestalten. So können grundsätzlich alle Bestandteile des Netzes angepasst werden. Beispielsweise ist es denkbar, dass Verbindungen zwischen Neuronen gelöscht beziehungsweise hinzugefügt werden oder dass Neuronen aus dem Netzwerk ausgeschlossen beziehungsweise diesem hinzugefügt werden. In der Regel werden für das Training die Gewichte der einzelnen Neuronen verändert^{50 51}.

Grundsätzlich können für das Training von neuronalen Netzen sowohl Algorithmen des unsupervised Learning als auch des supervised Learning verwendet werden. Bei letzterem wird das Netzwerk anhand von gegebenen Lerndaten trainiert, wodurch jeweils Datenpaare aus bereits bekannten Ergebnissen sowie neuen Daten entstehen. Zunächst werden die neuen Daten in das neuronale Netz integriert und im Anschluss berechnet. Nach erfolgreicher Berechnung der Daten, können diese mit den vorhandenen Ergebnissen verglichen werden. Die Differenz der Resultate kann durch eine Verlustfunktion ermittelt werden⁵². Ziel des Trainings ist es, durch die Optimierung der Gewichtung der Neuronen die Verlustfunktion für alle neuen beziehungsweise gegebenen Daten zu minimieren. Die Berechnung der neuen Gewichtungen kann beispielsweise über statistische Gradientenverfahren minimiert werden⁵³.

Wenn Methoden des unsupervised Learning verwendet werden, sind zunächst keine Ergebnisse bekannt, sondern nur die eingegebenen Daten. Ziel des Algorithmus ist es, basierend auf den Eingabedaten möglichst nahe Ausgabedaten zu ermitteln⁵⁴.

Die Methoden des Deep Learnings haben den Vorteil, dass vorab keine detaillierten Informationen über die zu verarbeitenden Daten bekannt sein müssen und somit der Algorithmus dynamisch konfiguriert werden kann, sodass mit Rohdaten gearbeitet werden kann. Da es möglich ist, den Algorithmus dynamisch zu konfigurieren, ist es nicht notwendig, diesen für jedes neue Anwendungsszenario neu zu gestalten⁵⁵. Zudem ist DL

⁵⁰ Vgl. (LeCun et al., 2015), Seite 436.

⁵¹ Vgl. (Chilimbi, Suzue, Apacible, & Kalyanaraman, 2014), Seite 573.

⁵² Vgl. (LeCun et al., 2015), Seite 436.

⁵³ Vgl. (Chilimbi et al., 2014), Seite 573.

⁵⁴ Vgl. (Kruse et al., 2015), Seite 45.

⁵⁵ Vgl. (LeCun et al., 2015), Seite 436.

in der Lage, große Mengen an Trainingsdaten effizienter zu verarbeiten als traditionelle Machine Learning-Verfahren⁵⁶.

2.3 Natural Language Processing und Word Embedding

NLP gilt als Oberbegriff für die Verarbeitung natürlicher Sprache durch Maschinen und wird innerhalb der Empirie als interdisziplinäres Feld gesehen, das die Forschungsgebiete Computer Linguistik, Informatik, Kognitionswissenschaft sowie künstliche Intelligenz kombiniert⁵⁷. Als zentrale Komponente innerhalb von NLP gilt es, die Ansätze der Linguistik mit Verfahren der Informatik zu verbinden. Innerhalb der Linguistik wird die Anwendung natürlicher Sprache grundsätzlich durch beispielsweise den anerkannten Zeichenraum, die Syntaktik und die Semantik definiert. In der Informatik werden diese Definitionen verwendet und zusätzlich auf beispielsweise statistische Datenerhebungen, bezogen auf die Häufigkeit der Verwendung einzelner Wörter, zurückgegriffen. Basierend auf den gewonnenen Erkenntnissen werden in der Informatik stochastische Algorithmen sowie Programmlogiken implementiert, die es ermöglichen natürliche Sprache maschinell verarbeiten zu können⁵⁸. Durch den Einsatz von intelligenten und selbstlernenden Systemen existieren zunehmend Ansätze innerhalb des Forschungsgebietes der künstlichen Intelligenz, die in der Lage sind natürliche Sprache kognitiv zu interpretieren⁵⁹.

NLP hat eine Vielzahl von Anwendungsformen, die nicht neu sind. Bereits im Jahr 1994 zählte Manaris (1994)⁶⁰ unterschiedliche Anwendungen wie beispielsweise maschinelle Übersetzungssysteme, Textgenerierungssoftware, Sprachinterpretationssysteme oder auch Textverarbeitungslogiken auf.

Eine Anwendungsform der automatisierten Textverarbeitung innerhalb von NLP findet sich im Word Embedding (WE). Mit dem Begriff WE wird in der Empirie ein Verfahren bezeichnet, das die Bedeutung von Wörtern vektoriell darstellt. WE nutzt eine, aus verschiedenen Wissenschaftsbereichen stammende Kombination von NLP-

⁵⁶ Vgl. (Paterakis, Mocanu, Gibescu, Stappers, & van Alst, 2017), Seite 5.

⁵⁷ Vgl. (Rodriguez, Carver, & Mahmoud, 2018), Seite 2.

⁵⁸ Vgl. (Y. Wang & Berwick, 2012), Seite 1 f.

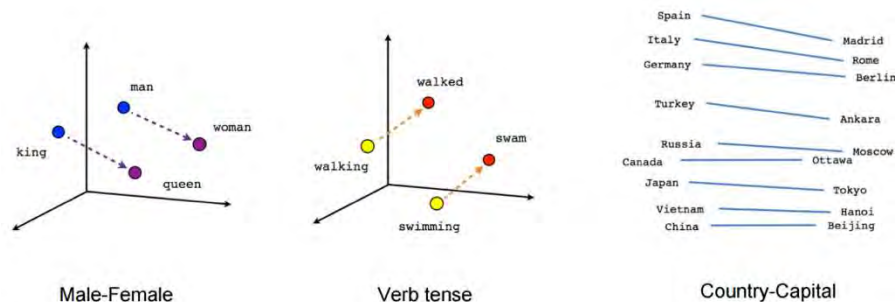
⁵⁹ Vgl. (Fahad & Yahya, 2018), Seite 1.

⁶⁰ Vgl. (Manaris, 1994), Seite 228.

Methodiken⁶¹. Es werden beispielsweise aus dem Bereich der Computerlinguistik die Verteilungssemantik und die damit verbundenen verteilungssemantischen Modelle verwendet⁶². Allgemein ist die Annahme innerhalb der verteilungssemantischen Modelle, dass der Kontext eines Wortes anhand der Verwendung in einem Text abgeleitet werden kann⁶³. Des Weiteren wird davon ausgegangen, dass Wörter, die in einem ähnlichen Kontext gebraucht werden, semantisch ähnlich sind⁶⁴. Um die Wörter und deren Kontext zusammenzuführen, werden durch statistische Analysen semantische Repräsentationsmodelle in Form von hochdimensionalen Vektorräumen erzeugt⁶⁵.

Innerhalb dieser Modelle können dann beispielsweise einfache Abstandsrechnungen durchgeführt werden. In der nachfolgenden Abbildung sind die Wörter ‚king‘, ‚man‘, ‚women‘ und ‚queen‘ nah beieinander.

Abbildung 3: semantische Beziehungen von Wörtern durch Word Embedding



Quelle: ("TensorFlow," 2019), Seite 1

Ein WE liefert zu jedem dieser Wörter einen Vektor in Fließkommazahl und einer festen Länge. Jeder dieser Vektoren enthält die vollumfängliche Semantik des jeweiligen Wortes. Somit können basierend auf vorhandenen Wörtern semantisch passende Wörter mathematisch ermittelt und anschließend ergänzt werden⁶⁶.

Für die Ermittlung semantisch ähnlicher Wörter kann beispielsweise folgende mathematische Formel verwendet werden:

⁶¹ Vgl. (Lai, Liu, He, & Zhao, 2016), Seite 5.

⁶² Vgl. (Bruni, Uijlings, Baroni, & Sebe, 2012), Seite 1.

⁶³ Vgl. (J. Mitchell & Lapata, 2010), Seite 1389.

⁶⁴ Vgl. (Sahlgren, 2008), Seite 1.

⁶⁵ Vgl. (J. Mitchell & Lapata, 2010), Seite 1389.

⁶⁶ Vgl. (Mikolov, Yih, et al., 2013), Seite 748.

Formel 1: Distanzberechnung zur Ermittlung ähnlicher Wörter

$$y = x_b - x_a + x_c$$

Quelle: (Mikolov, Yih, & Zweig, 2013), Seite 748

Angenommen, die Wörter ‚king‘, ‚man‘ und ‚women‘ sind vorhanden. Wird nun vom Vektor des Wortes ‚king‘ der Vektor des Wortes ‚man‘ abgezogen und der Vektor von ‚women‘ aufaddiert, resultiert daraus das Wort ‚queen‘, da es semantisch am besten zu den gegebenen Worten passt. Die Methodik hinter dem aufgeführten Beispiel kann ebenso auf andere Fragestellungen, wie beispielsweise in der zuvor aufgeführten Abbildung dargestellt, angewandt werden.

Word Embeddings werden grundsätzlich auf der Basis von Textkörpern erzeugt, die aus variabel gewählten Texten bestehen können⁶⁷. Für die Erzeugung von WE können verschiedene Methodiken beziehungsweise Algorithmen angewandt werden. Grundlegend kann zwischen counting-based und prediction-based Ansätzen unterschieden werden⁶⁸.

Bei counting-based Ansätzen wie beispielsweise One-Hot oder 1-of-n wird eine Vektormatrix aufgestellt, in der die eindeutige Kennung eines Wortes durch eine 1 und alle anderen Wörter im Textkorpus durch eine 0 markiert werden⁶⁹. Das folgende Beispiel soll die Methodik näher erläutern. Angenommen, es ist folgender Vektor V gegeben: V = { Have, a, good, day }. Der Vektor V hat vier Dimensionen. Nach Anwendung eines counting-based Verfahrens würden die Wörter folgendermaßen als Vektor wiedergegeben werden können: Have = [1, 0, 0, 0]; a = [0, 1, 0, 0]; good = [0, 0, 1, 0]; day = [0, 0, 0, 1]. Wie im Beispiel zu erkennen, entsprechen die Dimensionen solcher Vektormatrizen dem jeweiligen Vokabular⁷⁰, was zu Nachteilen in der Verarbeitungsgeschwindigkeit sowie in deren Weiterverwendbarkeit innerhalb von NLP-Anwendungen führen kann.

⁶⁷ Vgl. (Z. Wang, Zhang, Feng, & Chen, 2014), Seite 1592.

⁶⁸ Vgl. (Baroni, Dinu, & Kruszewski, 2014), Seite 239 f.

⁶⁹ Vgl. (Goldberg, 2016), Seite 349 ff.

⁷⁰ Vgl. (Goldberg, 2016), Seite 349 ff.

Prediction-based Ansätze verfolgen eine andere Algorithmik. Die Verwendung von neuronalen Netzen ermöglicht es, dass einzelne Wortbestandteile eines eingegebenen Textes klassifiziert werden können. Dieses Verfahren wird unter anderem auch als neuronales Word-Embedding bezeichnet. Funktional werden die Wörter und der jeweilige Kontext in einen niederdimensionalen Raum, innerhalb eines neuronalen Netzes, eingebettet, wobei jedes Netzwerk die Vorteile verdichteter Kontextinformationen sowie effizienter Berechenbarkeit erbt⁷¹. Durch die Verwendung neuronaler Netze ist es möglich, große Textkörper in kurzer Zeit effizient zu berechnen.

WE findet derzeit Anwendung in unterschiedlichsten NLP-Anwendungen. Populäre Ansätze finden sich in den Implementierungen Word2Vec(W2V) von Mikolov et al. (2013)⁷² sowie GloVe von Pennington et al. (2014)⁷³, die nachfolgend näher erläutert sowie prototypisiert werden.

⁷¹ Vgl. (Mikolov, Yih, et al., 2013), Seite 746.

⁷² Vgl. (Mikolov, Sutskever, et al., 2013)

⁷³ Vgl. (Pennington et al., 2014)

3. Verfahren des Word Embedding

3.1 Word2Vec

3.1.1 Definition und Parameter

Das von Mikolov et al. (2013)⁷⁴ entwickelte Verfahren W2V bezeichnet einen WE-Ansatz, der Wörter als Vektoren darstellt⁷⁵. W2V verwendet ein neuronales Netzwerk und kann zu den Verfahren des neuronalen Word Embedding gezählt werden⁷⁶. Das Verfahren findet Anwendung in vielen unterschiedlichen Bereichen. So beispielsweise in der Spracherkennung, der Biologie oder auch in der Textklassifikation mit NLP^{77 78}. Insbesondere innerhalb der automatisierten Textklassifikation weist W2V schnelle Ausführzeiten sowie hohe Accuracy Raten auf⁷⁹, was den Algorithmus gegenwärtig zu einem populären Ansatz des WE macht.

Ziel von W2V ist es Wörter, die im gleichen Kontext stehen, als Vektoren möglichst ähnlich darzustellen, sodass eine Relation zwischen den jeweiligen Wörtern erkennbar wird⁸⁰. Zur Zielerreichung erzeugt der Ansatz einen N-Dimensionalen Vektorraum, wobei jedem einzelnen Wort ein eigener Vektor zugewiesen wird. Während der Zuweisung werden die einzelnen Wortvektoren im Vektorraum so positioniert⁸¹, dass semantische Gemeinsamkeiten durch Rechenoperationen, wie beispielsweise der Berechnung Ähnlichkeit des Kosinus, erkannt werden können⁸². Das Ergebnis der Kosinus-Ähnlichkeit kann zwischen minus eins und plus eins liegen. Je näher das Ergebnis an der positiven Eins liegt, desto größer ist die semantische Ähnlichkeit^{83 84}. Eine hundertprozentige Ähnlichkeit, sprich eine positive Eins, kann nur erreicht werden, wenn die Kosinus-Ähnlichkeit eines Wortes zu sich selbst berechnet wird.

Grundsätzlich kann für die Durchführung des WE zwischen zwei verschiedenen Architekturen neuronaler Netze, Skip-Gram (SG) und Continuous Bag of Words

⁷⁴ Vgl. (Mikolov, Chen, Corrado, & Dean, 2013)

⁷⁵ Vgl. (Al-Amin, Islam, & Das Uzzal, 2017), Seite 1.

⁷⁶ Vgl. (Ma & Zhang, 2015), Seite 2895.

⁷⁷ Vgl. (Ge & Moh, 2017), Seite 1796.

⁷⁸ Vgl. (Tian, Li, & Li, 2018), Seite 9452.

⁷⁹ Vgl. (Lilleberg, Zhu, & Zhang, 2015), Seite 139.

⁸⁰ Vgl. (Xiao & Shi, 2015), Seite 172.

⁸¹ Vgl. (Grawe, Martins, & Bonfante, 2017), Seite 409.

⁸² Vgl. (Jin, Zhang, & Liu, 2018), Seite 13.

⁸³ Vgl. (Al-Amin et al., 2017), Seite 187.

⁸⁴ Vgl. (C. Zhang, Wang, Yu, & Wang, 2018), Seite 341.

(CBOW), gewählt werden. Für das Training beider Ansätze stehen ebenso unterschiedliche Lernalgorithmen, Negative Sampling (NS) und Hierarchical Softmax (HS), zur Verfügung. Die verschiedenen Architekturen und deren Lernalgorithmen werden in den folgenden Kapiteln näher erläutert. Für das Training der Wort Vektoren ist es erforderlich verschiedene Parameter einzustellen. Eine Auswahl der elementaren Parameter für W2V wird folgend kurz vorgestellt:

Corpus File

Der Parameter Corpus File definiert den Dateipfad zu den Trainingsdaten beziehungsweise zu dem für das Training angedachten Textkorpus. Ein Textkorpus besteht in der Regel aus einer großen Ansammlung von Texten. Damit die Wortvektoren so präzise wie möglich erzeugt werden, bedarf es eines großen Textkörpers⁸⁵.

Size

In wie vielen vektoriellen Dimensionen ein einzelnes Wort abgebildet werden soll, wird über den Parameter Size definiert. Oftmals wird in verschiedenen mathematischen Formeln n als Ausdruck für die Anzahl der Dimensionen verwendet. Bei der Implementierung von W2V wird n durch den Parameter Size dargestellt. Grundsätzlich sollte die Anzahl der Dimensionen eines Wortes relational zum Textkörper gewählt werden, um einem möglichen Informationsverlust aufgrund von nicht abbildbaren Relationen zwischen Wörtern vorbeugen zu können. In der Empirie wird für kleine Textkörper oftmals ein Wert zwischen hundert Dimensionen, hingegen werden dreihundert bis eintausend Dimensionen für große Textkörper gewählt^{86 87 88 89}.

Min Count

Durch Setzen des Parameters Min Count kann die mindestens vorhandene Frequenz eines einzelnen Wortes innerhalb eines Textkörpers definiert werden. Grundsätzlich soll durch die Algorithmen von W2V für jedes Wort ein eigener Wortvektor erstellt und im Anschluss im vektoriellen Raum platziert werden. Nichtsdestotrotz ist es, insbesondere

⁸⁵ Vgl. (Alshari, Azman, Doraisamy, Mustapha, & Alkeshr, 2017), Seite 123 f.

⁸⁶ Vgl. (Ge & Moh, 2017), Seite 1798.

⁸⁷ Vgl. (Xinrui Zhang et al., 2016), Seite 305.

⁸⁸ Vgl. (Yuan, Wu, Li, & Wang, 2018), Seite 4.

⁸⁹ Vgl. (Simonton & Alaghband, 2017), Seite 1.

bei umfangreichen Textkörpern, populär, die Frequenz eines Wortes zu bestimmen, um innerhalb des WE nur Wörter zu verwenden, die eine ausreichende Präsenz im Text enthalten, um semantisch wertvoll zu sein. Je nach Umfang der Trainingsdaten, kann dieser Wert entsprechend variieren⁹⁰.

Workers

Der Parameter Workers bestimmt die Anzahl der Threads, die für das Training des neuronalen Netzwerkes verwendet werden sollen. Durch die Verwendung dieses Parameters ist es möglich, die Verarbeitung von W2V mittels unterschiedlicher Threads zu parallelisieren⁹¹. Die hiermit verbundene Applikationsgeschwindigkeit ermöglicht es große Datenmengen in kalkulierbarer Zeit zu verarbeiten.

Windows

Windows definiert die maximale Fenstergröße, mit der benachbarte Wörter zur Berechnung eines einzelnen Wortvektors betrachtet werden sollen. Das zu betrachtende Fenster geht jeweils von dem fokussierten Wort aus und analysiert grundsätzlich den ganzen Textkörper, um die semantische Ähnlichkeit des Wortes zu anderen zu evaluieren⁹².

Iter

Mit dem Parameter Iter wird die Anzahl der Trainingsiterationen angegeben. Mit jeder Iteration versucht der Algorithmus die Verlustfunktion zu minimieren, um somit die Genauigkeit der Vektoren zu optimieren. Eine zu hohe Anzahl von Iterationen kann zu einem Overfitting führen, sprich der Algorithmus ist zu optimal auf die Trainingsdaten angepasst, eine zu niedrige Anzahl kann jedoch ein nicht ausreichendes Training verursachen⁹³. Des Weiteren steigt mit jeder Iteration die Trainingsdauer an, sodass in der

⁹⁰ Vgl. (Sienčnik, 2015), Seite 240.

⁹¹ Vgl. (Simonton & Alaghband, 2017), Seite 2.

⁹² Vgl. (Islam, Amin, & Das Uzzal, 2016), Seite 364.

⁹³ Vgl. (Aghahadi & Talebpour, 2018), Seite 305.

Empirie zumeist Werte zwischen 5 und 100 Iterationen als bestmöglich erachtet werden⁹⁴

⁹⁵ ⁹⁶ ⁹⁷ .

Sg

Der Parameter Sg definiert, ob Skip-Gram Sg=1 (wobei: 1 = true) beziehungsweise CBOW Sg=0 (wobei: 0 = false) als neuronale Architektur verwendet werden soll.

Negative

Der Parameter ist nur relevant, sofern NS als Lernalgorithmus gewählt wird. Negative Sampling wird im Nachgang genauer spezifiziert.

Hs

Mit dem Parameter Hs wird angegeben, ob HS Hs=1 (wobei: 1 = true) als Lernalgorithmus beziehungsweise NS Hs=0 (wobei: 0 = false) Negative $\hookrightarrow$ 0 verwendet werden soll.

Alpha

Alpha beschreibt die zu verwendende Learning Rate (LR). Mit der LR wird festgelegt, wie groß die jeweiligen Einzelschritte zur Optimierung der Verlustfunktion sein sollen. In der Literatur wird oftmals eine LR zwischen 0.01 und 0.1 verwendet, um bestmögliche Resultate zu erzielen⁹⁸ ⁹⁹ ¹⁰⁰ ¹⁰¹.

3.1.2 Neuronale Architekturen

Im Rahmen der Berechnung von Wortvektoren kann innerhalb des W2V-Ansatzes zwischen zwei Architekturen neuronaler Netze gewählt werden. Dabei wird zwischen folgenden architektonischen Ansätzen unterschieden:

CBOW

⁹⁴ Vgl. (Putra & Khodra, 2016), Seite 4.

⁹⁵ Vgl. (Simonton & Alaghband, 2017), Seite 5.

⁹⁶ Vgl. (Aghahadi & Talebpour, 2018), Seite 304.

⁹⁷ Vgl. (Lochter, Pires, Bossolani, Yamakami, & Almeida, 2018), Seite 4.

⁹⁸ Vgl. (Putra & Khodra, 2016), Seite 4.

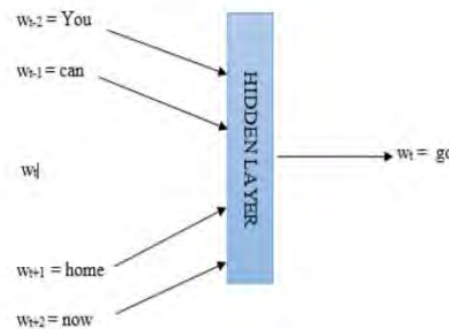
⁹⁹ Vgl. (Fu, Ch'ng, Aickelin, & See, 2017), Seite 3.

¹⁰⁰ Vgl. (Chang, Lee, & Lai, 2017), Seite 566.

¹⁰¹ Vgl. (Shen, Sung, & Lee, 2018a), Seite 2233.

CBOW ist eine neuronale Architektur, die insbesondere in Kombination mit W2V eingesetzt wird, um semantische Relationen zwischen unterschiedlichen Wörtern zu ermitteln. Im Jahr 2013 von Mikolov et al.¹⁰² veröffentlicht, beschreibt der Ansatz ein klassisches neuronales Netz mit drei Schichten: Input-Layer, Projection-Layer sowie Output-Layer¹⁰³. Wie in der nachfolgenden Abbildung illustriert, ist es das Ziel des Ansatzes, Wörter anhand des gegebenen Kontextes vorherzusagen¹⁰⁴. Beispielsweise ist der Algorithmus in der Lage, auf Basis der Wörter ‚You‘, ‚can‘, ‚home‘, ‚now‘ das fehlende Wort ‚go‘ zu ermitteln. Aufgrund der erhöhten Performance bei der Optimierung von Neuronen¹⁰⁵ eignet sich CBOW insbesondere für die performante Verarbeitung von großen Textkörpern¹⁰⁶.

Abbildung 4: Aufbau der CBOW-Architektur



Quelle: (Chaitanya et al., 2018), Seite 1139

Skip-Gram

Das von Mikolov et al. (2013)¹⁰⁷ im Jahr 2013 entwickelte Skip-Gram-Modell repräsentiert ähnlich wie CBOW eine neuronale Netzwerkarchitektur, die einen Ansatz mit den drei Schichten Input-Layer, Projection-Layer und Output-Layer verfolgt. Grundsätzlich wurden bei der Konzeption von Word2Vec weniger tiefe beziehungsweise

¹⁰² Vgl. (Mikolov, Chen, et al., 2013)

¹⁰³ Vgl. (Ling et al., 2017), Seite 969.

¹⁰⁴ Vgl. (Dodal & Kulkarni, 2018), Seite 2.

¹⁰⁵ Vgl. (Sultana Ritu, Nowshin, Mahadi Hasan Nahid, & Ismail, 2018), Seite 3.

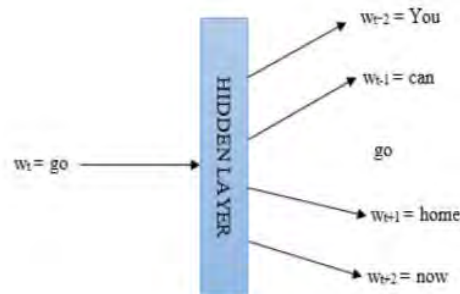
¹⁰⁶ Vgl. (J. Chen, Tandon, & Melo, 2015), Seite 225.

¹⁰⁷ Vgl. (Mikolov, Chen, et al., 2013), Seite 1-12.

komplexe neuronale Netze verwendet, um eine höhere Verarbeitungsgeschwindigkeit sowie ein effizienteres Training großer Datenmengen zu ermöglichen¹⁰⁸.

Anders als bei CBOW prognostiziert SG den Kontext auf Basis eines vorhandenen Wortes¹⁰⁹. So können beispielsweise, wie in der folgenden Abbildung dargestellt, die Wörter ‚You‘, ‚can‘, ‚home‘ sowie ‚now‘ bezugnehmend auf das gegebene Wort ‚go‘ in einen gemeinsamen Kontext gebracht werden.

Abbildung 5: Aufbau der Skip-Gram-Architektur



Quelle: (Chaitanya et al., 2018), Seite 1139

SG eignet sich aufgrund der Durchführungsgeschwindigkeit sowie der erzielten Trainingsergebnisse für die Verarbeitung großer Textkörper und findet darüber hinaus beispielsweise Anwendung im Clustering oder in der Erkennung von Synonymen¹¹⁰.

3.1.3 Lernalgorithmen

Um die zuvor beschriebenen neuronalen Netze zu trainieren, stehen verschiedene Lernalgorithmen zur Verfügung. Die nachfolgend erläuterten Algorithmen verfolgen unterschiedliche Vorgehensweisen, um bestmögliche Ergebnisse innerhalb des Trainings neuronaler Netzwerke zu erzielen.

Negative sampling

Als NS wird ein Algorithmus bezeichnet, der für das Training von neuronalen Netzwerken verwendet werden kann. Anders als bei anderen Ansätzen, die die semantische Ähnlichkeit eines gegebenen Wortes zu anderen Wörtern berechnen, basiert

¹⁰⁸ Vgl. (Mikolov, Chen, et al., 2013), Seite 2 f.

¹⁰⁹ Vgl. (Bhoir, Ghorpade, & Mane, 2017), Seite 2.

¹¹⁰ Vgl. (Mingzhen Zhao, Bo Xu, Hongfei Lin, Yang, & Jian Wang, 2015), Seite 1766.

dieser Algorithmus auf der Annahme, dass ein zufällig aus dem Textkörper gewähltes Wort mit großer Probabilität dem gegebenen Wort unähnlich ist¹¹¹.

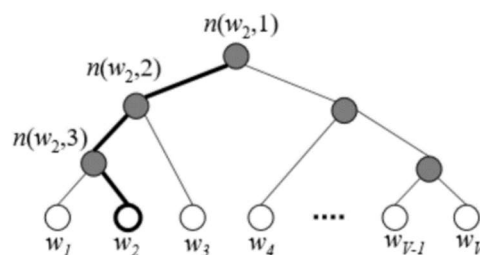
NS findet insbesondere innerhalb von W2V Anwendung und erzielt überzeugende Ergebnisse bei der Bearbeitung von häufig auftretenden Wörtern im Textkorpus¹¹².

Hierarchical softmax

HS bezeichnet eine auf dem herkömmlichen Softmax Lernalgorithmus basierende Weiterentwicklung, mit der das Training neuronaler Netzwerke effizienter gestaltet werden kann¹¹³. HS wurde erstmals 2005 von Morin und Bengio (2005) im Rahmen des Trainings neuronaler Sprachmodelle eingeführt¹¹⁴ und wird seitdem, aufgrund von Performance sowie Modellgenauigkeit, für beispielsweise das Training von W2V-Anwendungen gebraucht¹¹⁵.

Grundlegend wird bei der Verwendung von HS ein Wörterbuch, entsprechend des Textkorpus als Binärbaum nach Huffman, wie in der nachfolgenden Abbildung dargestellt, aufgebaut.

Abbildung 6: Hierarchical Softmax mit Huffman Tree



Quelle: (Mohammed & Umaashankar, 2018), Seite 1091

Innerhalb des binären Baums haben die am häufigsten auftretenden Wörter die kürzeste Codierung¹¹⁶, woraus ein geringerer Speicherverbrauch sowie eine verbesserte Durchführungsgeschwindigkeit hervorgeht. Das Verfahren erfolgt in zwei Schritten: Im

¹¹¹ Vgl. (Yi, Zheng, Wang, Qin, & Chen, 2017), Seite 586.

¹¹² Vgl. (Li Zhang, Li, & Wang, 2017), Seite 5630.

¹¹³ Vgl. (Mikolov, Sutskever, et al., 2013), Seite 3.

¹¹⁴ Vgl. (Morin & Bengio, 2005), Seite 249 f.

¹¹⁵ Vgl. (Mikolov, Sutskever, et al., 2013), Seite 3.

¹¹⁶ Vgl. (Huffman, 1952), Seite 1099.

ersten Schritt wird entsprechend den Trainingsdaten ein Wörterbuch erstellt, wobei zusätzlich eine initiale Textbereinigung durch beispielsweise Löschung der Stoppwörter beziehungsweise der im Text selten auftretenden Wörter (Parameter: `min_count`) vorgenommen werden kann¹¹⁷. Im darauffolgenden Schritt wird die semantische Ähnlichkeit der sich im gegebenen Textkorpus befindenden Wörter berechnet und für jedes Wort ein n -dimensional (Parameter: n =Vektorgröße) großer Vektor als Ergebnisrepräsentation erzeugt¹¹⁸.

3.2 GloVe

3.2.1 Definition des Ansatzes sowie Besonderheiten

Der von Pennington et al. (2014) entwickelte unsupervised Machine Learning-Ansatz, bezeichnet einen Algorithmus, mit dem Word Embeddings erstellt werden können¹¹⁹. Die erstellten Word Embeddings werden durch eine Vektorenrepräsentation, ähnlich wie beim Verfahren Word2Vec, dargestellt. Anders als Word2Vec verwendet GloVe ein zählungsbasiertes Word Embedding-Modell anstelle eines prädiktiven¹²⁰. Des Weiteren betrachtet das Verfahren semantische Gemeinsamkeiten verschiedener Wörter global. Die Kontextberechnung findet somit in Bezug auf den gesamten Trainingskorpus statt¹²¹.

Wie Sravani et al. (2018) beschreiben, wird für den Ansatz eine Co-Occurrence Matrix erzeugt, mit der alle vektorbasierten Berechnungen sowie Vorhersagen anstelle des eigentlichen Korpus durchgeführt werden¹²². Die Co-Occurrence Matrix und deren Funktionalität werden im Nachgang näher beschrieben. Durch die Matrix führt der Algorithmus eine Reduktion der Dimensionen durch und verfolgt, wie häufig welche Wörter im Trainingskorpus auftreten¹²³.

Darüber hinaus erläutern Lauren et al. (2017), dass jeder Eintrag in der Co-Occurrence Matrix gewichtet wird, um so die Wortabstände einzelner Wörter eines Kontextes zu bestimmen. Dabei gilt: je größer das jeweilige Gewicht, desto näher liegen die Wörter semantisch beieinander und je kleiner das jeweilige Gewicht, desto geringer ist der

¹¹⁷ Vgl. (Islam et al., 2016), Seite 364.

¹¹⁸ Vgl. (Xiaobo Zhang & Yu, 2017), Seite 262.

¹¹⁹ Vgl. (Pennington et al., 2014), Seite 1532.

¹²⁰ Vgl. (Ziak & Kern, 2016), Seite 194.

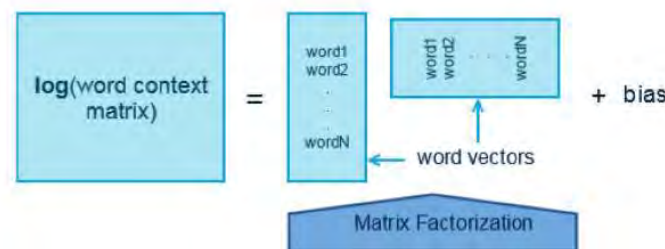
¹²¹ Vgl. (Yu, Wang, Lai, & Zhang, 2018), Seite 672.

¹²² Vgl. (Sravani, Reddy, & Thara, 2018), Seite 2379.

¹²³ Vgl. (Lauren et al., 2017), Seite 1818.

semantische Kontext. Die Gewichtung sowie das Befüllen der Co-Occurrence Matrix erfordert mindestens einen Durchlauf durch den Trainingskorpus¹²⁴. Um die Größe der Matrix zu beschränken und somit Speicherproblematiken vorzubeugen, erörtern Chowdhury et al. (2018), dass eine Faktorisierung der Co-Occurrence Matrix vorgenommen wird¹²⁵. Lauren et al. (2017) veranschaulichen den Prozess der Matrix-Faktorisierung sowie der Verwendung der Logarithmusfunktion zur Bildung von Word Embeddings in der nachfolgenden Darstellung¹²⁶.

Abbildung 7: Matrix-Faktorisierung bei GloVe



Quelle: (Lauren et al., 2017), Seite 1818

Gupta et al. (2017) beschreiben GloVe als Modell der globalen logarithmischen Regression, das die Vorteile einer verallgemeinerten Koordinatentabelle mit den Methoden lokaler prädiktiver Word Embedding-Ansätze wie beispielsweise Word2Vec kombiniert und dabei die gewonnenen statistischen Informationen effektiv nutzt, indem nur gewichtete Wörter ungleich null durch den Machine Learning-Algorithmus trainiert werden^{127 128}. Dies führt zu einer Steigerung der Trainingseffizienz, insbesondere hinsichtlich der Durchführungszeiten¹²⁹.

Für das Training mittels GloVe ist es erforderlich eine Parametrisierung vorzunehmen. Die verschiedenen Parameter sollen folgend kurz vorgestellt werden. Da viele Parameter

¹²⁴ Vgl. (Lauren et al., 2017), Seite 1818.

¹²⁵ Vgl. (Chowdhury, Imon, & Islam, 2018), Seite 4.

¹²⁶ Vgl. (Lauren et al., 2017), Seite 1818.

¹²⁷ Vgl. (Gupta & Tripathy, 2017), Seite 1043.

¹²⁸ Vgl. (Y. Sharma, Agrawal, Jain, & Kumar, 2017), Seite 230.

¹²⁹ Vgl. (Pennington et al., 2014), Seite 1540.

mit denen von Word2Vec äquivalent sind, gilt es nur die für GloVe spezifischen zu erörtern:

X_Max

Suzuki et al. (2018) beschreiben den Parameter als Bestandteil der Gewichtungsfunktion¹³⁰. Der Parameter, so Pennington et al. (2014), kalibriert die Sättigung der Funktion und sollte je nach Größe des Trainingskorpus variieren¹³¹.

Memory

Durch den Parameter Memory wird die maximale Arbeitsspeichergröße definiert, die zur Prozessierung der Daten verwendet werden darf. Da oftmals mit großen Trainingskörpern gearbeitet wird, ist dieser Parameter elementar, um den Ressourcenverbrauch des Trainings zu beschränken.

Aufgrund der Effizienz bei der Durchführung des Trainings sowie der hohen, bei der Evaluierung erzielten, Accuracy Werte¹³² gilt GloVe als populärer Word Embedding-Ansatz. Der Algorithmus findet Anwendung in verschiedenen Bereichen der Empirie und wird oftmals zusammen mit Word2Vec gebraucht, um die Ergebnisse der jeweiligen Studie durch beide Verfahren variabler zu gestalten und zu vergleichen.

3.2.2 Co-Occurrence Matrix

Als counting-based Word Embedding-Ansatz verwendet GloVe eine Co-Occurrence Matrix, um die Häufigkeit auftretender Wörter im Trainingskorpus zu identifizieren. Basierend auf der Häufigkeit einzelner Wörter in Kombination mit anderen, ermittelt der Algorithmus Rückschlüsse über die Ähnlichkeit der auftretenden Wörter untereinander¹³³. Der GloVe-Algorithmus greift zur Ermittlung der Ähnlichkeit zwischen einzelnen Worten auf die globale Co-Occurrence Matrix zurück¹³⁴. Das unten dargestellte Schaubild zeigt eine Co-Occurrence Matrix wie sie beispielsweise durch den GloVe-Algorithmus aufgebaut und verwendet wird.

¹³⁰ Vgl. (Suzuki & Nagata, 2015), Seite 187.

¹³¹ Vgl. (Pennington et al., 2014), Seite 1535.

¹³² Vgl. (Pennington et al., 2014), Seite 1540 f.

¹³³ Vgl. (Hill & Simha, 2016), Seite 26.

¹³⁴ Vgl. (Y. Sharma et al., 2017), Seite 279.

Abbildung 8: Co-Occurrence Matrix im GloVe-Verfahren

Probability and Ratio	K=solid	K=gas	K=water	K=fashion
P(k ice)	1.9×10^{-3}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
P(k Steam)	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
P(k ice)/P(k Steam)	8.9	8.5×10^{-2}	1.36	0.96

Quelle: (Pennington et al., 2014), Seite 1534

Für die Verwendung einer Co-Occurrence ist der zuvor beschriebene Parameter Window elementar¹³⁵. Dieser definiert, wie viele benachbarte Wörter, ausgehend vom zu betrachtenden Wort, gemeinsam hinsichtlich des Kontextes analysiert werden sollen. Allgemein baut der Algorithmus für jedes betrachtete Wort eine Matrix mit der gesetzten Parametergröße auf¹³⁶. Die Logik zählt dabei die Häufigkeit von gemeinsam auftretenden Wörtern innerhalb eines Kontexts^{137 138}. Wenn Wörter gemeinsam auftreten, dann wird eine positive Zahl, basierend auf der Anzahl gemeinsamer Vorkommen, gesetzt. Sollten Wörter nicht gemeinsam gefunden werden, dann wird der Co-Occurrence-Wert auf null gesetzt.

Nachfolgend soll zur vereinfachten Erklärung der Logik einer Co-Occurrence Matrix ein Beispiel aufgebaut werden. Hierfür soll der Parameter Window = 1 gesetzt werden. Somit wird jeweils ein Wort links und rechts des beobachteten Wortes betrachtet. Das Beispiel wird anhand von folgendem Satz durchgeführt werden: „I love Programming. I love Math. I tolerate Biology.“¹³⁹:

Tabelle 1: Logische Anwendung einer Co-Occurrence Matrix

	I	love	Programming	Math	tolerate	Biology	.
I	0	2	0	0	1	0	2
love	2	0	1	1	0	0	0
Programming	0	1	0	0	0	0	1
Math	0	1	0	0	0	0	1

¹³⁵ Vgl. (X. Wu & Li, 2017), Seite 1999.

¹³⁶ Vgl. (Sravani et al., 2018), Seite 2379.

¹³⁷ Vgl. (Jun-Li, Xiao-Min, & Ya-Xing, 2017), Seite 2565.

¹³⁸ Vgl. (Chowdhury et al., 2018), Seite 4.

¹³⁹ Vgl. (Vargas, 2017), Seite 1.

tolerate	1	0	0	0	0	1	0
Biology	0	0	0	0	1	0	1
.	1	0	1	1	0	1	0

Quelle: In Anlehnung an (Vargas, 2017), Seite 1

Die Matrix lässt erkennen, dass die Wörter ‚Programming‘ und ‚Math‘ denselben Co-Occurrence-Wert haben und somit eine semantische Ähnlichkeit aufweisen. Diese zählungsbasierte Logik kann auf alle sich in der Matrix befindenden Wörter übertragen werden.

4. Methodische Vorgehensweise

4.1 Datenbeschaffung

4.1.1 Beschaffung der Evaluierungsdaten

Um die trainierten Wortvektoren hinsichtlich ihrer Richtigkeit beziehungsweise Prädiktion zu validieren, sollen Onlineproduktbewertungen des international agierenden Versandhändlers Amazon.com (Amazon) verwendet werden. Das Unternehmen Amazon verwendet zur Identifikation der Artikel eine eigene Produktnummer, die Amazon Standard Identification Number, kurz ASIN¹⁴⁰. Die ASIN ist für die Spielekonsole ‚Super NES Classic‘ lautet beispielsweise B0721GGGS9 und kann aus der URL sowie aus dem Quelltext der Produktseite herausgelesen werden (siehe nachfolgende Abbildung).

Abbildung 9: Selektion von ASIN-Nummern aus HTML-Quelltexten



Da Amazon den automatisierten Zugriff auf den Quelltext schützt, wird zunächst Quelltext händisch in eine lokale Datei kopiert. Im Anschluss wird der Quelltext dann auf die in Abbildung 9 angegebene Zeichenfolge durchsucht und im Anschluss auf ASIN-Nummern gefiltert. Es wurden Produktübersichten verschiedener Kategorien besucht, um mit dem beschriebenen Verfahren heterogene ASIN-Nummern zu selektieren. Mittels des beschriebenen Wegs konnten im Zeitraum vom 10.12.2018 bis 13.01.2019 500 ASIN-Nummern gesammelt werden. Mittels dieser konnten dann Onlineproduktbewertungen heruntergeladen werden. Zum Download der Produktbewertungen wurde ein von Esuli (2015) entwickeltes Perl-Skript verwendet¹⁴¹, das über die nachfolgend beschriebene Evaluierungsplattform bereitgestellt wurde. Das auch in der Empirie etablierte^{142 143 144} Skript baut eine URL mittels ASIN Nummer auf und fragt die Onlineproduktwebseite ab, um so die jeweilige Onlineproduktbewertung zu crawlen. Im Anschluss daran werden die

¹⁴⁰ Vgl. (Amazon.com, 2019), Seite 1.

¹⁴¹ Vgl. (Esuli, 2015), Seite 1.

¹⁴² Vgl. (Fry & Manna, 2016), Seite 375.

¹⁴³ Vgl. (Saad & Mathiak, 2013), Seite 581.

¹⁴⁴ Vgl. (Kaushik, Sangwan, & Hansen, 2013), Seite 240.

sich im Rohformat befindenden Bewertungen aufbereitet und in eine CSV-Datei übertragen. Aufgrund der Beschränkung seitens Amazon.com auf maximal 60 Anfragen pro Minute konnten im Zeitraum vom 10.12.2018 bis 13.01.2019 ca. 100 Megabyte an Bewertungsdaten heruntergeladen werden. Darüber hinaus konnten die bereits im Rahmen der Masterarbeit von Florian Bosten (2016)¹⁴⁵ nach demselben Prinzip gecrawlten Onlineproduktbewertungsdaten verwendet werden. Insgesamt wurden so ca. 2,5 Gigabyte an Bewertungsdaten, was etwa 1.900.000 Millionen Bewertungen entspricht, zusammengetragen, die innerhalb dieser Arbeit sowohl als Evaluierungs- als auch als Trainingsdaten für die zu erzeugenden Wortvektoren verwendet werden können.

4.1.2 Beschaffung der Trainingsdaten

Für die Durchführung eines möglichst umfangreichen sowie effizienten Trainings von Wortvektoren, bedarf es einer großen Menge an heterogenen Wörtern beziehungsweise Texten¹⁴⁶. Für das in dieser Arbeit durchgeführte Training sollen Daten aus der englischsprachigen Wikipedia verwendet werden.

Die englischsprachige Wikipedia umfasste im Dezember 2018 ungefähr 36.000.000 Millionen Seiten mit Inhalt¹⁴⁷. In der Empirie wird der Wikipedia-Korpus, insbesondere aufgrund der hohen Qualität der Texte, oftmals für das Training von Wortvektoren eingesetzt^{148 149 150}. Wikipedia bietet die Möglichkeit unterschiedliche Ansammlungen von Artikeln kostenfrei aus dem Internet herunterzuladen. Die Daten enthalten sowohl reine Texte als auch Metadaten und befinden sich im XML-Dateiformat¹⁵¹.

Des Weiteren stehen Textsammlungen mit verschiedenen Volumina zur Verfügung, was der Untersuchung des Einflusses verschiedener Textgrößen auf die Prädiktion von Wortvektoren als Grundlage dient.

Zhang und Chen (2016) führten Experimente zum Einfluss der Größe auf die Prädiktion eines Textkorpus durch und fanden heraus, dass sich ein größerer Textkorpus positiv auf

¹⁴⁵ Vgl. (Bosten, 2016), Seite 1.

¹⁴⁶ Vgl. (Van Tuan & Sato, 2017), Seite 1.

¹⁴⁷ Vgl. (Wikimedia Foundation, 2018)

¹⁴⁸ Vgl. (Urata & Maeda, 2017), Seite 344.

¹⁴⁹ Vgl. (Semberecki & Maciejewski, 2017), Seite 357.

¹⁵⁰ Vgl. (Mori & Satoh, 2018), Seite 486.

¹⁵¹ Vgl. (Wikimedia Foundation, 2019), Seite 1.

die Prädiktion auswirkt¹⁵². Basierend auf diesem Ergebnis soll folgende Hypothese für diese Arbeit formuliert werden:

- H1 – Je größer der Textkorpus, desto höher ist die Prädiktion der Vektoren bezogen auf die Amazon-Bewertungen

Um die aufgestellte Hypothese zu validieren, wurden jeweils drei Artikelsammlungen mit einem Volumen von 500, 300 sowie 150 Megabyte der englischsprachigen Wikipedia heruntergeladen.

4.2 Datenaufbereitung

4.2.1 Aufbereitung der Evaluierungsdaten

Zur Sicherstellung einer störungsfreien Evaluierung sind vorbereitende Textaufbereitungen zu realisieren. Da der Originalwortlaut der Amazon-Rezessionen zur Prüfung verwendet werden soll, gilt es, beispielsweise HTML-Tags wie `<br/>` oder `<b></b>` sowie ASCII-Zeichen aus den Daten zu entfernen. Dies hat das Ziel, negative Einflüsse hinsichtlich der Prädiktion der Wortvektoren zu vermeiden.

Für die Aufbereitung der Evaluierungsdaten soll das Funktionsmodul `Prepare_Texts` der von Di Stefano (2019) sowie Erhardt (2019) entwickelten Machine Learning-Plattform TMoML genutzt werden^{153 154}. Hier kann eine spezielle Variante für Amazon-Onlinebewertungstexte verwendet werden, die aus den, in einer CSV-Datei gespeicherten, gecrawlten (erläutert in Kapitel 4.1.1) Bewertungsdaten einzelne Textdateien mit jeweils einer Bewertung erstellt.

Um den Originalwortlaut der jeweiligen Bewertung nicht zu verändern, sollen die Evaluierungsdaten lediglich von HTML-Tags, vordefinierten ASCII-Zeichen sowie überflüssigen Leerzeichen bereinigt werden. Eine Auflistung der definierten und zu entfernenden Zeichen befindet sich im Anhang A. Zur Ersetzung der definierten Zeichen verwendet TMoML die Funktionen `Search & Replace` sowie `Regular Expressions (Regex)`.

¹⁵² Vgl. (Lei Zhang & Chen, 2016), Seite 169.

¹⁵³ Vgl. (Di Stefano, 2019), Seite 35.

¹⁵⁴ Vgl. (Erhardt, 2019), Seite 49-53.

Neben der Textentfernung beziehungsweise der Zeichenersetzung wird eine Tokenization der Bewertungstexte durchgeführt, sprich die Texte werden in verschiedene Zeichenketten nach beispielsweise Wörtern oder Phrasen aufgeteilt¹⁵⁵. Eine Tokenization ist, wie Solangi et al. (2018) erläutern, für die Weiterverarbeitung der Daten in verschiedenen NLP-Anwendungen notwendig¹⁵⁶.

Darüber hinaus wird eine Rechtschreibprüfung mit dem Tool Hunspell¹⁵⁷, das in vielen Open Source Projekten wie beispielsweise Libre Office oder Mozilla Firefox verwendet wird¹⁵⁸, durchgeführt. Der Parameter Dictionary wird für die Evaluierungsdaten mit en_US übergeben, da es sich bei den Daten um englischsprachige Produktbewertungen handelt. Zudem kann der zugelassene sowie zu prüfende Zeichenbestand über das Setzen der Sprache Englisch auf A-z sowie 0-9 beschränkt werden.

4.2.2 Aufbereitung der Trainingsdaten

Da sich die heruntergeladenen Wikipedia-Daten nach dem Download im XML-Format befinden, für das Training jedoch reine Textdateien benötigt werden, gilt es die Daten aufzubereiten.

Zur Aufbereitung der Trainingsdaten soll ein selbst entwickeltes Programm verwendet werden. Dieses Programm beziehungsweise Skript wird mittels der Programmiersprache Python in der Version 3.6 realisiert. Zusätzlich soll auf die von Rehurek und Sojka (2010) vorgestellte Gensim Softwarebibliothek zurückgegriffen werden¹⁵⁹. Mit der Funktion WikiCorpus bietet das Softwareframework eine gekapselte Implementierung zur Aufbereitung von Wikipedia-Artikeln beziehungsweise -Dumps an.

Sun und Hao (2017) beschreiben, dass sich die Wikipedia-Dumps im bz2-Dateiformat befinden müssen, um mit der Funktion WikiCorpus transformiert werden zu können¹⁶⁰. Neben der Bereinigung der Wikipedia-Dumps von XML-Tags und Metadaten wird ein

¹⁵⁵ Vgl. (Jettakul et al., 2018), Seite 2.

¹⁵⁶ Vgl. (Solangi et al., 2018), Seite 2.

¹⁵⁷ Vgl. (Pirinen & Linden, 2010), Seite 477-484.

¹⁵⁸ Vgl. (Clark & Araki, 2011), Seite 8.

¹⁵⁹ Vgl. (Rehurek & Sojka, 2010), Seite 46-50.

¹⁶⁰ Vgl. (Sun & Hao, 2017), Seite 3.

Tokenizing der Artikel durchgeführt¹⁶¹, sodass die fertig aufbereiteten Texte in verschiedenen Word Embedding-Ansätzen weiterverarbeitet werden können.

Der Ablauf des Aufbereitungsskriptes soll durch das sich im Anhang B befindende UML-Diagramm dargestellt werden. UML steht als Abkürzung für Unified Modeling Language und ist eine Implementierungssprache zur Modellierung von Software, die sich großer Beliebtheit innerhalb der Softwareentwicklung sowie des Softwareengineerings erfreut¹⁶². Die für die Modellierung benötigten Diagramme sollen mit der von Auer et al. (2003) kostenfrei zur Verfügung gestellten Modellierungssoftware UMLet erstellt werden¹⁶³.

Innerhalb des Skriptes wird zunächst die Startzeit gespeichert. Dies dient nach erfolgreicher Datentransformation zur Kalkulation der konsumierten Gesamtzeit der Verarbeitung. Darauffolgend wird die Funktion `parse_wiki()` aufgerufen, in der die Methode `WikiCorpus` unter Verwendung von mehreren Prozessen durchgeführt wird. Sobald die Methode erfolgreich prozessiert ist, wird eine Textdatei angelegt, in die die aus dem Datenkorpus aufbereiteten Daten transferiert werden. Für den Datentransfer aus dem Datenkorpus in die Textdatei soll die vom Gensim-Framework zur Verfügung gestellte Methode `get_texts()` verwendet werden. `Get_texts()` wird referenzierend auf das von `WikiCorpus` erzeugte Modell aufgerufen werden. Nach 10.000 prozessierten Datensätzen soll eine Meldung an die Konsole gegeben werden, die den Benutzer über den Fortschritt der Prozessierung informiert. Sobald der Datentransfer in die Textdatei abgeschlossen ist, wird die benötigte Zeit berechnet, mittels der Funktion `time_to_string()` formatiert und im Anschluss über die Konsole an den Benutzer übermittelt. Sollte ein Prozess Fehler bei der Durchführung verursachen, wird eine Meldung an den Benutzer ausgegeben und das Skript beendet.

Eine Rechtschreibprüfung sowie anschließender Korrektur soll für die Wikipedia-Dumps nicht durchgeführt werden, da Wikipedia bei der Veröffentlichung eines Artikels bereits eine Rechtschreibprüfung vornimmt.

¹⁶¹ Vgl. (Rehurek, 2019), Seite 1.

¹⁶² Vgl. (Gotti & Mbarki, 2016), Seite 1.

¹⁶³ Vgl. (Auer, Tschurtschenthaler, & Biffel, 2003), Seite 267-272.

4.3 Training von Word Embeddings

4.3.1 Auswahl der Trainingsmethoden

Um Word Embeddings zu erzeugen, gibt es verschiedene Herangehensweisen. Zwei derzeit populäre Ansätze wurden bereits im Kapitel ‚Verfahren des Word Embedding‘ (Kapitel 3.) näher betrachtet. Im Rahmen der vorliegenden wissenschaftlichen Arbeit sollen eigene Word Embeddings beziehungsweise Wortvektoren trainiert werden. Als Basis für das Training sollen die zuvor gesammelten und anschließend aufbereiteten Trainingsdaten verwendet werden.

Damit die aufgestellten Hypothesen möglichst unabhängig von der angewandten Herangehensweise betrachtet werden können, soll das Training beziehungsweise die Erstellung der Wortvektoren mit verschiedenen methodischen Ansätzen gelöst werden. Ausgehend von dieser Anforderung wurden zunächst verschiedene Ansätze zur Erstellung von Wortvektoren miteinander verglichen. Im Rahmen des Vergleichs werden WordNet als lexikalische Datenbank, die Wörter in Gruppen von Synonymen, sogenannte Synsets, aufteilt¹⁶⁴ ¹⁶⁵, FastText, ein Textklassifizierer ohne Verwendung neuronaler Netzwerke¹⁶⁶, Word2Vec als Word Embedding-Ansatz, der Wörter basierend auf dem Kontext prädiktiv gruppiert¹⁶⁷, und GloVe als zählungsbasierter Ansatz betrachtet. Aufgrund ihrer Popularität in der Empirie sollen Word2Vec und GloVe als Verfahren zur Erstellung von Wortvektoren verwendet werden. Iacobacci et al. (2015) führten einen Vergleich über die verschiedenen Vorgehensweisen durch und stellten heraus, dass Word2Vec sowie GloVe die besten Ergebnisse hinsichtlich der Prädiktion liefern¹⁶⁸.

Bei den von Sravani et al. (2018) durchgeführten Studien werden mit dem Verfahren GloVe, im direkten Vergleich zu Word2Vec, die besten Ergebnisse erzielt¹⁶⁹. Shancheng et al. (2018) bestätigen¹⁷⁰ diese Beobachtung ebenso wie Pennington et al. (2014)¹⁷¹. Aus

¹⁶⁴ Vgl. (Q. Li et al., 2016), Seite 4014.

¹⁶⁵ Vgl. (Miller, 1995), Seite 40.

¹⁶⁶ Vgl. (Alessa, Faezipour, & Alhassan, 2018), Seite 367.

¹⁶⁷ Vgl. (Amorim et al., 2018), Seite 2.

¹⁶⁸ Vgl. (Iacobacci, Pilehvar, & Navigli, 2015), Seite 101.

¹⁶⁹ Vgl. (Sravani et al., 2018), Seite 2380.

¹⁷⁰ Vgl. (Shancheng, Yunyue, & Fuyu, 2017), Seite 312 f.

¹⁷¹ Vgl. (Pennington et al., 2014), Seite 1537-1541.

den Studien von Wu et al. (2017)¹⁷², Baroni et al. (2014)¹⁷³ sowie Stratos et al. (2015)¹⁷⁴ geht jedoch hervor, dass geeignetere Ergebnisse mit dem Verfahren Word2Vec erzielt werden können.

Bezugnehmend auf die zuvor genannten Forschungsbeiträge soll im Rahmen der vorliegenden Arbeit folgende Leitfrage beantwortet werden:

- L1 – Welches Word Embedding-Verfahren erzielt, bezogen auf die Amazon-Onlineproduktbewertungen, die besten Prädiktionsergebnisse?

4.3.2 Prototypisierung der Trainingsmethoden

Für eine praktische Validierung der Hypothesen wird für die ausgewählten Word Embedding-Ansätze Word2Vec und GloVe jeweils ein Prototyp implementiert.

Die Implementierung beider Ansätze soll im Nachgang je Ansatz näher erläutert werden.

GloVe

Da GloVe in der vorliegenden Thesis verwendet werden soll, um Wortvektoren basierend auf verschiedenen Trainingskörpern zu bilden, gilt es, einen Prototyp des Algorithmus zu implementieren. Hierfür wird auf die von Pennington et al. (2014) zur Verfügung gestellte Programmierung zurückgegriffen¹⁷⁵, die auf der Filehosting-Plattform GitHub von ebendiesen veröffentlicht wurde¹⁷⁶.

Der Word Embedding-Ansatz besteht aus den vier miteinander zusammenhängenden Einzelmodulen Vocab_count, Cooccur, Shuffle und GloVe. Vocab_Count wird für eine Vorselektion des Trainingskörpers, basierend auf den festgelegten Hyperparametern, wie beispielsweise der Mindestwortfrequenz und der maximalen Vokabular Größe, verwendet. Dies ist erforderlich, um den Umfang des Word Embedding zu limitieren. Das Modul Cooccur baut, basierend auf dem aus Vocab_Count stammenden Ergebnis, eine Co-Occurrence Matrix auf. Bei der Ausführung ist es möglich, die Größe der Matrix durch den Parameter Memory zu limitieren. Im Anschluss wird die Matrix dann durch

¹⁷² Vgl. (C. Wu et al., 2017), Seite 272 f.

¹⁷³ Vgl. (Baroni et al., 2014), Seite 242 f.

¹⁷⁴ Vgl. (Stratos, Collins, & Hsu, 2015), Seite 1289.

¹⁷⁵ Vgl. (Pennington et al., 2014), Seite 1532-1543.

¹⁷⁶ Vgl. (Pennington, Socher, & Manning, 2019a), Seite 1.

Shuffle gemischt und anschließend wieder zusammengeführt. Nachdem die drei vorhergehenden Module erfolgreich prozessiert sind, wird das GloVe-Modell durch den GloVe-Algorithmus trainiert. Dabei wird sowohl die aus Vocab_Count erstellte Vokabeldatei sowie die aufbereitete Co-Occurrence Matrix verwendet. Weiterhin können dem Modul unterschiedliche Hyperparameter mitgegeben werden¹⁷⁷.

Grundsätzlich ist das GloVe-Modell für unixbasierte Systeme konzipiert und implementiert. Um die Programmierung unter dem Betriebssystem Windows 10 durchzuführen, ist ein Cross-Compiling erforderlich. Die Kompilation soll durch die Entwicklungsoberfläche Dev C++ durchgeführt werden¹⁷⁸. Dev C++ ist ein GNU Compiler, mit dem es möglich ist, sowohl C als auch C++ Dateien zu kompilieren¹⁷⁹.

Um die kompilierten Dateien in korrekter Reihenfolge durchzuführen, wird eine Batch Datei erstellt, mit der die zuvor genannten Module sequenziell mit verschiedenen Hyperparametern aufgerufen werden. Das UML-Aktivitätsdiagramm zur Visualisierung des Ablaufs der Batch-Datei findet sich im Anhang C.

Die beim Aufruf des GloVe-Modells benötigten sowie optional setzbaren Hyperparameter können sowohl über die Kommandozeile an das Skript übergeben als auch über eine externe Konfigurationsdatei definiert werden. Innerhalb des Skriptes wird die Konfigurationsdatei ausgelesen. Sollte das Skript bereits mit Parametern aufgerufen worden sein, so werden diese priorisiert behandelt.

Word2Vec

Zunächst soll für die Implementierung des Word2Vec-Prototypen ein passendes Framework ausgewählt werden. Word2Vec wird durch verschiedene Softwarebibliotheken ausgeliefert¹⁸⁰, wie beispielsweise innerhalb des von Abadi et al. (2016)¹⁸¹ veröffentlichten Machine Learning-Frameworks Tensorflow (TF)¹⁸² oder von dem Python Framework Gensim¹⁸³. Weiterhin kann Word2Vec auch eigenständig in der

¹⁷⁷ Vgl. (Pennington, Socher, & Manning, 2019b), Seite 1.

¹⁷⁸ Vgl. (Anoidgit, 2019), Seite 1.

¹⁷⁹ Vgl. (Samiah, Aziz, & Ikram, 2007), Seite 692.

¹⁸⁰ Vgl. (Sultana Ritu et al., 2018), Seite 1.

¹⁸¹ Vgl. (Abadi et al., 2016), Seite 273.

¹⁸² Vgl. (Ordentlich et al., 2016), Seite 1144.

¹⁸³ Vgl. (Dekhtyar & Fong, 2017), Seite 487.

Programmiersprache Python abgebildet werden¹⁸⁴. Dabei wird lediglich das von van der Walt et al. (2011)¹⁸⁵ vorgestellte Array NumPy angewandt.

Im Rahmen der vorliegenden Arbeit soll das von Rehurek et al. (2010) entworfene Softwareframework Gensim, aufgrund seiner Kompatibilität genutzt werden¹⁸⁶. Dieses stellt unterschiedliche NLP-Algorithmen sowie Anwendungen, wie unter anderem Word2Vec, zur Verfügung und kann ebenso auf dem Betriebssystem Windows 10 verwendet werden. Die Gensim-Softwarebibliothek basiert auf der Programmiersprache Python und kann über dessen Application Programming Interface (API) von verschiedenen Programmiersprachen angesprochen werden¹⁸⁷. Ein weiterer Weg die Software zu verwenden besteht darin, Gensim als Softwarebibliothek in ein Python-Skript einzubinden.

Aufbauend auf dem gewählten Machine Learning-Framework, soll Python als Programmiersprache verwendet werden, um das Gensim-Framework über dessen API anzusprechen. Als Python wird eine höhere und universelle Programmiersprache bezeichnet, die durch einen Interpreter ausgeführt wird. Die Programmiersprache findet derzeit, insbesondere durch die Verwendung in Machine Learning-Anwendungen, große Popularität in der Empirie¹⁸⁸ (siehe Abbildung 11).

¹⁸⁴ Vgl. (Rooy, 2018), Seite 1.

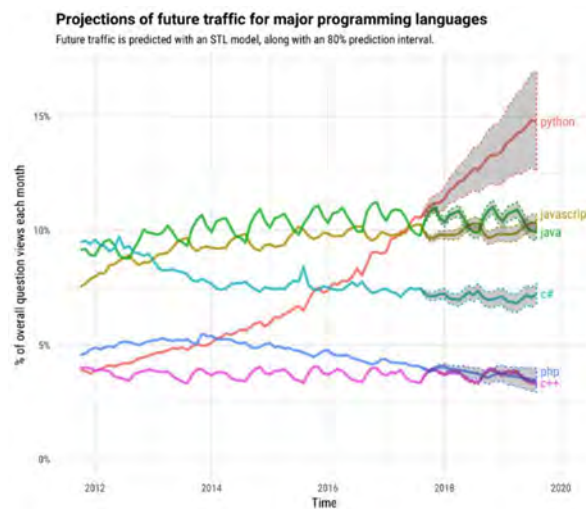
¹⁸⁵ Vgl. (van der Walt, Colbert, & Varoquaux, 2011), Seite 22-30.

¹⁸⁶ Vgl. (Rehurek & Sojka, 2010), Seite 46-50.

¹⁸⁷ Vgl. (Buitinck et al., 2013), Seite 12.

¹⁸⁸ Vgl. (Innes Stefan Karpinski Viral Shah et al., 2018), Seite 1.

Abbildung 10: Frequenz verwendeter Programmiersprachen



Quelle: (Robinson, 2017), Seite 1

Um die Aktualität der Implementierung zu gewährleisten, und aufgrund des verwendeten Frameworks wird die Programmiersprache in der Version 3.6 genutzt. Die Programmierung soll, um die Einfachheit sowie Verständlichkeit zu unterstreichen, in einem funktionalen Stil, wie von Hinsen (2009) beschrieben, erfolgen¹⁸⁹.

Das sich im Anhang D befindende UML-Aktivitätsdiagramm soll den Ablauf des zu implementieren Programmes visualisieren. Im Aktivitätsdiagramm ist zu erkennen, dass die Logik in einzelne Funktionen gekapselt wird Als zentrale Funktion dient `main()`, von der aus alle notwendigen Logiken aufgerufen werden. Dabei werden nach der Konfiguration des Loggenerators zunächst verschiedene notwendige Parameter aus einer externen Konfigurationsdatei ausgelesen. Die in der Konfigurationsdatei gespeicherten Trainingsparameter sind im Kapitel ‚Verfahren des Word Embedding‘ (Kapitel 3.) näher beschrieben. Sollten sich beim Auslesen der Konfigurationsdatei oder bei folgenden Funktionsaufrufen Fehler ergeben, so wird eine Meldung in das Applikationslog geschrieben und die Anwendung wird unterbrochen.

Nachdem die Parameter erfolgreich ausgelesen sind, sollen im nächsten Schritt die Trainingsdaten ausgelesen werden, um diese dann an die Funktion `gensim_execute()` zu übergeben. Innerhalb dieser Funktion wird, wie auch in den Studien von Yang et al.

¹⁸⁹ Vgl. (Hinsen, 2009), Seite 86-90.

(2016) oder Jia et al. (2018), die im Gensim-Framework bereitgestellte Methode Word2Vec durchgeführt^{190 191}. Dabei soll die Methode mit den aus der Konfigurationsdatei ausgelesenen Parametern aufgerufen werden. Sobald das Training sowie die Bildung der Word Embeddings abgeschlossen ist, wird durch das Prozessieren der Funktion `convert_to_txt()` das erzeugte Modell in ein Textfile transformiert und kann so durch verschiedene NLP-Anwendungen weiter verarbeitet werden.

Um eine Anbindung an die von Di Stefano (2019)¹⁹² und Erhardt (2019)¹⁹³ entwickelte Machine Learning-Plattform, die im Kapitel ‚Evaluierung der erstellten Wort Embeddings‘ (Kapitel 4.5) näher erläutert werden soll, vereinfacht zu gewährleisten, wird das erstellte Python-Skript in eine EXE-Datei umgewandelt. Zarghoon et al. (2017) verwenden in ihren Forschungen für die Umwandlung eines pythonbasierten Skriptes den im Programmierframework Python 3.6 bereitgestellten PyInstaller¹⁹⁴. Dieses Tool soll auch in der vorliegenden Thesis für die Transformation der Dateien verwendet werden.

Des Weiteren wird ein Batch-Skript entwickelt, das wiederum von der Machine Learning-Plattform angesprochen werden kann, um die EXE-Datei aufzurufen und die Word2Vec-Logik durchzuführen. Die Logik, eine ausführbare Datei mittels Batch-Skript aufzurufen, orientiert sich an der Veröffentlichung von Jackson et al. (2004)¹⁹⁵. Beim Aufrufen der Word2Vec EXE-Datei können die für das Training benötigten Parameter mitgegeben werden. So wird ermöglicht, dass Varianten über die Machine Learning-Plattform angelegt werden können. Alternativ zum Aufruf mit Trainingsparametern kann die zuvor erwähnte Konfigurationsdatei für das Setzen der Trainingsparameter verwendet werden.

4.3.3 Trainingsdurchführung

Beim Training von WE können verschiedene Faktoren elementaren Einfluss auf die Wertigkeit des Ergebnisses haben. So können beispielsweise die Auswahl-Parameter¹⁹⁶

¹⁹⁰ Vgl. (Yang & Xia, 2016), Seite 309.

¹⁹¹ Vgl. (Jia, Zhang, Wang, & Liu, 2018), Seite 281.

¹⁹² Vgl. (Di Stefano, 2019), Seite 1.

¹⁹³ Vgl. (Erhardt, 2019), Seite 1.

¹⁹⁴ Vgl. (Zarghoon, Awan, Disso, & Dennis, 2017), Seite 270.

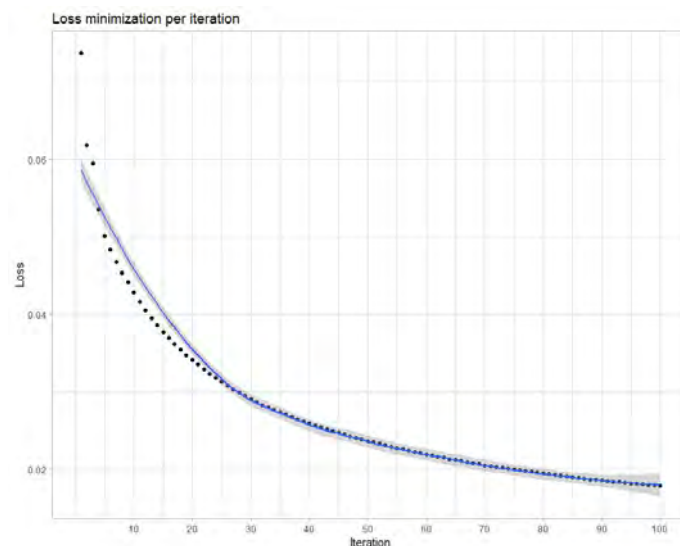
¹⁹⁵ Vgl. (Jackson, Levine, Grizzard, & Owen, 2004), Seite 12 f.

¹⁹⁶ Vgl. (Heyman, Vulićvulić, & Moens, 2017), Seite 1092.

oder die Hardware des Trainingssystems^{197 198} Einfluss auf das Resultat haben. Im Rahmen der vorliegenden Arbeit soll der Einfluss des Parameters Iterationen auf die Prädiktion untersucht werden.

Hellrich und Hahn (2016) zeigen auf, dass die Richtigkeit der trainierten Wortvektoren in Abhängigkeit zur Anzahl der Trainingsdurchläufe steht. In ihren Studien stellten sie heraus, dass insbesondere bei kleineren Trainingskörpern eine höhere Richtigkeit durch eine größere Anzahl Iterationen erzielt werden kann¹⁹⁹. Allgemein ist anzunehmen, dass mit jedem Durchlauf eine Annäherung der Verlustfunktion erzielt und somit der Verlust reduziert wird. Eigene Tests, wie in der nachfolgenden Abbildung dargestellt, bestätigen diese Aussage.

Abbildung 11: Verlustreduzierung per Iteration beim Training von Wortvektoren



Ju et al. (2015) zeigen darüber hinaus auf, dass sich ein geringerer Verlust positiv auf die Richtigkeit der trainierten Wortvektoren auswirkt²⁰⁰. Basierend auf den zuvor aufgeführten Studien sowie den eigenen Resultaten soll Folgendes hypothetisiert werden:

- H2 – Je größer die Anzahl der Trainingsdurchläufe, desto besser ist die Prädiktion der Wortvektoren bezogen auf die Evaluierungsdaten.

¹⁹⁷ Vgl. (L. Li, Liu, Qin, Wang, & Huang, 2016), Seite 671.

¹⁹⁸ Vgl. (Simonton & Alagband, 2017), Seite 4 f.

¹⁹⁹ Vgl. (Hellrich & Hahn, 2016), Seite 2791 f.

²⁰⁰ Vgl. (Ju, Zhou, Li, & Liu, 2015), Seite 2281.

4.4 Visualisierung der erstellten Vektoren

Nach erfolgreich durchgeführtem Training soll eine Möglichkeit geschaffen werden, die erstellten Word Embeddings zu visualisieren. Für eine Ergebnisvisualisierung stehen grundsätzlich verschiedene Möglichkeiten zur Verfügung. Hierzu gehören der von Smilkov et al. (2016) vorgestellte Embedding Projector, der in das Softwareframework Tensorflow integriert ist²⁰¹, die von van der Maaten und Hinton (2008) vorgestellte Technik t-distributed Stochastic Neighbor Embedding (t-SNE)²⁰² sowie berechnete Projektionen durch PCA-2D-Plot²⁰³.

Im Rahmen der vorliegenden Arbeit soll eine Visualisierung der generierten Word Embeddings durch eine Kombination aus t-SNE und PCA erfolgen. Wie Liu et al. (2017) in ihrem Artikel über die Illustration semantischer Beziehungen innerhalb von neuronalen Word Embeddings aufzeigen, handelt es sich bei t-SNE um eine effiziente und zugleich hilfreiche Form der Visualisierung erzeugter Wortvektoren²⁰⁴.

Khagi et al. (2018) beschreiben t-SNE als ein Verfahren zur Reduktion der Dimensionalität²⁰⁵, was insbesondere für hochdimensionale Word Embeddings unermesslich für eine Illustration ist. Darüber hinaus zeigen Pezzotti et al. (2017) auf, dass t-SNE neben der Dimensionsreduzierung Berechnungen bezüglich der semantischen Ähnlichkeit der Wortvektoren durchführt, um diese optimiert in einem niedrigdimensionalen Raum zu präsentieren²⁰⁶.

Für die Visualisierung soll ein Python basiertes eigenerstelltes Skript mit t-SNE Logik verwendet werden. Wie Zhang et al. (2017) beschreiben, wird der t-SNE-Algorithmus durch das Python-Softwareframework Scikit-Learn bereitgestellt²⁰⁷. Das Softwareframework soll innerhalb des Skriptes für die Implementierung des t-SNE-Algorithmus verwendet werden. Darüber hinaus wird der Array Typ Numpy verwendet²⁰⁸.

²⁰¹ Vgl. (Smilkov et al., 2016), Seite 1-4.

²⁰² Vgl. (Van Der Maaten & Hinton, 2008), Seite 2579-2605.

²⁰³ Vgl. (White & Wilson, 2007), Seite 42.

²⁰⁴ Vgl. (Liu et al., 2018), Seite 561.

²⁰⁵ Vgl. (Khagi, Lee, & Kwon, 2018), Seite 2.

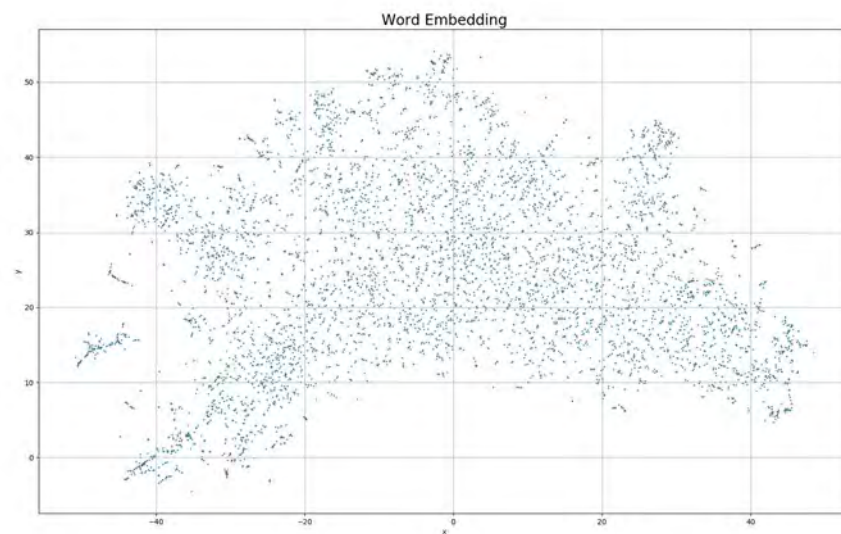
²⁰⁶ Vgl. (Pezzotti et al., 2017), Seite 1741.

²⁰⁷ Vgl. (S. Zhang et al., 2017), Seite 2200.

²⁰⁸ Vgl. (van der Walt et al., 2011), Seite 1.

Innerhalb der Implementierung wird zunächst ein bereits trainiertes Wortvektorenmodell in die Applikation geladen. Dieses Modell wird in ein Numpy Array transformiert. Darüber hinaus wird eine Wortliste über die im Modell vorhandenen Vokabeln erstellt. Nach erfolgreichem Aufbau der Wortliste wird für diese der t-SNE-Algorithmus prozessiert. Sobald der Algorithmus durchlaufen ist, wird ein DataFrame als Vorbereitung für die grafische Ausgabe erstellt. Der DataFrame soll dabei vom Typen der von McKinney (2010) veröffentlichten Pandas-Softwarebibliothek gewählt werden²⁰⁹. Im letzten Schritt wird eine grafische Darstellung unter Verwendung des von Hunter (2007) bereitgestellten Frameworks Matplotlib erstellt²¹⁰. Die Illustration der erstellten Word Embeddings sollte dann wie folgt aussehen:

Abbildung 12: Darstellung von Wortvektoren mit t-SNE



Die notwendigen Parameter `number_of_words` (zur Bestimmung der Anzahl der zu visualisierenden Wörter), `dimension` (zur Anzahl der zu visualisierenden Dimensionen) sowie der Modellpfad müssen über eine externe Konfigurationsdatei an das Programm übergeben werden.

Ein detaillierter Ablauf des Programmes kann dem entsprechenden UML-Diagramm aus dem Anhang F entnommen werden.

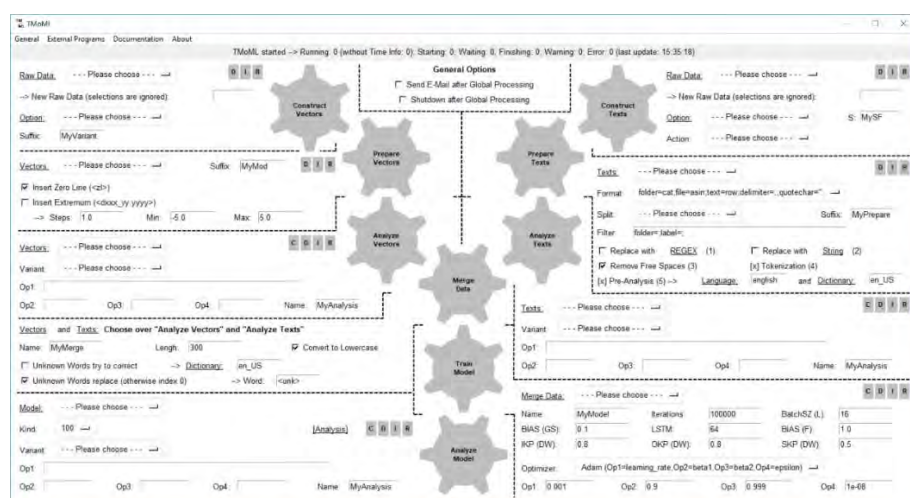
²⁰⁹ Vgl. (McKinney, 2010), Seite 52.

²¹⁰ Vgl. (Hunter, 2007), Seite 90.

4.5 Evaluierung der erstellten Word Embeddings

Die Evaluierung der erstellten Wortvektoren soll über die von Di Stefano (2019)²¹¹ sowie Erhardt (2019)²¹² erstellte Machine Learning-Plattform erfolgen. Die primär auf der Programmiersprache Python in der Version 3.6 basierte Plattform besteht aus mehreren funktional abhängigen Modulen, die jeweils mit verschiedenen Parametern über eine grafische Oberfläche aufgerufen werden können.

Abbildung 13: Oberfläche der Evaluierungsplattform



Quelle: (Di Stefano, 2019), Seite 1 / (Erhardt, 2019), Seite 1

Wie in der vorherigen Abbildung zu erkennen ist, arbeitet die Plattform sequenziell. Nach erfolgreichem Konstruieren sowie Aufbereiten der Vektoren und Evaluierungsdaten werden sowohl Vektoren als auch Texte zusammengeführt. Bei der Aufbereitung der Texte wird neben einer Rechtschreibprüfung ein für NLP-Anwendungen notwendiges Tokenizing durchgeführt. Nachdem Texte und Vektoren erfolgreich zusammengeführt sind, kann ein Machine Learning-Modell trainiert werden. Für das Training nutzt die Plattform ein Rekurrentes neuronales Netz (RNN) mit der Funktion Long Short-Term Memory (LSTM).

RNN bezeichnet eine Form des Deep Learning, die beim Trainieren die bereits berechneten Informationen mit einbezieht, wodurch sich eine starke Vorhersagekraft bei

²¹¹ Vgl. (Di Stefano, 2019), Seite 1.

²¹² Vgl. (Erhardt, 2019), Seite 1.

Problemen mit sequenziellen Anpassungsschritten ergibt²¹³. LSTM unterstützt ein RNN bei der Berechnung von wiederholt auftretenden Mustern über einen langen beziehungsweise viele Trainingsepochen andauernden Trainingszeitraum und resultiert in einer Verbesserung der Resultate²¹⁴. Da bei der Verwendung von RNN mit der Funktion LSTM die Gefahr eines Overfittings besteht, sprich eine zu starke Anpassung des neuronalen Netzes auf die Trainingsdaten²¹⁵, kann durch Drop-Outs (zufälliges Verwerfen von Gewichtungen) entgegen gesteuert werden²¹⁶.

Die Machine Learning-Plattform bietet die Möglichkeit aus verschiedenen Optimierungsalgorithmen zu wählen. Darüber hinaus können Parameter wie unter anderem Learning Rate, LSTM und Iterations für das Training gesetzt werden. Zudem ist für eine Messung des Trainingsfortschritts das externe Tool Tensorboard aus dem Machine Learning-Softwareframework Tensorflow²¹⁷ integriert. So kann sowohl die Prädiktion beziehungsweise Richtigkeit als auch der aktuell berechnete Verlust der Verlustfunktion illustriert werden.

Im Anschluss an das Training können über die Plattform Analysen, wie beispielsweise die Messung der Prädiktion beziehungsweise Richtigkeit, auf das erstellte Machine Learning-Modell gefahren werden. Weitere Details zur Anforderungsanalyse, Konstruktion sowie Implementierung der Plattform, deren Module und Funktionen können den Masterarbeiten von Di Stefano (2019)²¹⁸ und Erhardt (2019)²¹⁹ entnommen werden.

Für die Evaluierung der im Rahmen dieser Arbeit erstellten Word Embeddings soll jeweils ein Machine Learning-Modell über die Plattform erstellt werden. Das Modul ‚Train Model‘ soll für alle zu erstellenden Modelle mit den in der nachfolgenden Tabelle Trainingsparametern aufgerufen werden.

Tabelle 2: Parametrisierung der Evaluierungsplattform

Trainingsparameter	Wert
--------------------	------

²¹³ Vgl. (Yao & Jiang, 2018), Seite 2321.

²¹⁴ Vgl. (Shi, Xu, & Li, 2018), Seite 5274.

²¹⁵ Vgl. (Zhao et al., 2015), Seite 453.

²¹⁶ Vgl. (Yongzhi Zhang, Xiong, He, & Pecht, 2018), Seite 5699.

²¹⁷ Vgl. (Abadi et al., 2016), Seite 265-283.

²¹⁸ Vgl. (Di Stefano, 2019), Seite 1.

²¹⁹ Vgl. (Erhardt, 2019), Seite 1.

Iterations	30.000
BatchSZ (L)	16
BIAS (GS)	0.1
LSTM	128
BIAS (F)	1.0
IKP (DW)	0.8
OKP (DW)	0.8
SKP (DW)	0.5

Quelle: (Di Stefano, 2019) / (Erhardt, 2019)

Die zuvor aufgeführten Trainingsparameter sind basierend auf den Studien von Di Stefano (2019) als Default für das Trainingsmodul gegeben²²⁰. Neben der Parametrisierung bietet die Plattform die Möglichkeit aus unterschiedlichen Trainingsalgorithmen beziehungsweise Optimizern zu wählen. Aus diesen wird für das Training des Modells der von Chilimbi et al. (2014) veröffentlichte Adaptive Moment Estimation(Adam)-Optimizer ausgewählt²²¹. Ein von Taqi et al. (2018) durchgeführter Vergleich verschiedener Optimizer zeigt auf, dass der Adam-Optimizer überzeugende Ergebnisse in der Praxis aufweist²²². In der von Alshahrani et al. (2017) durchgeführten Untersuchung scheint der Adam-Optimizer als am geeignetsten²²³, was die Wahl des Optimizers für die vorliegende Arbeit zusätzlich unterstreicht. Neben den bereits aufgeführten und für neuronale Netzwerke relevanten Parametern, muss der Optimizer ebenfalls parametrisiert werden. Für die gesetzten Parameter des Optimizers gilt, dass für alle durchzuführenden Evaluierungen einheitliche Werte verwendet werden, um eine Vergleichbarkeit der erstellten Word Embeddings zu schaffen. In der nachfolgenden Tabelle findet sich die Parametrisierung des zu verwendenden Adam-Optimizers.

Tabelle 3: Parametrisierung des Optimizers der Evaluierungsplattform

Trainingsparameter	Wert
Learning Rate	0.005
Beta1	0.9

²²⁰ Vgl. (Di Stefano, 2019), Seite 41 ff.

²²¹ Vgl. (Chilimbi et al., 2014)

²²² Vgl. (Taqi, Awad, Al-Azzo, & Milanova, 2018), Seite 145.

²²³ Vgl. (Alshahrani, Alzahrani, Alshehri, Alharthi, & Fu, 2017), Seite 1475.

Beta2	0.999
Epsilon	1e-08

Quelle: (Di Stefano, 2019) / (Erhardt, 2019)

Für die Evaluation der Prädiktion beziehungsweise Richtigkeit der Wortvektoren während des Trainings soll das Tool Tensorboard verwendet werden. Dieses erstellt einen Verlaufsgrafen des berechneten Verlustes sowie der Richtigkeit²²⁴ der Wortvektoren bezogen auf die Evaluierungsdaten. Das Tensorboard evaluiert die Richtigkeit des Modells an 100 verschiedenen Datenpunkten²²⁵, was bei einer Gesamtanzahl von 30.000 Iterationen alle 300 Trainingsiterationen bedeutet. Somit kann durch die Verwendung des Tensorboard eine Prognose über die erzielte Richtigkeit der verschiedenen Wortvektoren, bezogen auf die Evaluierungsdaten, gezogen werden. Dennoch sollte die ermittelte Richtigkeit zusätzlich zur Verwendung des Tensorboards validiert werden. Hierzu kann die Funktion ‚Accuracy of Model‘ im Modul ‚Analyse Model‘ der Evaluierungsplattform angewandt werden, anhand derer die Richtigkeit für jeden Durchlauf kalkuliert wird. Dabei werden zufällige Datensätze evaluiert. Für wie viele Daten eine Analyse durchgeführt werden soll, kann per Parameter über die grafische Oberfläche mitgegeben werden²²⁶.

Im Rahmen der vorliegenden Arbeit soll sowohl vom externen Tool Tensorboard und dessen Richtigkeitsberechnung sowie von der Funktion ‚Accuracy of Model‘ bei einer Parametrisierung von 1000 Testdurchläufen Gebrauch gemacht werden. Darüber hinaus soll sichergestellt werden, dass die Evaluierungsergebnisse auch auf andere Szenarien übertragbar sind. Um die Übertragbarkeit der Resultate nachzuweisen, empfehlen Sharma et al. (2018), ein Modell nur auf einem Teil der Evaluierungsdaten zu bilden, mit diesen das Training durchzuführen und diese im Nachgang gegen die zurückgehaltenen Daten zu evaluieren²²⁷. Bezugnehmend auf diese Empfehlung sollen nur 80 Prozent der Evaluierungsdaten für ein Training verwendet werden. Die zurückgehaltenen 20 Prozent der Daten sollen nach erfolgreichem Abschluss des Trainings gebraucht werden, um die Übertragbarkeit des Modells zu evaluieren.

²²⁴ Vgl. (Shiddieqy, Hariadi, & Adiono, 2018), Seite 2.

²²⁵ Vgl. (Brophy, Dominguez, Wang, & Ward, 2018), Seite 3.

²²⁶ Vgl. (Erhardt, 2019), Seite 70.

²²⁷ Vgl. (R. Sharma, Bhattacharyya, Dandapat, & Bhatt, 2018), Seite 973.

5. Anwendung von Word Embedding-Verfahren

Für die Bewertung der Hypothesen sowie zur Beantwortung der Leitfrage werden verschiedene Word Embedding-Modelle erzeugt und trainiert. Die Modelle sollen dabei auf Basis der aufbereiteten Trainingsdaten generiert werden.

Zur Erzeugung der Modelle werden die eigens für die vorliegende Arbeit implementierten Trainingsskripte verwendet, durch die die ausgewählten Trainingsalgorithmen GloVe und Word2Vec betrachtet werden können.

Das Training mittels beider Verfahren sowie die anschließende Prädiktionsevaluierung mit der Machine Learning-Plattform wird auf Systemen mit folgenden Konfigurationen durchgeführt:

Tabelle 4: Konfigurationen der Trainingssysteme

Konfiguration	System 1	System 2	System 3	System 4
Betriebssystem	Windows 10 Home Version 10.0.17134	Windows Server 2016	Windows Server 2016	Windows Server 2016
Prozessor	AMD Ryzen 7 2700 Eight-Core Processor, 3200 MHz, 8 Kern(e), 16 logische Kerne	Intel® Xeon® CPU E5-2670 v3 @ 2.30 GHz (16 Prozessoren)	Intel® Xeon® CPU E5-2670 v3 @ 2.30 GHz (8 Prozessoren)	Intel® Xeon® CPU E5-2670 v3 @ 2.30 GHz (8 Prozessoren)
RAM	16 GB Hauptspeicher	36 GB Hauptspeicher	20 GB Hauptspeicher	20 GB Hauptspeicher
Grafikkarte	Radeon(TM) RX550, 2 GB	Intel Onboard	Intel Onboard	Intel Onboard
Festplatte(n)	SSD 120 GB, HDD 1 TB	Red Hat VirtIO SCSI Disk Device	Red Hat VirtIO SCSI Disk Device	Red Hat VirtIO SCSI Disk Device

Für jede aufgestellte Hypothese sollen grundsätzlich verschiedene Modelle erstellt werden, die es aus differentiellen Textquellen zu erzeugen gilt.

Zur Beantwortung der Hypothese H1 „Je größer der Textkorpus, desto höher ist die Prädiktion der Vektoren bezogen auf die Amazon-Bewertungen“ werden je Verfahren, GloVe und Word2Vec, mehrere Modelle trainiert. Entscheidend beim Training der Modelle für die Beantwortung von Hypothese H1 ist, dass die jeweilige Textquelle sich in der Größe differenziert. Wie bereits beim Aufstellen der Hypothese erwähnt, sollen drei verschiedene Textquellen zu jeweils 150 MB, 300 MB sowie 500 MB verwendet

werden. Die Parametrisierung der Trainingsalgorithmen soll für die drei Textquellen identisch gehalten werden. Insgesamt sollen neun Modelle pro Textgröße trainiert werden.

Um die Hypothese H2 ‚Je größer die Anzahl der Trainingsdurchläufe, desto besser ist die Prädiktion der Wortvektoren bezogen auf die Evaluierungsdaten‘ zu validieren, gilt es mehrere Modelle zu trainieren, bei denen der Trainingsparameter Iteration variiert. Für jeweils neun erzeugte Modelle soll der Parameter Iteration=15, Iteration=50 sowie Iteration=100 gesetzt werden. Die ausgewählten Werte sind grundsätzlich willkürlich ausgewählt. Wie im Kapitel ‚Verfahren des Word Embedding‘ (Kapitel 3.) beschrieben, wird in der Empirie ein Wert zwischen 5 und 100 Iterationen als üblich erachtet. Somit befinden sich die drei gewählten Werte innerhalb der Empfehlungen.

Die Leitfrage L1 ‚Welches Word Embedding-Verfahren erzielt, bezogen auf die Amazon-Onlineproduktbewertungen, die besten Prädiktionsergebnisse?‘ soll durch insgesamt 27 Modelle pro Trainingsalgorithmus validiert werden. Um diese Anzahl an Trainingsmodellen bereitzustellen, wird auf die bereits für die Hypothesen erstellten Word Embeddings zurückgegriffen.

Allgemein sollen identische Parameter für das Training des jeweiligen Modells gewählt werden. Dabei gilt es zwischen verfahrensspezifischen sowie allgemeinen Trainingsparametern zu unterscheiden. Es sollen nachfolgend nur Attribute beschrieben werden, die direkten Einfluss auf den Trainingserfolg haben können. Somit werden Parameter wie beispielsweise Num_Thread (GloVe) beziehungsweise Workers (Word2Vec) identisch gesetzt, allerdings nicht detaillierter begründet.

Grundsätzlich soll für beide Trainingsalgorithmen Min_Count=100 gesetzt werden, um die Word Embeddings auf Wörter zu beschränken, die mindestens eine Präsenz von 100 innerhalb des Trainingskorpus aufweisen. Diese Beschränkung soll durchgeführt werden, um möglichst nur Wörter mit semantischem Einfluss innerhalb des Trainingskorpus zu berücksichtigen. Darüber wird durch den Parameter Vector_Size=300 eine Dimensionsgröße gewählt, die, wie im Kapitel ‚Verfahren des Word Embedding‘ (Kap. 3.) beschrieben, durch die Empirie empfohlen ist. Um beim Training sowohl semantische Informationen zu berücksichtigen als auch weiterhin das Untersuchungswort zu fokussieren, wird mit Window=5 ein mittlerer Wert gewählt, durch den beide Faktoren

abgedeckt werden können. Als initiale Learning Rate wird in der derzeitigen Forschung oftmals ein Wert zwischen 0.01 und 0.1 verwendet (siehe Kapitel 3.1.1). Für das Training der Modelle sollen die Parameter $\text{ETA}=0.05$ für GloVe sowie $\text{Alpha}=0.05$ für Word2Vec verwendet werden. Diese Werte befinden sich zwischen den in der Empirie empfohlenen Werten und sollen gewählt werden, um ein Underfitting, sprich ein zu schwaches Training²²⁸, sowie ein Overfitting zu vermeiden.

Bei dem Trainingsalgorithmus GloVe soll aus Performancegründen zudem gemäß der Empfehlung von Pennington et al. (2014) die Gewichtungsfunktion durch das Setzen des Parameters $x_max=100$ beschränkt werden²²⁹.

Chowdhury et al. (2018) zeigen Szenarien auf, in denen mit der neuronalen Architektur Skip-Gram bessere Resultate hinsichtlich der Prädiktion erzielt werden können als bei der Verwendung von CBOW²³⁰. Sumit et al. (2018) sowie Chaitanya et al. (2018) bestätigen diese Ergebnisse durch das Aufzeigen weiterer Anwendungsfälle^{231 232}. Somit soll auch in der vorliegenden Arbeit Skip-Gram als neuronale Architektur genutzt werden. Dies bedeutet, dass der Parameter $\text{Sg}=1$ gesetzt ist. Darüber hinaus wird Hierarchical softmax mit dem Setzen des Parameters $\text{Hs}=1$ anstelle von Negative Sampling $\text{Negative}=0$ als Lernalgorithmus angewandt. Mikolov et al. (2013) zeigen auf, dass insbesondere bei Wörtern, die eine niedrigere Frequenz im Trainingskorpus aufweisen, mit Hierarchical Softmax überzeugende Ergebnisse erzielt werden können²³³. Da durch den Parameter $\text{Min_count}=100$ eine erhöhte Mindestfrequenz der Wörter vorhanden sein muss, wird Hierarchical Softmax für das Training der Modelle angewandt. Die folgende Tabelle soll die verwendeten Trainingsparameter für beide Verfahren zusammengefasst darstellen.

Tabelle 5: Parametrisierung der Verfahren

Parameter	GloVe	Word2Vec
Verbose	2	-
Memory	15	-

²²⁸ Vgl. (S. Li, Chua, Zhu, & Miao, 2016), Seite 667.

²²⁹ Vgl. (Pennington et al., 2014), Seite 1535.

²³⁰ Vgl. (Chowdhury et al., 2018), Seite 5.

²³¹ Vgl. (Sumit, Hossan, Al Muntasir, & Sourov, 2018), Seite 3 f.

²³² Vgl. (Chaitanya et al., 2018), Seite 1140 f.

²³³ Vgl. (Mikolov, Sutskever, et al., 2013), Seite 6.

Vocab_Min_Count / Min Count	100	100
Vocab_Max_Count	100.000	-
Vector_Size / Size	300	300
Max_Iter / Iterations	100 / 50 / 15	100 / 50 / 15
Window_Size / Window	5	5
Binary	2	-
Num_Threads / Workers	8	8
X_Max	100	-
ETA / Alpha	0.05	0.05
Sg	-	1
Hs	-	1
Negative	-	0

6. Empirische Untersuchung

6.1 Präsentation der Ergebnisse

Wie im vorangehenden Kapitel beschrieben, wurden verschiedene Word Embeddings erzeugt und im Anschluss über die zur Verfügung stehende Machine Learning-Plattform evaluiert. Die Evaluierung liefert dabei unterschiedliche Ergebnisse, die im Nachgang für jede aufgestellte Hypothese sowie pro Word Embedding-Verfahren aufgeführt werden sollen. Zur Visualisierung der jeweiligen Resultate sowie deren Verläufen während des Trainings sollen die durch Tensorboard erzeugten Diagramme verwendet werden. Nachfolgend sollen die Ergebnisse detailliert beschrieben werden. Eine tabellarische Zusammenfassung der erzielten Prädiktionswerte pro Hypothese sowie je Verfahren findet sich im Anhang F.

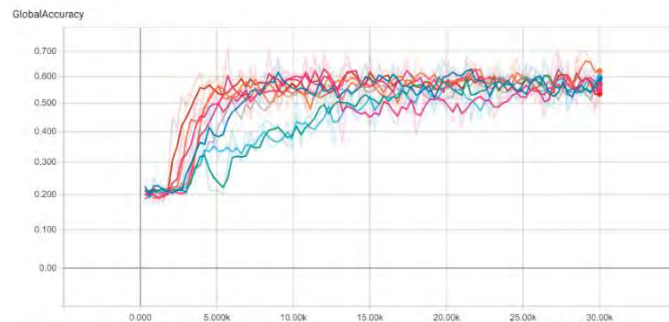
Hypothese H1

Zur Beantwortung der ersten aufgestellten Hypothese sind pro Textkorpus-Größe jeweils 9 differentielle Modelle trainiert worden. Dabei wurden sowohl Word2Vec als auch GloVe als Verfahren zur Erzeugung der Wortvektoren verwendet.

- **GloVe**

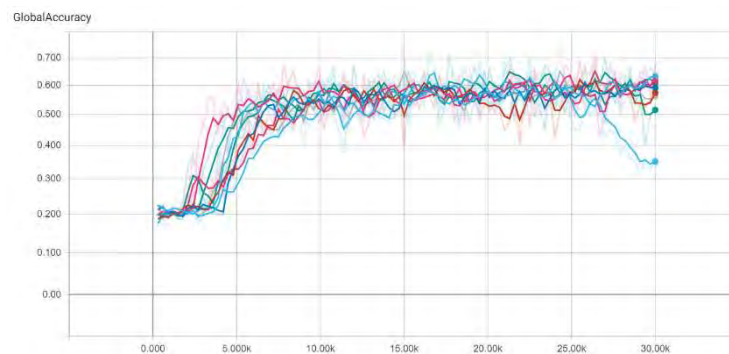
Die nachfolgende Grafik illustriert die erzielten Trainingsergebnisse für das Word Embedding-Verfahren GloVe bei einer Größe des Trainingskorpus von 150 Megabyte. Das Diagramm zeigt auf, dass sich der durchschnittliche Prädiktionswert der Modelle zwischen 50 und 60 Prozent befindet. Zur präziseren Ergebnisanalyse wird die bereits beschriebene Funktion ‚Accuracy of Model‘ der Evaluierungsplattform genutzt. Die Funktion bildet bei der Beurteilung jeweils einen Mittelwert pro Modell. Basierend auf den neun gesammelten Mittelwerten wurde ein finaler Durchschnittswert berechnet. Für die Größe 150 Megabyte liegt dieser bei 60,114 Prozent.

Abbildung 14: H1a - Prädiktionsmessung von GloVe mit 150 Megabyte



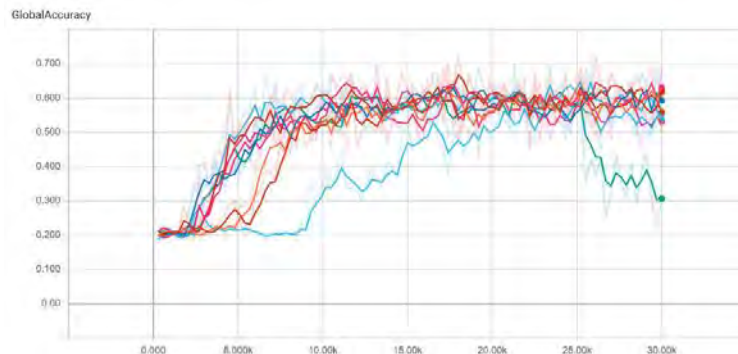
Das nachfolgende Diagramm illustriert die Prädiktion für Trainingsmodelle, die auf Trainingskörpern mit einem Umfang von 300 Megabyte basieren. Das Schaubild zeigt auf, dass die Prädiktion geringer ist als bei den 150 Megabyte großen Trainingskörpern. Der ermittelte durchschnittliche Prädiktionswert von 56,990 Prozent bestätigt dies.

Abbildung 15: H1a - Prädiktionsmessung von GloVe mit 300 Megabyte



Der gemessene durchschnittliche Accuracy-Wert der Trainingskörper mit einer Größe von 500 Megabyte liegt bei 58,712 Prozent. Die Abbildung zeigt den Prädiktionsverlauf während des Trainings. Darüber hinaus zeigt sie, dass insbesondere ein trainiertes Modell sehr niedrige Accuracy-Werte erreicht hat, was sich negativ auf den finalen Durchschnittswert auswirkt.

Abbildung 16: H1a - Prädiktionsmessung von GloVe mit 500 Megabyte

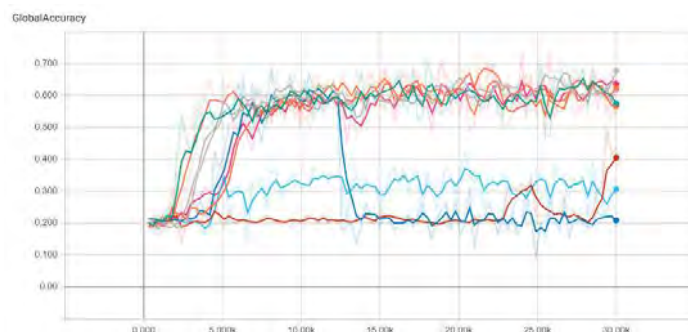


- **Word2Vec**

Für das Word Embedding-Verfahren Word2Vec sind ebenso wie für GloVe verschiedene Machine Learning-Modelle trainiert worden. Um eine Vergleichbarkeit zwischen dem Verfahren Word2Vec sowie GloVe zu schaffen, sind für beide Algorithmen identische Trainingskörper der englischsprachigen Wikipedia verwendet worden.

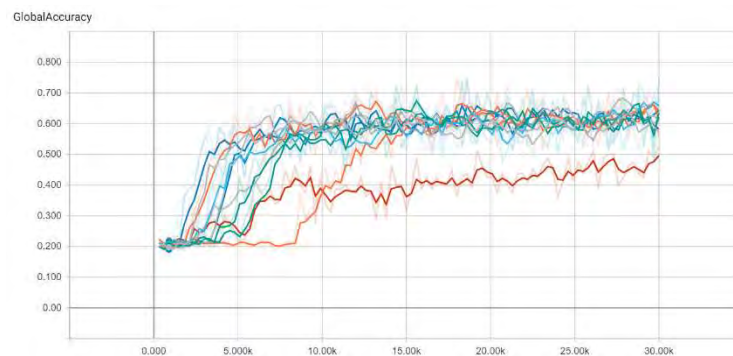
Die nachfolgende Abbildung zeigt die Prädiktion für Modelle an, bei denen die Wortvektoren auf Basis 150 Megabyte großer Trainingskörper trainiert wurden. Der durchschnittliche Prädiktionswert der Wortvektoren beträgt, basierend auf den Ergebnissen von neun unterschiedlichen Trainingskörpern, 54,439 Prozent. Für dieses Modell soll exemplarisch ein t-SNE-Datenplot (in Kapitel 4.4 ‚Visualisierung der erstellten Vektoren‘) durchgeführt werden, um die Möglichkeiten der Visualisierung von Word2Vec basierten Wortvektoren darzustellen. Dabei werden 10.000 Wörter mit einer Dimensionalität von 300 dargestellt. Der dazugehörige Datenplot findet sich im Anhang G.

Abbildung 17: H1b - Prädiktionsmessung von Word2Vec mit 150 Megabyte



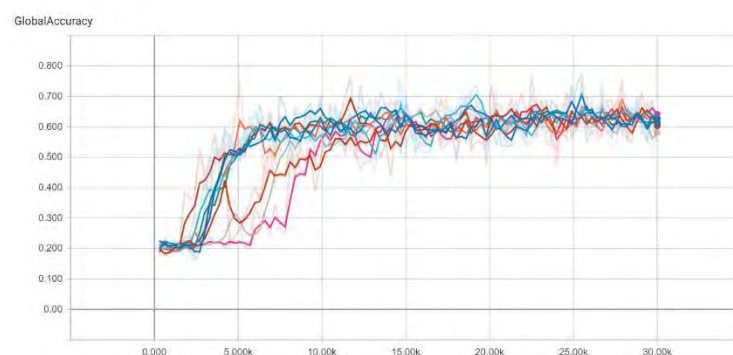
Bei 300 Megabyte großen Basiskörpern beträgt die durchschnittliche Accuracy 62,854 Prozent. Im Vergleich zu den erzeugten Modellen mit einem Umfang von 150 Megabyte kann somit ein Zuwachs innerhalb der Prädiktion von 8,415 Prozent vermerkt werden. Die anschließende Abbildung visualisiert den Trainingsverlauf der Modelle, dessen Basis auf 300 Megabyte großen Textkörpern basiert.

Abbildung 18: H1b - Prädiktionsmessung von Word2Vec mit 300 Megabyte



Für die mit Word2Vec-Wortvektoren mit Trainingskörpern der Größe 500 Megabyte erstellten Evaluierungsmodelle konnte ein durchschnittlicher Prädiktionswert von 65,914 Prozent berechnet werden. Somit konnten mit den umfangreichsten Textkörpern die besten Resultate erzielt werden. Ob der Unterschied zwischen den verschiedenen Korpus-Größen auch ausreichend signifikant ist, um die Hypothese als bestätigt zu klassifizieren, soll im nachfolgenden Kapitel näher betrachtet werden.

Abbildung 19: H1b - Prädiktionsmessung von Word2Vec mit 500 Megabyte



Hypothese H2

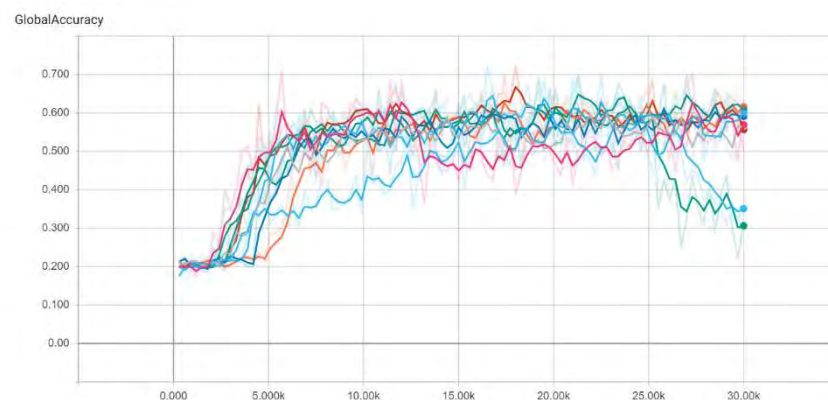
Die Hypothese H2 befasst sich mit dem Einfluss der Anzahl der Iterationen beim Training von Wortvektoren auf die Prädiktion der Machine Learning-Modelle. Zur Evaluierung dieser Hypothese wurden neun verschiedene Modelle pro Vergleichskomponente sowie

pro Word Embedding-Verfahren trainiert. Ebenso wie bei der Evaluierung von Hypothese H1 wird auch für Hypothese H2 die Funktionalität ‚Accuracy of Model‘ der Trainingsplattform mit einem Durchlauf von 1000 Prädiktionsberechnungen verwendet. Im Nachgang wird dann basierend auf den gebildeten Mittelwerten der jeweiligen Modelle ein Gesamtmittelwert pro Vergleichskomponente gebildet. Bei der Hypothese H2 erfolgt eine Gruppierung der Resultate nach der Anzahl der Trainingsiterationen der Wortvektoren.

- **GloVe**

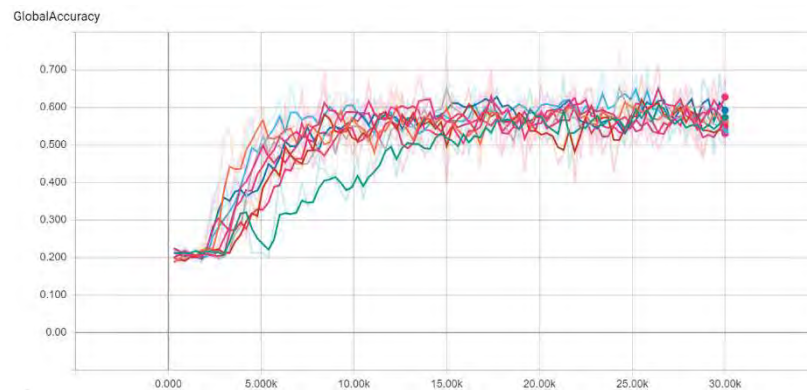
Bei den mit GloVe trainierten Modellen konnte ein durchschnittlicher Prädiktionswert von 53,931 Prozentpunkten bei 15 Trainingsiterationen ermittelt werden. Die nachfolgende Grafik zeigt den Trainingsfortschritt, gemessen an 100 Datenpunkten.

Abbildung 20: H2a -Prädiktionsmessung von GloVe mit 15 Iterationen



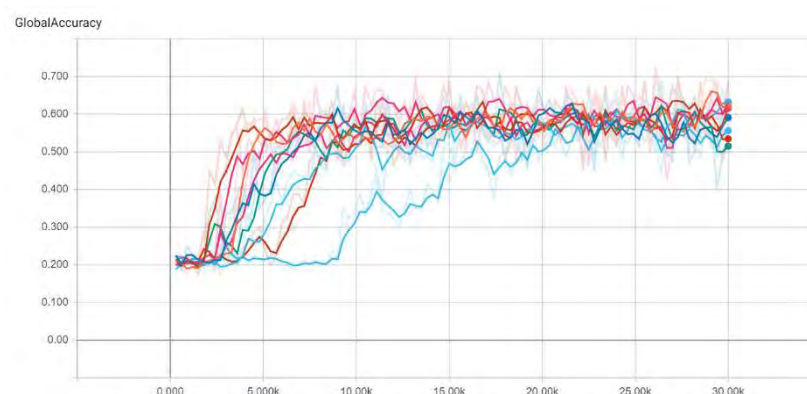
Der errechnete Mittelwert für die neun Modellen, welche beim Training 50 Iterationen durchlaufen haben, beträgt 60,799 Prozent. Anhand der folgenden Abbildung aus dem Tensorboard lässt sich erkennen, dass es im Gegensatz zu den Trainings mit 15 Iterationen keine elementaren Abweichungen zwischen den verschiedenen Modellen bestehen.

Abbildung 21: H2a - Prädiktionsmessung von GloVe mit 50 Iterationen



Für die Machine Learning-Modelle, bei denen trainierte Wortvektoren mit einer Trainingslänge von 100 Trainingsiterationen verwendet wurden, konnte ein Prädiktwert im Mittel von 61,087 Prozent kalkuliert werden. Der Trainingsverlauf kann dem nachfolgenden Bild entnommen werden.

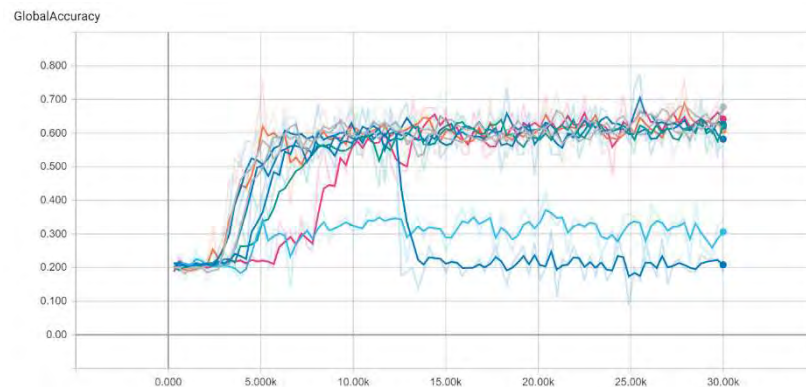
Abbildung 22: H2a - Prädiktionsmessung von GloVe mit 100 Iterationen



- **Word2Vec**

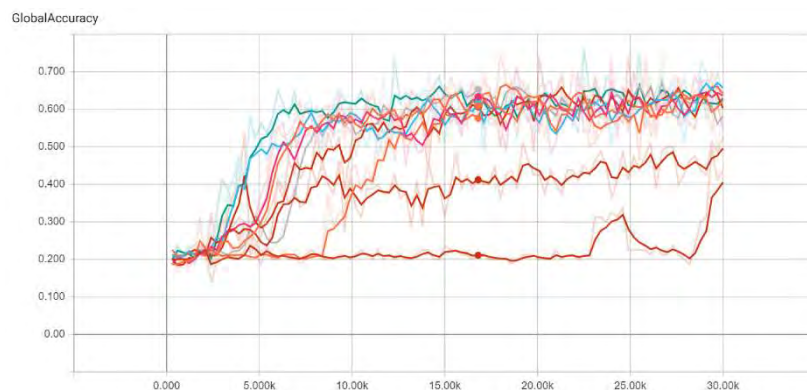
Ebenso wie für GloVe wurde die aufgestellte Hypothese H2 für Word2Vec anhand von ermittelten Prädiktwerten untersucht. Für die neun ausgebildeten Modelle, die aus mit 15 Iterationen-Durchläufen gebildeten Wortvektoren bestehen, konnte eine mittlere Gesamtprädiktion von 56,556 Prozentpunkten gemessen werden (siehe Abbildung 25).

Abbildung 23: H2b - Prädiktionsmessung von Word2Vec mit 15 Iterationen



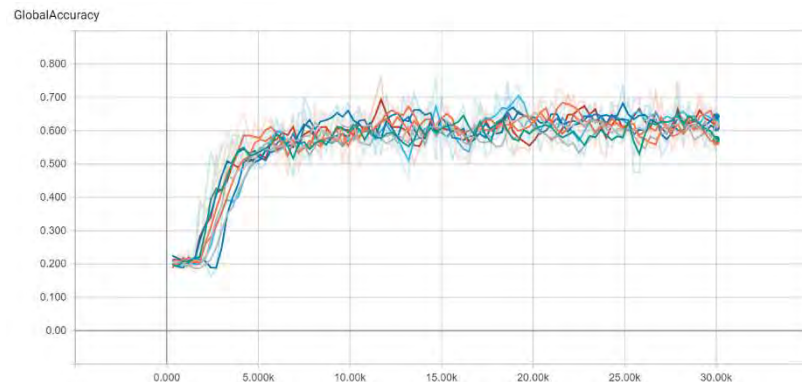
Bei den auf 50 Iterationen basierenden Wortvektor-Modellen kann ein durchschnittlicher Accuracy-Wert von 62,575 Prozent vermerkt werden. Es kann eine Differenz von 6,019 Prozentpunkten im Vergleich zu den auf 15 Iterationen basierenden Modellen erfasst werden. Die nachfolgende Abbildung lässt deutliche Unterschiede des Verlaufes während des Trainings erkennen. Insbesondere zwei Modelle erzielen geringere Richtigkeitswerte, was negativen Einfluss auf die Gesamtprädiktion hat.

Abbildung 24: H2b - Prädiktionsmessung von Word2Vec mit 50 Iterationen



Die anschaulich besten Werte erzielen die Modelle, bei denen Wortvektoren verwendet werden, die mit 100 Iterationen trainiert sind. Dabei kann insbesondere erkannt werden, dass keines der Modelle besondere Abweichungen aufweist. Der mittlere Prädiktionswert liegt für die neun trainierten Modelle bei 64,177 Prozent. Für das Verfahren Word2Vec können somit mit 100 Trainingsiterationen die besten Prädiktionswerte erzielt werden. Die folgende Abbildung stellt den Trainingsverlauf gemessen an 100 Datenpunkten dar.

Abbildung 25: H2b - Prädiktionsmessung von Word2Vec mit 100 Iterationen



6.2 Statistische Ergebnisevaluation

6.2.1 Durchführung einer Varianzanalyse zur Hypothesenuntersuchung

Nachdem die erzielten Prädiktionsergebnisse in Kapitel 6.1 ‚Präsentation der Ergebnisse‘ aufgezeigt wurden, soll nun die Signifikanz dieser Resultate statistisch betrachtet werden. Für die Prüfung der Unterschiede hinsichtlich ihrer Signifikanz wird eine Varianzanalyse verwendet. Diese ist ein Werkzeug der Statistik, anhand dessen untersucht werden kann, ob sich die Mittelwerte mehrerer Gruppen signifikant voneinander unterscheiden. Anders als beim t-Test ist es mit der Varianzanalyse möglich, mehr als zwei unabhängige Gruppen miteinander zu vergleichen.

Grundsätzlich kann zwischen einer ein- und zweifaktoriellen Varianzanalyse unterschieden werden. Bei der einfaktoriellen wird ein Faktor in die Überprüfung einbezogen²³⁴, wie beispielsweise die Trainingsiteration. Bei einer zweifaktoriellen Analyse kann ein zweiter Untersuchungsfaktor hinzugezogen werden²³⁵, wie beispielsweise der Umfang des Trainingskorpus. Somit kann der Einfluss mehrerer Faktoren auf das Ergebnis untersucht werden. Für die vorliegende Validierung der Hypothesen soll die einfaktorielle Varianzanalyse verwendet werden, da für jede Hypothese der Einfluss jeweils nur für einen Faktor analysiert werden soll. Die Analyse mittels Varianzanalyse wird pro Hypothese und für jedes Word Embedding-Verfahren durchgeführt.

²³⁴ Vgl. (Scheffe, 1959), Seite 55-60.

²³⁵ Vgl. (Scheffe, 1959), Seite 90-98.

Hypothese H1

- **GloVe**

Die sich im Anhang H befindende Abbildung zeigt die Ergebnisse der Evaluierung im Boxplot-Format. Durch diese Form der Darstellung soll der Unterschied der Resultate hervorgehoben werden. Somit können bereits erste Schlüsse über das Ergebnis der Varianzanalyse gezogen werden. Wie die Illustration zeigt, bestehen Unterschiede zwischen den Gruppierungen. Auffällig ist insbesondere, dass die 300 Megabyte großen Modelle niedrigere Prädiktionswerte aufweisen als die Modelle mit einem Umfang von 150 Megabyte und trotzdem ein höherer Mittelwert dargestellt wird.

Darüber hinaus zeigt die Darstellung die großen Abweichungen einzelner Modelle als vereinzelte Punkte. Modelle mit abweichenden Prädiktionswerten werden dem Anschein nach nur bei Trainingskörpern mit größerem Umfang erzeugt.

Abbildung 26: H1a - Varianzanalyse für GloVe

```
> #perform anova
> summary(aov(h1_training_corpus_size_anova$Accuracy ~ h1_training_corpus_size_anova$Corpus_Size))
```

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
h1_training_corpus_size_anova\$Corpus_Size	1	6.1	6.12	0.081	0.778
Residuals	25	1882.5	75.30		

```
>
```

Die Varianzanalyse zur Evaluierung der Hypothese H1 für den GloVe-Algorithmus zeigt, dass der untersuchte Faktor ‚Umfang des Trainingskorpus‘ keinen signifikanten Einfluss auf die erzielten Accuracy-Werte der Modelle hat. Dies kann erkannt werden, da bei oben aufgeführter Varianzanalyse der p-Wert 0.778 größer ist als das verwendete Signifikanzniveau. Die Bedingung $p\text{-Wert} > \alpha$ ist somit erfüllt und das Ergebnis nicht signifikant differentiell.

- **Word2Vec**

Anders als beim GloVe-Algorithmus können in der Abbildung, die sich ebenso im Boxplot-Format im Anhang I befindet, Differenzen zwischen den verschiedenen Größen der Trainingskörper erkannt werden. Zu erwähnen ist insbesondere die abnehmende Breite der Prädiktionswerte. Ist die Streuung bei den Modellen mit Körpern von 150 Megabyte 47,825 Prozentpunkte groß, so beträgt sie bei den auf 500 Megabyte großen Texten nur 2,556 Prozent. Dies entspricht einem Rückgang der Streuung von 45,269

Punkten. Durch den Rückgang der Streuung kann darauf geschlossen werden, dass der Umfang der Basiskörper Einfluss auf die Prädiktion der Modelle haben kann.

Abbildung 27: H1b - Varianzanalyse für Word2Vec

```
> #perform anova
> summary(aov(h1_training_corpus_size_anova$Accuracy ~ h1_training_corpus_size_anova$Corpus_Size))
```

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
h1_training_corpus_size_anova\$Corpus_Size	1	315.4	315.40	7.518	0.0111 *
Residuals	25	1048.8	41.95		

```
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Die durchgeführte Varianzanalyse bestätigt diese These (siehe vorherige Abbildung). Da der berechnete p-Wert von 0.0111 die Bedingung $p\text{-Wert} \leq \alpha$ erfüllt, kann der Nullhypothese, sprich sämtliche einbezogene Mittelwerte sind gleich, widersprochen werden. Die untersuchte Variable ‚Umfang der Trainingskörpers‘ hat Einfluss auf die Prädiktion der Modelle. Diese Aussage kann mit einem Risiko von fünf Prozent getroffen werden, da die Varianzanalyse ein Signifikanzniveau von 0,05 aufweist.

Hypothese H2

- GloVe

Für die Betrachtung der Hypothese H2 wurde zunächst ein Boxplot-Diagramm erstellt (siehe Anhang J), das die Differenzen zwischen den unterschiedlichen Betrachtungswerten der Vergleichsvariable Iteration grafisch darstellt. Auf dem Schaubild ist zu erkennen, dass die Modelle, bei denen die Wortvektoren mit 50 Iterationen trainiert wurden, den höchsten Median aufzeigen. Allerdings ist die Streuung der erzielten Prädiktionswerte für die Modelle mit 100 Trainingsdurchläufen am geringsten.

Abbildung 28: H2a - Varianzanalyse für GloVe

```
> #perform anova
> summary(aov(h2_training_iterations_anova$Accuracy ~ h2_training_iterations_anova$Iterations))
```

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
h2_training_iterations_anova\$Iterations	1	204.1	204.07	3.029	0.0941 .
Residuals	25	1684.5	67.38		

```
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Das Ergebnis der Varianzanalyse zeigt, dass die Differenz zwischen den gebildeten Mittelwerten der Prädiktion signifikant ist, da der p-Wert von 0.0941 kleiner beziehungsweise gleich dem Signifikanzniveau von 0.1 ist. Somit kann die Bedingung $p\text{-Wert} \leq \alpha$ als erfüllt angesehen werden. Es besteht ein Risiko von 10 Prozent, dass die

gemessene Differenz der Vergleichswerte tatsächlich nicht vorhanden ist, da ein eher schwaches Signifikanzniveau von 0.1 erzielt wurde. Nichtsdestotrotz kann basierend auf den Ergebnissen der Analyse geschlussfolgert werden, dass die Anzahl der Iterationen Einfluss auf die Prädiktion der Amazon Machine Learning-Modelle hat.

- **Word2Vec**

Ebenso wie bei den GloVe-Modellen zeigt das Boxplot-Diagramm (siehe Anhang K) an, dass der höchste Median den Modellen mit Wortvektoren mit 50 Trainingsdurchläufen zugeordnet werden kann. Auch beim Word2Vec-Algorithmus zeigen die Vektoren mit 100 Iterationen die geringste Streuung der Ergebnisse auf.

Abbildung 29: H2b - Varianzanalyse für Word2Vec

```
> #perform anova
> summary(aov(h2_training_iterations_anova$Accuracy ~ h2_training_iterations_anova$Iterations))
```

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
h2_training_iterations_anova\$Iterations	1	93.6	93.63	1.842	0.187
Residuals	25	1270.6	50.82		

Anders als bei den Modellen des GloVe-Verfahrens sind die Differenzwerte zwischen den Durchschnittswerten der Vergleichswerte zu gering, um Signifikanz aufzuweisen. Mit 0.187 liegt der kalkulierte p-Wert unterhalb des Signifikanzniveaus. Somit ist die Bedingung $p\text{-Wert} > \alpha$ erfüllt und es kann geschlussfolgert werden, dass keine Anzeichen für signifikante Unterschiede zwischen den Prädiktionswerten der Modelle bestehen.

6.2.2 Vergleich der Datenrelationen mittels Post-Hoc-Test

Da die Varianzanalyse keinen Aufschluss über die Unterschiede der einzelnen Untersuchungsgruppen untereinander gibt, soll zusätzlich zur in Kapitel 6.2.1 ‚Durchführung einer Varianzanalyse‘ durchgeführten Varianzanalyse ein Post-Hoc-Test nach Tukey erfolgen.

Grundsätzlich bestimmt die Varianzanalyse, ob Werte signifikant voneinander abweichen. Nicht aber welche Werte welcher Vergleichsgruppen sich untereinander unterscheiden. Der von John Tukey im Jahr 1953 entwickelte Honestly Significant Difference-(HSD)Test führt einen paarweisen Vergleich von Gruppen durch und kann somit dabei behilflich sein, die Differenzen zwischen den jeweiligen Vergleichsgruppen zu betrachten²³⁶. Im Rahmen der vorliegenden Arbeit soll der paarweise Vergleich

²³⁶ Vgl. (Abdi & Williams, 2010), Seite 1.

durchgeführt werden, um Aufschluss über die Unterschiede zwischen den jeweiligen Gruppen zu geben. Der Vergleich wird für die Ergebnisse der Varianzanalyse vollzogen, die einen signifikanten Unterschied aufweisen.

Hypothese H1

Da bei den Ergebnissen der Varianzanalyse für Hypothese H1 nur das Verfahren Word2Vec signifikante Unterschiede innerhalb der Prädiktion aufweist, wird ausschließlich Word2Vec mittels Post-Hoc-Test näher betrachtet.

Abbildung 30: H1 - Post-Hoc-Test für Word2Vec

```
> TukeyHSD(aov(h1_training_corpus_size_post_hoc$Accuracy ~ h1_training_corpus_size_post_hoc$Corpus_Size))
Tukey multiple comparisons of means
95% family-wise confidence level

Fit: aov(formula = h1_training_corpus_size_post_hoc$Accuracy ~ h1_training_corpus_size_post_hoc$Corpus_Size)

$h1_training_corpus_size_post_hoc$Corpus_Size
      diff      lwr      upr    p adj
H1_300-H1_150 5.565778 -2.1351759 13.26673 0.1893885
H1_500-H1_150 8.524444  0.8234908 16.22540 0.0280516
H1_500-H1_300 2.958667 -4.7422870 10.65962 0.6088970
```

Anhand der Spalte diff des Post-Hoc-Tests kann die Differenz zwischen den Mittelwerten der jeweiligen Gruppen erkannt werden. Bei der Hypothese H1 ist zu erkennen, dass die Differenz der Mittelwerte beim Vergleich der Gruppen H1_500 und H1_150 am größten ist. In diesem Fall beträgt der Unterschied zwischen der Größe 500 und 150 Megabyte 8,5244, sprich die Modelle mit Wortvektoren zu 500 Megabyte erzielen eine höhere Prädiktion als die auf 150 Megabyte basierenden Machine Learning-Modelle. Ob der Unterschied zwischen den Gruppen signifikant ist, lässt sich am Wert der Spalte p adj ablesen. Liegt der berechnete p-Wert unterhalb des Signifikanzniveaus von 0.05, so gilt die Differenz beider Gruppen als signifikant. In diesem Fall beträgt der p-Wert 0.028. Somit ist die Bedingung $p\text{-Wert} \leq \alpha$ erfüllt.

Hypothese H2

Für die Hypothese H2 konnten mit dem Verfahren GloVe bedeutsame Unstimmigkeiten zwischen den Resultaten analysiert werden. Folglich sollen die Unterschiede zwischen den einzelnen Vergleichsgruppen durch den Post-Hoc-Test nach Tukey näher betrachtet werden.

Abbildung 31: H2 - Post-Hoc-Test für GloVe

```

> TukeyHSD(aov(h2_training_iterations__post_hoc$Accuracy ~ h2_training_iterations__post_hoc$Iterations))
Tukey multiple comparisons of means
 95% family-wise confidence level

Fit: aov(formula = h2_training_iterations__post_hoc$Accuracy ~ h2_training_iterations__post_hoc$Iterations)

$h2_training_iterations__post_hoc$Iterations
      diff       lwr       upr      p adj
H2_15-H2_100 -7.1561111 -16.747787  2.435564 0.1711996
H2_50-H2_100 -0.2878889  -9.879564  9.303787 0.9969078
H2_50-H2_15   6.8682222  -2.723453 16.459898 0.1948574

```

Obwohl die Varianzanalyse einen signifikanten Unterschied zwischen den Vergleichsdaten aufzeigt, bestätigt der Post-Hoc-Test dieses Ergebnis nicht. Mit der Analyse des Post-Hoc-Tests kann festgestellt werden, dass die Vergleichsgruppen im jeweiligen Vergleich keine signifikanten Differenzen untereinander aufweisen. Die höchste Prädiktionsdifferenz zwischen den Gruppen besteht bei H2_15 und H2_100 mit 7,156 Prozent. Der errechnete p-Wert liegt jedoch bei 0.1711 und damit oberhalb des Signifikanzniveaus. Die Bedingung $p\text{-Wert} > \alpha$ ist dementsprechend erfüllt und folglich kann keine Signifikanz zwischen den Vergleichsgruppen determiniert werden.

6.2.3 Anwendung eines t-Tests zum Verfahrensvergleich

Zur Beantwortung der Leitfrage L1 sind insgesamt 27 verschiedene Modelle pro Word Embedding-Verfahren trainiert worden. Basierend auf den erzielten Prädiktionswerten sollen beide Verfahren miteinander verglichen werden. Ziel des Vergleiches ist es festzustellen, ob der Unterschied der Prädiktion signifikant ist, und darüber hinaus, mit welchem Verfahren die höchsten Prädiktionswerte erzielt werden können.

Für den Vergleich der beiden Verfahren soll ein t-Test als statistisches Vergleichsinstrument verwendet werden. Mithilfe des t-Tests können Mittelwerte beim Vergleich hinsichtlich Signifikanz geprüft werden. Dabei wird allgemein zwischen einem Einstichproben-t-Test und einem Zweistichproben-t-Test unterschieden. Beim einstichproben t-Test können Mittelwerte innerhalb einer einzelnen Vergleichsgruppe überprüft werden. Durch den zweistichproben t-Test können zwei Vergleichsgruppen, wie beispielsweise die Verfahren Word2Vec und GloVe, miteinander verglichen werden. Das Ergebnis des Tests gibt Aufschluss darüber, ob die Differenz zwischen den beiden Vergleichsgruppen signifikant ist. Zur Untersuchung der Leitfrage L1 soll ein zweistichproben t-Test verwendet werden.

Der sich im Anhang L befindende Boxplot stellt die gemessenen Ergebnisse beider Verfahren grafisch dar. Dabei ist zu erkennen, dass beim Verfahren GloVe die

Schwankungsbreite innerhalb der Prädiktionswerte größer ist als bei Word2Vec. Dennoch können bei Word2Vec mit fünf mehr Messausreißer identifiziert werden als bei GloVe. Diese Ausreißer liegen außerhalb des Interquartilsabstands. Grundsätzlich sind solche Messausreißer ein Indikator dafür, dass beim Training der Modelle etwas entgegen der Erwartungen ist. Der nachfolgende t-Test soll mehrfach durchgeführt werden.

Zunächst sollen die abweichenden Prädiktionswerte mitberücksichtigt werden. In einem zweiten t-Test sollen diese Werte für beide Verfahren aus dem Test ausgeschlossen werden. Somit entsteht ein sowohl bereinigtes Ergebnis, bei dem nur Werte innerhalb des Interquartilsabstands berücksichtigt werden, als auch ein tatsächliches Ergebnis, bei dem alle gemessenen Prädiktionswerte berücksichtigt werden sollen. Anhand des Boxplots lässt sich weiterhin erkennen, dass die Prädiktionswerte für das Verfahren Word2Vec tendenziell am größten sind, da sowohl im Mittel als auch im Maximum die höchsten Werte erzielt werden konnten.

Zunächst soll der t-Test für die vollständige Ergebnismenge durchgeführt werden.

Abbildung 32: L1 - t-Test mit unbereinigten Daten

```
> # perform t-Test
> t.test(Accuracy ~ Approach)

Welch Two Sample t-test

data: Accuracy by Approach
t = -1.6172, df = 50.683, p-value = 0.1121
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -7.8032384  0.8410903
sample estimates:
 mean in group GloVe mean in group Word2Vec
      58.60544         62.08652
```

Das Ergebnis des durchgeführten Tests zeigt, dass die Differenz zwischen den Verfahren, obwohl eine solche zwischen den beiden Mittelwert von 3,48108 Prozentpunkten besteht, nicht signifikant ist. Für den t-Test wurde zunächst die Nullhypothese aufgestellt, dass sich beide Verfahren nicht signifikant hinsichtlich der erzielten Prädiktionswerte voneinander unterscheiden. Dabei wird auf ein Konfidenzniveau von 95 Prozent zurückgegriffen. Sprich es wird angenommen, dass 95 Prozent der Proben die Differenz aus der Grundgesamtheit enthalten. Dieses Konfidenzniveau setzt die Software R beim t-Test als Standard.

Der ermittelte p-Wert von 0.1121 liegt oberhalb des zulässigen Signifikanzniveaus. Somit ist die Bedingung $p\text{-Wert} > \alpha$ erfüllt und die Nullhypothese wird nicht verworfen.

Abbildung 33: L1 - t-Test mit bereinigten Daten

```

> # perform t-Test
> t.test(Accuracy ~ Approach)

Welch Two Sample t-test

data: Accuracy by Approach
t = -8.617, df = 41.071, p-value = 9.438e-11
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -5.076480 -3.148847
sample estimates:
mean in group GloVe mean in group Word2Vec
60.89420             65.00686

```

Bei der Durchführung des t-Tests mit abweichungsbereinigten Ergebnissen ist die Differenz der erzielten Prädiktionswerten beider Verfahren signifikant. Mit einem kalkulierten p-Wert von $9.438e-11$ ist die Bedingung $p\text{-Wert} \leq \alpha$ erfüllt und der Wert liegt unterhalb des Signifikanzniveaus. Somit kann die Nullhypothese verworfen werden und die Differenz ist signifikant. Darüber hinaus verändern sich durch die Bereinigung die Mittelwerte beider Verfahren, sodass GloVe einen durchschnittlichen Wert von 60,894 Prozent und Word2Vec von 65,006 Prozent aufweisen. Somit kann festgestellt werden, dass für die Modelle, die auf Word2Vec basieren, bessere Ergebnisse hinsichtlich der Prädiktion erzielt werden konnten.

6.3 Validierung der Thesen

Für beide Hypothesen wurden jeweils Modelle für zwei differentielle Word Embedding-Verfahren trainiert. Die Trainingsergebnisse sind hinsichtlich der Prädiktion für beide Algorithmen einzeln betrachtet und im Anschluss statistisch begutachtet worden. Somit sollen auch beide Hypothesen zweifach validiert werden. Lediglich die Leitfrage L1 wird gesamtheitlich betrachtet, da ein direkter Vergleich zwischen den Word Embedding-Logiken Word2Vec und GloVe vorgenommen wird.

Somit ergibt sich aus der Hypothese H1 sowohl die Hypothese H1a als auch Hypothese H1b. Selbiges wird für die Hypothese H2 vorgenommen. Im Folgenden sollen die aufgestellten Hypothesen sowie die Leitfrage sukzessive validiert und anschließend interpretiert werden.

Die Tabelle 6 dient der vereinfachten Darstellung der Ergebnisse. Die Varianzanalyse zeigt, dass beide Parameter bei den verwendeten Verfahren unterschiedliche Ergebnisse erzielen.

Tabelle 6: Validierung der Hypothesen

Untersuchungs-gegenstand	Hypothese	Annahme	Beobachtung	Ergebnis Varianzanalyse	Ergebnis Post-Hoc
Korpusgröße GloVe	H1a	$p\text{-Wert} \leq \alpha$	$p\text{-Wert} > \alpha$	verworfen	n. relevant
Korpusgröße Word2Vec	H1b	$p\text{-Wert} \leq \alpha$	$p\text{-Wert} \leq \alpha$	bestätigt	bestätigt
Iterationen GloVe	H2a	$p\text{-Wert} \leq \alpha$	$p\text{-Wert} \leq \alpha$	bestätigt	verworfen
Iterationen Word2Vec	H2b	$p\text{-Wert} \leq \alpha$	$p\text{-Wert} > \alpha$	verworfen	n. relevant

n=nicht

- **Hypothese H1**

Bezogen auf das Verfahren GloVe hat der Parameter ‚Umfang der Trainingskörper‘ keinen Einfluss auf die Prädiktion. Die Modelle mit Wortvektoren aus Trainingskörpern von 150 Megabyte weisen, in Zahlen ausgedrückt, die besten Ergebnisse auf. Der prozentual gemessene Richtigkeitswert liegt hier bei 60,114 Prozentpunkten und ist somit höher als der für die mit 500 Megabyte trainierten Wortvektoren von 58,712 Prozent. Das Ergebnis der Varianzanalyse zeigt auf, dass für GloVe keine Signifikanz zwischen den Vergleichswerten besteht. Folglich wurde, basierend auf diesen Erkenntnissen, kein Post-Hoc-Test durchgeführt. Die Hypothese H1a kann somit verworfen werden.

Anders als beim GloVe-Algorithmus kann für Word2Vec ein signifikanter Unterschied zwischen den erzielten Prädiktionswerten durch die Varianzanalyse festgestellt werden. Die gemessenen Werte zeigen auf, dass mit Wortvektoren, die auf Textkörpern mit höherem Umfang trainiert werden, bessere Resultate erzielt werden können. Können für die Modelle, basierend auf 150 Megabyte großen Textkörpern, 54,439 Prozent Richtigkeit im Durchschnitt gemessen werden, so übertreffen die Werte der 300 Megabyte großen Trainingskörper diese mit einem Mittelwert von 62,955 Prozent. Die besten Resultate können mit 500 Megabyte großen Texten erzielt werden. Hier liegt der Prädiktionsmittelwert von neun Modellen bei 65,914 Prozent. Somit übertrifft dieser Wert ebenso den maximal erzielten Richtigkeitswert von GloVe. Der Post-Hoc-Test nach Tukey bestätigt die Resultate der Varianzanalyse und zeigt auf, dass in direkter Gegenüberstellung der Vergleichsgruppen H1_500 und H1_150 die höchste Differenz gemessen werden kann und dieser Unterschied signifikant ist. Nach beiden Tests kann die Hypothese H1b bestätigt werden.

- **Hypothese H2**

Der Trainingsparameter Iterationen hat beim Training mit GloVe Einfluss auf die Prädiktion. Das Ergebnis der Varianzanalyse zeigt auf, dass die Differenz zwischen den Gruppen und deren Resultaten signifikant ist. Der gebildete Mittelwert liegt für Modelle, die auf Wortvektoren mit einem Trainingsdurchlauf von 15 Iterationen trainiert wurden, bei 53,931 Prozent. Durch die Erhöhung der Trainingsiterationen auf 50 erhöht sich der durchschnittliche Prädiktionswert bei neun Modellen um 6,868 Prozentpunkte auf 60,799 Prozent. Eine weitere Aufstockung der Iterationen auf 100 steigert den mittleren Prädiktionswert bei exponentiell ansteigender Trainingszeit um gerade einmal 0,288 Prozent. Da durch die Varianzanalyse ein signifikanter Unterschied zwischen den Gruppen determiniert werden kann, wurde ein Post-Hoc-Test durchgeführt, um die Unterschiede zwischen den einzelnen Vergleichsgruppen zu beleuchten.

Der Post-Hoc-Test zeigt auf, dass sich der größte Unterschied hinsichtlich der Prädiktion zwischen den Gruppen H1_15 sowie H1_100 befindet, dieser jedoch nicht signifikant ist. Somit wird die Hypothese H2a trotz Ergebnis der Varianzanalyse sowie der Differenz der Prädiktionswerte verworfen, da die Signifikanz der Unterschiede zwischen den jeweiligen Betrachtungsgruppen nicht gegeben ist.

Für Word2Vec lassen sich Unterschiede bei den Messwerten feststellen. Bei Wortvektoren mit 15 Iterationen fällt der durchschnittliche Richtigkeitswert der Machine Learning-Modelle auf 56,556 Prozent. Wird die Anzahl der Durchläufe auf 50 gesteigert, kann ein mittlerer Prädiktionswert von 62,575 Prozent und so ein Anstieg von 6,019 Prozentpunkten erzielt werden. Durch eine weitere Erhöhung der Trainingsiterationen von 50 auf 100 kann der Accuracy-Wert bei verlängerter Trainingszeit um weitere 1,602 Punkte auf 64,177 Prozent verbessert werden. Die Diskrepanz zwischen den ermittelten Prädiktionswerten erfährt jedoch, wie die Varianzanalyse zeigt, keine Signifikanz und somit wird kein Post-Hoc-Test für die Hypothese H2b durchgeführt. Die Hypothese H2b wird aufgrund mangelnder Signifikanz verworfen.

- **Leitfrage**

Die Leitfrage wurde durch zwei t-Tests analysiert. Der erste t-Test wurde mit allen zur Verfügung stehenden Messungen beider Verfahren praktiziert. Die insgesamt 27

Messungen pro Verfahren enthalten stark abweichende Richtigkeitswerte, bei denen davon ausgegangen werden kann, dass das Training der Machine Learning-Modelle oder der Wortvektoren fehlerbehaftet war.

Der mit jeweils 27 unbereinigten Modellen durchgeführte t-Test liefert einen p-Wert von 0.1121 sowie einen mittleren Prädiktionswert von 58,605 Prozent für GloVe und 62,086 Prozent für das Verfahren Word2Vec. Der errechnete Unterschied von 3,481 Prozentpunkten ist dem p-Wert nach nicht signifikant.

Das Ergebnis des zweiten t-Tests, bei dem nur Daten verwendet wurden, die innerhalb des Interquartilsabstands des Boxplots liegen, liefert einen p-Wert von $9.438e-11$. Die durchschnittlichen Prädiktionswerte entsprechen hier 60,894 Prozent für das GloVe-Verfahren sowie 65,006 Prozent für Word2Vec. Somit ergibt sich eine Differenz von 4,112 Prozentpunkten. Der t-Test zeigt mittels kalkulierte p-Wert auf, dass der Unterschied zwischen den Prädiktionswerten beider Verfahren signifikant ist.

Da davon ausgegangen wird, dass die abweichenden Werte auf fehlerhaften Trainings der einzelnen Komponenten beruhen, wird zur Beantwortung der Leitfrage das Resultat des zweiten t-Tests zurate gezogen. Da der Differenzbetrag zwischen den Verfahren signifikant ist, kann die Leitfrage wie folgt beantwortet werden: Das Verfahren Word2Vec liefert, gemessen an der Prädiktion, bessere Ergebnisse bei der Anwendung auf Onlineproduktbewertungen des Unternehmens Amazon.

7. Diskussion der Resultate

7.1 Wissenschaftliche Betrachtung

Da die empirische Untersuchung in Kapitel 6. ‚Empirische Untersuchung‘ abgeschlossen ist, sollen nun die erzielten Ergebnisse mit bereits vorhandenen wissenschaftlichen Erkenntnissen abgeglichen werden.

- **Einfluss der Textgröße auf die Prädiktion**

Hinsichtlich des Einflusses der Textgröße auf die Prädiktion von Wortvektoren kann die Aussage von Pennington et al. (2014) und Chiu et al. (2016), dass die Größe des Trainingskorpus nicht pauschal Einfluss auf die Prädiktion hat, bestätigt werden^{237 238}. Wie in die Untersuchungen dieser Arbeit zeigen, wirkt sich die Größe der Trainingstexte bei GloVe sogar negativ auf die Prädiktion aus. Bei Anwendung des Verfahren Word2Vec zeigen die Tests jedoch, dass der Umfang der Texte Einfluss auf die Richtigkeit der Wortvektoren, bezogen auf die Onlineproduktbewertungen, hat und bestätigen somit die Studien von Al-Amin et al. (2017) und Du et al. (2017)^{239 240}.

- **Einfluss der Trainingsiterationen auf die Prädiktion**

Die Erkenntnisse von Hellrich und Hahn (2016) sowie Dekhtyar und Fong (2017), dass durch mehr Trainingsdurchläufe die Accuracy verbessert wird, können grundsätzlich durch die eigenen Proben bestätigt werden^{241 242}. Eine Erhöhung des Hyperparameters Iteration kann dazu beitragen, eine Verbesserung der Prädiktion der Wortvektoren, bezogen auf die Amazon-Onlineproduktbewertungen, zu erzielen. Für beide Verfahren kann dieser Effekt beobachtet werden. Auch zeigt sich, dass eine Steigerung insbesondere zwischen 15 und 50 Iterationen erzielt werden kann, da sich die Accuracy ab etwa 50 Iterationen stabilisiert. Somit können die Studien von Savigny und Purwarianti (2017)

²³⁷ Vgl. (Pennington et al., 2014), Seite 1540.

²³⁸ Vgl. (Chiu, Crichton, Korhonen, & Pyysalo, 2016), Seite 166.

²³⁹ Vgl. (Al-Amin et al., 2017), Seite 189.

²⁴⁰ Vgl. (Du, Zheng, Xiao, & Yin, 2017), Seite 264.

²⁴¹ Vgl. (Hellrich & Hahn, 2016), Seite 2791 f.

²⁴² Vgl. (Dekhtyar & Fong, 2017), Seite 489.

sowie Aghahadi und Talebpour (2018), dass sich die Prädiktion ab einer bestimmten Anzahl an Trainingsdurchläufen nicht signifikant verändert, bestätigt werden^{243 244}.

- **Verfahrensvergleich von GloVe und Word2Vec**

Entgegen den Erkenntnissen von Sravani et al. (2018)²⁴⁵ und Pennington et al. (2014)²⁴⁶, kann durch die Tests dieser Arbeit aufgezeigt werden, dass das Verfahren Word2Vec höhere Richtigkeitswerte liefert als GloVe. Allerdings ist zu beachten, dass die gesammelten Prädiktionswerte der erzeugten Modelle zunächst von Datenerfassungen bereinigt werden mussten, die außerhalb des Interquartils liegen. Die Auffassung von Bengio et al. (2014)²⁴⁷, Stratos et al. (2015)²⁴⁸ sowie TH et al. (2015)²⁴⁹, dass prediction-based Ansätze bessere Resultate hinsichtlich der Accuracy erzielen, kann bestätigt werden. Katic und Milicevic (2018) zeigen in ihren Studien auf, dass mit dem GloVe-Algorithmus bessere Richtigkeitswerte bei der Evaluierung mit Amazon-Produktbewertungen erzielt werden können als mit Word2Vec²⁵⁰. Dieser Studie kann durch eigene Tests widersprochen werden, da mit Word2Vec eine höhere Prädiktion erzielt werden konnte. Somit können die Ergebnisse der Studie von Wint et al. (2018), die ebenfalls die Accuracy beider Verfahren für Amazon-Onlineproduktbewertungen analysiert haben²⁵¹, bestätigt werden.

7.2 Implikationen für die Praxis

- **Bedeutung für die automatisierte Textverarbeitung**

Word Embedding-Verfahren und die daraus resultierenden Wortvektoren bilden die Grundlage für die Textanalyse. Grundsätzlich werden Wortvektoren immer dann benötigt, wenn zusammenhängende Texte oder auch einzelne Wörter automatisch weiterverarbeitet werden und als Input für beispielsweise NLP-Anwendungen oder neuronale Netze dienen sollen. Da die Texte für eine automatische Verarbeitung in ein

²⁴³ Vgl. (Savigny & Purwarianti, 2017), Seite 4.

²⁴⁴ Vgl. (Aghahadi & Talebpour, 2018), Seite 306.

²⁴⁵ Vgl. (Sravani et al., 2018), Seite 2380.

²⁴⁶ Vgl. (Pennington et al., 2014), Seite 1537-1541.

²⁴⁷ Vgl. (Baroni et al., 2014), Seite 242 f.

²⁴⁸ Vgl. (Stratos et al., 2015), Seite 1289.

²⁴⁹ Vgl. (TH, Sahu, & Anand, 2015), Seite 161.

²⁵⁰ Vgl. (Katic & Milicevic, 2018), Seite 287 f.

²⁵¹ Vgl. (Wint, Manabe, & Aritsugi, 2018), Seite 95.

Format umgewandelt werden müssen, das der Computer interpretieren kann, bedarf es einer Methodik, die Wörter mathematisch beziehungsweise verarbeitbar darstellt. Zudem müssen bei der automatischen Verarbeitung oftmals semantische Beziehungen zwischen Wörtern berücksichtigt werden, um diese korrekt zu interpretieren. Hauptaufgabe des Word Embedding ist es, Relationen zwischen Wörtern darzustellen.

- **Universelle Anwendungsmöglichkeiten**

Die Wortvektoren lassen sich in unterschiedlichen Anwendungen nutzen. Eine Anwendungsmöglichkeit stellt die Klassifikationsmöglichkeit von Rezensionen hinsichtlich Nützlichkeit dar. Darüber hinaus finden Word Embedding-Verfahren Anwendung in vielen verschiedenen Gebieten der Empirie, wie beispielsweise der Medizin, Biologie oder Linguistik.

Obwohl das Kernanwendungsgebiet sicherlich innerhalb der Textverarbeitung liegt, finden sich beispielsweise auch Anwendungen innerhalb der Audioverarbeitung²⁵². Aufgrund der breitgestreuten Anwendungsmöglichkeiten ist es elementar, dass die auf Word Embeddings gebildete Basis bestmöglich trainiert ist, um die Ergebnisse der Anwendungsfälle nicht negativ zu beeinflussen, sondern möglichst durch Präzision zu unterstützen.

7.3 Reflexion des Vorgehens

- **Implementierung**

Durch die Planung der Software durch die im Software Engineering üblichen UML-Diagramme können bereits im Vorfeld logische Programmabläufe durchdacht werden. Des Weiteren dient die Verwendung dieser Methodik dazu, eine strukturierte sowie geplante Programmierung zu gewährleisten. Im Rahmen dieser Arbeit wird ein UML-Diagramm für jede Programmentwicklung angefertigt. Dabei sollen nur UML-Anwendungsfall-Diagramme verwendet werden, da diese den Programmablauf adäquat abbilden können. Da die Programme im funktionalen Stil implementiert sind, werden keine UML-Klassendiagramme gebraucht.

²⁵² Vgl. (Shen, Sung, & Lee, 2018b), Seite 2231.

Die UML-Diagramme erwiesen sich als sehr hilfreich, da viele Fehlerszenarien im Vorfeld durchdacht werden konnten. So konnte insbesondere Zeit bei der Entwicklung und dem Test der Implementierungen eingespart werden. Durch die Konvertierung der Programme vom Python-Format zu ausführbaren Dateien konnte eine Möglichkeit geschaffen werden, die Tools auf jedem Computer zu verwenden, ohne dass die Runtime-Umgebung Python installiert sein muss. Die Parametrisierung der Programme über eine externe INI-Datei gestaltet sich als sehr hilfreich, denn so können dem Verwender ausgewählte Parameter einfach administrierbar zur Verfügung gestellt werden. Darüber hinaus muss für eine Änderung der Parametrisierung kein Programmcode genutzt werden, sodass auch Benutzer ohne Programmierkenntnisse Gebrauch von den Tools machen können.

- **Training**

Die Word Embeddings werden in dieser Arbeit ausschließlich durch die Verfahren GloVe und Word2Vec gebildet. Interessant wäre es, noch weitere Word Embedding Algorithmen, wie beispielsweise FastText, zu untersuchen. Beim Training werden ausschließlich Wortvektoren mit 300 Dimensionen verwendet. Hier wäre es lohnenswert zu untersuchen, ob sich die Prädiktion bei Wortvektoren mit größeren Dimensionen steigert. Darüber hinaus basieren alle gebildeten Word Embeddings auf Texten von Wikipedia. Interessant wäre es neben dem Einfluss der Textgröße auch die Auswirkungen der Spezifität der Basistexte auf die Prädiktion zu analysieren. Nach Aussage von Dusserre und Padro (2017) können mit spezifischeren Textkörpern bessere Resultate erzielt werden als mit größeren²⁵³. Hier könnte ein Wörterbuch auf Basis der Amazon-Rezessionen selbst gebildet werden.

- **Analyse**

Einen elementaren Punkt dieser Arbeit stellt die Literaturrecherche als Werkzeug der qualitativen Forschung dar. Durch diese sollen wichtige theoretische Grundlagen sowie der aktuelle Stand der Forschung aufgezeigt werden. Darüber hinaus wird dieses Werkzeug verwendet, um bestehende Erkenntnisse der Wissenschaft zu identifizieren. Dieses bereits bestehende Wissen dient dazu Hypothesen aufzustellen, die im Rahmen

²⁵³ Vgl. (Dusserre, 2017), Seite 4.

der vorliegenden Thesis untersucht werden sollen. Um die Aktualität sowie Qualität der herangezogenen Literatur und der darin enthaltenen Aussagen sicherzustellen, werden nur Artikel aus wissenschaftlichen Fachzeitschriften sowie Konferenzschriften verwendet.

Als weiterer wesentlicher Punkt gilt es die empirische Untersuchung hervorzuheben. Innerhalb der Untersuchung werden die aufgestellten Hypothesen sowie die Leitfrage mit Hilfe statistischer Werkzeuge untersucht. Dabei kommt sowohl eine Varianzanalyse mit anschließendem Post-Hoc-Test als auch ein t-Test zum Einsatz. Die einfaktorielle Varianzanalyse wird angewandt, da pro Hypothese drei Faktoren untersucht werden und die Zielvariable metrisch ist. Der t-Test wird in Form eines zweistichproben Tests durchgeführt, da zwei unabhängige Stichproben miteinander verglichen werden sollen.

7.4 Beschränkungen der Arbeit

- **Betriebssystem**

Da die Evaluierungsplattform nur für das Betriebssystem Windows zur Verfügung steht, werden beide Word Embedding-Algorithmen GloVe sowie Word2Vec ebenfalls für dieses Betriebssystem implementiert, um die Integrität der Plattform und der Verfahren zu gewährleisten. Die Implementierung unter Windows birgt jedoch Einschränkungen. GloVe ist von Pennington et al. (2014) für Systeme mit Unix-Kernel konzipiert²⁵⁴ und muss daher Cross-kompiliert werden. Da nicht alle Funktionen innerhalb der Programmiersprache C auch unter Windows verfügbar sind, müssen hierfür Alternativen gefunden werden, bei denen zum Teil Performancelücken in Kauf genommen werden. Der Word2Vec-Algorithmus ist ebenfalls in der Originalimplementierung von Mikolov et al. (2013) für Betriebssysteme mit Unix-Kernel konzipiert²⁵⁵. Als Alternative wird auf die Implementierung von Word2Vec innerhalb der Softwarebibliothek Gensim zurückgegriffen²⁵⁶.

- **Trainings- und Evaluierungsdaten**

Aufgrund der langen Laufzeit bei der Erstellung und dem Training der Wortvektoren, wird die Größe der Trainingskörper auf 150, 300 und 500 Megabyte beschränkt. Diese

²⁵⁴ Vgl. (Pennington et al., 2014), Seite 1532-1543.

²⁵⁵ Vgl. (Mikolov, Sutskever, et al., 2013), Seite 1-9.

²⁵⁶ Vgl. (Rehurek & Sojka, 2010), Seite 46-50.

Limitation ist erforderlich, um in vertretbarer Zeit viele verschiedene Wortvektoren zu erstellen und diese im Anschluss zu evaluieren.

Zur Prädiktionsevaluierung werden ausschließlich Produktbewertungen des Unternehmens Amazon.com genutzt. Die E-Commerce-Plattform des Unternehmens Amazon.com zählt mit einem Umsatz von ungefähr 64 Milliarden Euro zu den umsatzstärksten ihrer Art weltweit²⁵⁷. Um eine internationale Vergleichbarkeit zu schaffen, sollen für die Evaluierung ausschließlich englische Produktbewertungstexte verwendet werden. Nach einer Bereinigung der Daten, beispielsweise hinsichtlich Dubletten oder HTML-Tags, können 2,21 Gigabyte an Rezensionen für die Evaluierung der Prädiktion genutzt werden.

- **Umfang der Arbeit**

Eine wesentliche Beschränkung besteht im zulässigen Gesamtumfang der vorliegenden Bachelorarbeit. Aufgrund der Limitation der Arbeit werden nur zwei mögliche Einflussfaktoren, Umfang des Trainingstextes sowie Einfluss der Trainingsiterationen, untersucht. Des Weiteren werden aufgrund der Bearbeitungszeit von vier Monaten insgesamt nur 27 Modelle für jedes Verfahren trainiert. Zur Evaluierung der Hypothesen stehen pro Vergleichsgruppe so nur neun verschiedene Datenerfassungen zur Verfügung.

²⁵⁷ Vgl. (Statista, 2019a), Seite 1.

8. Abschließende Betrachtung

8.1 Schlussfolgerung mit Fazit

Infolge der deutlich anwachsenden jährlichen Generierung von Daten gewinnen Text Mining und automatische Textanalysen immer mehr an Bedeutung. Damit die dadurch gewonnen Erkenntnisse möglich frei von Störfaktoren sind, müssen die Basiskomponenten optimale Resultate liefern. In der vorliegenden Arbeit wurde der Einfluss verschiedener Hyperparameter auf die Prädiktion von Wortvektoren bei der Evaluierung durch Onlineproduktbewertungsdaten untersucht. Darüber hinaus wurden die zwei populären Word Embedding-Verfahren Word2Vec und GloVe hinsichtlich Accuracy miteinander verglichen.

Da beide Verfahren als Algorithmen zur Verfügung stehen, aber keine Zugriffsmöglichkeit ohne die Verwendung von Programmierung, wurde zunächst eine Option geschaffen, die den Aufruf der Algorithmen per .exe Datei ermöglicht. Die Parametrisierung kann über jeweils eine Konfigurationsdatei ohne Anwendung einer Programmiersprache erfolgen. Die Datenevaluierung erfolgte durch die von Di Stefano (2019) und Erhardt (2019) bereitgestellte Machine Learning-Plattform, die sich während der Prüfung von insgesamt 54 verschiedenen Modellen als sehr nützliches Hilfsmittel erwies^{258 259}.

Innerhalb der Untersuchung der Hyperparameter weist pro Verfahren jeweils ein Parameter einen signifikanten Einfluss auf die Prädiktion der Wortvektoren auf. Die aufgestellten Hypothesen über den Einfluss der Hyperparameter stehen weitestgehend mit den bereits veröffentlichten empirischen Untersuchungen im Einklang und können die bekannten Ergebnisse bestätigen. Bei den Ergebnissen des Verfahrensvergleich stehen sich unterschiedliche Meinungen und Studien gegenüber. Eine potenzielle Erklärung für die Abweichungen liefern Berardi et al. (2015) über den Einfluss der Art und Herkunft der Evaluierungstexte²⁶⁰. Die Untersuchungen innerhalb dieser Arbeit zeigen, dass Word2Vec bei der Evaluierung der Amazon-Rezessionen bessere

²⁵⁸ Vgl. (Di Stefano, 2019), Seite 1-156.

²⁵⁹ Vgl. (Erhardt, 2019), Seite 1-130.

²⁶⁰ Vgl. (Berardi, Esuli, & Marcheggiani, 2015), Seite 6.

Prädiktionsresultate liefert als GloVe. Die Differenz beider Verfahren weist Signifikanz auf.

Zusammengefasst zeigt die Arbeit auf, dass die Auswahl der Hyperparameter je nach Algorithmus variieren sollte, da der Einfluss bei beiden Verfahren unterschiedlich starke Ausprägungen erzielt. Die Verwendung von Word2Vec erweist sich bei der Evaluierung von Amazon-Bewertungen als sinnvoll. Dabei sollte allerdings stets eine Validierung der Ergebnisse stattfinden, um mögliche Fehler während des Trainings zu minimieren.

8.2 Forschungsausblick

Durch die vorliegende Bachelorthesis werden zwei Hypothesen, aufgeteilt in wiederum zwei Bestandteile, sowie eine Leitfrage beantwortet, wodurch existierende Annahmen aus der Wissenschaft bestätigt werden können. Darüber hinaus besteht durch die Arbeit eine Möglichkeit, die verwendeten Word Embedding-Algorithmen GloVe sowie Word2Vec als ausführbare Dateien mit einer simplen Parametrisierung universell aufzurufen. Neben der empirischen Betrachtung sowie der technischen Bereitstellung, zeigt sie einen Forschungsausblick auf, der nachfolgend näher erläutert werden soll.

- **Umfang und Herkunft der Trainingsdaten**

Da es grundsätzlich möglich ist, Wortvektoren basierend auf beliebigen Textquellen zu trainieren, wäre es interessant, wie sich Texte differentieller Art auf die anschließende Prädiktion auswirken. Vorstellbar wäre dies in Bezug auf beispielsweise Wörterbücher aus den Rezessionsdaten selbst und dabei die Frage, wie hoch die Gefahr eines Overfittings bei einem solchen Training ist. Ebenfalls wären Wörterbücher auf Basis von sozialen Medien wie beispielsweise Twitter interessant zu prüfen, da Tweets oftmals in Alltagssprache verfasst sind. Dies würde den Gegensatz zu den in dieser Arbeit verwendeten sprach- und rechtschreibgeprüften Wikipedia-Texten bilden.

Des Weiteren sind mit der Thesis Texte von 150, 300 und 500 Megabyte Umfang getestet worden. Hier könnte untersucht werden, wie es sich mit der Prädiktion verhält, wenn die Texte um ein Vielfaches umfangreicher sind und somit mehr unterschiedliche Wörter enthalten.

- **Art und Herkunft der Evaluierungsdaten**

Zur Evaluierung der Prädiktion der Wortvektoren werden im Rahmen der vorliegenden Arbeit Rezessionsdaten des Unternehmens Amazon.com. Interessant wäre es zu untersuchen, ob die erzielten Prädiktionswerte auch auf Rezessionsdaten anderer E-Commerce-Plattformen übertragbar sind. Darüber hinaus besteht das Potenzial, die Wortvektoren auch mit anderen Textarten hinsichtlich Accuracy zu validieren.

- **Untersuchung weiterer Hyperparameter**

Mithilfe dieser Bachelorthesis wird die Wirkung von zwei bereits bestehenden Erkenntnissen aus der Empirie auf die Onlineproduktbewertungen übertragen. Analysepotenzial bieten insbesondere die nicht untersuchten Hyperparameter. Bezüglich beider Word Embeddings wäre es so beispielsweise lohnenswert, den Einfluss des Parameters Window, sprich des Betrachtungsfensters, zu untersuchen. Insbesondere dieser Parameter wird in der gegenwärtigen Forschung viel diskutiert²⁶¹ ²⁶². Außerdem bietet der Hyperparameter Min_Count für beide Verfahren Analysemöglichkeiten, da hier untersucht werden könnte, ob sich die Prädiktion erhöht, wenn stark frequentierte Wörter mit berücksichtigt werden.

²⁶¹ Vgl. (Chang et al., 2017), Seite 567-568.

²⁶² Vgl. (Bernier-Colborne & Drouin, 2016), Seite 57.

V. Literatur- und Quellenverzeichnis

- Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., ... Brain, G. (2016). TensorFlow: A System for Large-Scale Machine Learning. In *Proceedings of the 12th USENIX Symposium on Operating System Design and Implementation (OSDI '16)* (pp. 265–283). USENIX. Retrieved from <https://tensorflow.org>.
- Abdi, H., & Williams, L. J. (2010). Tukey's Honestly Significant Difference (HSD) Test. In *Encyclopedia of Research Design* (pp. 1–5). Thousand Oaks: Sage. Retrieved from <https://www.researchgate.net/publication/237426041>
- Aghahadi, Z., & Talebpour, A. (2018). Word Embedding in Small Corpora: A Case Study in Quran. In *2018 8th International Conference on Computer and Knowledge Engineering (ICCKE)* (pp. 303–307). IEEE. <https://doi.org/10.1109/ICCKE.2018.8566605>
- Al-Amin, M., Islam, M. S., & Das Uzzal, S. (2017). Sentiment analysis of Bengali comments with Word2Vec and sentiment information of words. In *2017 International Conference on Electrical, Computer and Communication Engineering (ECCE)* (pp. 186–190). IEEE. <https://doi.org/10.1109/ECACE.2017.7912903>
- Alessa, A., Faezipour, M., & Alhassan, Z. (2018). Text Classification of Flu-Related Tweets Using FastText with Sentiment and Keyword Features. In *2018 IEEE International Conference on Healthcare Informatics (ICHI)* (pp. 366–367). IEEE. <https://doi.org/10.1109/ICHI.2018.00058>
- Alshahrani, H., Alzahrani, A., Alshehri, A., Alharthi, R., & Fu, H. (2017). Evaluation of Gradient Descent Optimization: Using Android Applications in Neural Networks. In *2017 International Conference on Computational Science and Computational Intelligence (CSCI)* (pp. 1471–1476). IEEE. <https://doi.org/10.1109/CSCI.2017.257>
- Alshari, E. M., Azman, A., Doraisamy, S., Mustapha, N., & Alkeshr, M. (2017). Improvement of Sentiment Analysis Based on Clustering of Word2Vec Features. In *2017 28th International Workshop on Database and Expert Systems Applications (DEXA)* (pp. 123–126). IEEE. <https://doi.org/10.1109/DEXA.2017.41>
- Amazon.com. (2019). Amazon.de Hilfe: Über ISBN, ASIN und UPC/EAN. Retrieved January 16, 2019, from <https://www.amazon.de/gp/help/customer/display.html?nodeId=769476>
- Amorim, L. A., Freitas, M. F., Dantas, A., de Souza, E. F., Camilo-Junior, C. G., & Martins, W. S. (2018). A New Word Embedding Approach to Evaluate Potential Fixes for Automated Program Repair. In *2018 International Joint Conference on Neural Networks (IJCNN)* (pp. 1–8). IEEE. <https://doi.org/10.1109/IJCNN.2018.8489079>
- Anderson, A. J., Kiela, D., Clark, S., & Poesio, M. (2017). Visually Grounded and Textual Semantic Models Differentially Decode Brain Activity Associated with Concrete and Abstract Nouns. *Transactions of the Association for Computational Linguistics*, 5, 17–30. Retrieved from https://www.mitpressjournals.org/doi/pdfplus/10.1162/tacl_a_00043
- Anoidgit. (2019). GloVe-Win. Retrieved February 7, 2019, from <https://github.com/anoidgit/GloVe-win>
- Auer, Tschurtschenthaler, & Biffl. (2003). A flyweight UML modelling tool for software development in heterogeneous environments. In *Proceedings of the 20th IEEE Instrumentation Technology Conference (Cat No 03CH37412) EURMIC-03* (pp. 267–272).

- IEEE. <https://doi.org/10.1109/EURMIC.2003.1231600>
- Baroni, M., Dinu, G., & Kruszewski, G. (2014). Don't count, predict! A systematic comparison of context-counting vs. context-predicting semantic vectors. In *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics* (pp. 238–247). Association for Computational Linguistics. Retrieved from <http://ronan.collobert.com/senna/>
- Berardi, G., Esuli, A., & Marcheggiani, D. (2015). Word Embeddings Go to Italy: a Comparison of Models and Training Datasets. In *Italian Information Retrieval Workshop* (pp. 1–8). Retrieved from http://ceur-ws.org/Vol-1404/paper_11.pdf
- Bernier-Colborne, G., & Drouin, P. (2016). Evaluation of distributional semantic models: a holistic approach. In *Proceedings of the 5th International Workshop on Computational Technology* (pp. 52–61). Association for Computational Linguistics. Retrieved from <http://www.aclweb.org/anthology/W16-4707>
- Berral-Garcia, J. L. (2016). A quick view on current techniques and machine learning algorithms for big data analytics. In *2016 18th International Conference on Transparent Optical Networks (ICTON)* (pp. 1–4). IEEE. <https://doi.org/10.1109/ICTON.2016.7550517>
- Bhoir, S., Ghorpade, T., & Mane, V. (2017). Comparative analysis of different word embedding models. In *2017 International Conference on Advances in Computing, Communication and Control (ICAC3)* (pp. 1–4). IEEE. <https://doi.org/10.1109/ICAC3.2017.8318770>
- Bosten, F. (2016). *Analyse von Amazon-Produktbewertungen mittels Text Mining im Hinblick auf die vom Leser vorgenommene Nützlichkeitswertung*. FOM Hochschule für Oekonomie & Management.
- Brophy, E., Dominguez, J. J., Wang, Z., & Ward, T. E. (2018). A Machine Vision Approach to Human Activity Recognition using Photoplethysmograph Sensor Data. In *2018 29th Irish Signals and Systems Conference (ISSC)* (pp. 1–6). IEEE. <https://doi.org/10.1109/ISSC.2018.8585372>
- Bruni, E., Uijlings, J., Baroni, M., & Sebe, N. (2012). *Distributional Semantics with Eyes: Using Image Analysis to Improve Computational Representations of Word Meaning*. Retrieved from <http://wacky.sslmit.unibo.it>
- Buitinck, L., Louppe, G., Blondel, M., Pedregosa, F., Müller, A. C., Grisel, O., ... Varoquaux, G. (2013). API design for machine learning software: experiences from the scikit-learn project. In *European Conference on Machine Learning and Principles and Practices of Knowledge Discovery in Databases* (pp. 1–15). Retrieved from <https://github.com/scikit-learn>
- Chaitanya, I., Madapakula, I., Gupta, S. K., & Thara, S. (2018). Word Level Language Identification in Code-Mixed Data using Word Embedding Methods for Indian Languages. In *2018 International Conference on Advances in Computing, Communications and Informatics (ICACCI)* (pp. 1137–1141). IEEE. <https://doi.org/10.1109/ICACCI.2018.8554501>
- Chang, C.-Y., Lee, S.-J., & Lai, C.-C. (2017). Weighted word2vec based on the distance of words. In *2017 International Conference on Machine Learning and Cybernetics (ICMLC)* (pp. 563–568). IEEE. <https://doi.org/10.1109/ICMLC.2017.8108974>

- Chen, C., Takahashi, T., Nakagawa, S., Inoue, T., & Kusumi, I. (2015). Reinforcement learning in depression: A review of computational research. *Neuroscience and Biobehavioral Reviews*, 55, 247–267. <https://doi.org/10.1016/j.neubiorev.2015.05.005>
- Chen, J., Tandon, N., & Melo, G. de. (2015). Neural Word Representations from Large-Scale Commonsense Knowledge. In *2015 IEEE/WIC/ACM International Conference on Web Intelligence and Intelligent Agent Technology (WI-IAT)* (pp. 225–228). IEEE. <https://doi.org/10.1109/WI-IAT.2015.150>
- Chilimbi, T., Suzue, Y., Apacible, J., & Kalyanaraman, K. (2014). Project Adam: Building an Efficient and Scalable Deep Learning Training System. In *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation* (pp. 571–582). Broomfield: USENIX Association. Retrieved from <https://www.usenix.org/conference/osdi14/technical-sessions/presentation/chilimbi>
- Chiu, B., Crichton, G., Korhonen, A., & Pyysalo, S. (2016). How to Train Good Word Embeddings for Biomedical NLP. In *Proceedings of the 15th Workshop on Biomedical Natural Language Processing* (pp. 166–174). Association for Computational Linguistics. Retrieved from <http://www.aclweb.org/anthology/W16-2922>
- Chowdhury, H. A., Imon, M. A. H., & Islam, M. S. (2018). A Comparative Analysis of Word Embedding Representations in Authorship Attribution of Bengali Literature. In *2018 21st International Conference of Computer and Information Technology (ICCIT)* (pp. 1–6). IEEE. <https://doi.org/10.1109/ICCITECHN.2018.8631977>
- Clark, E., & Araki, K. (2011). Text normalization in social media: progress, problems and applications for a pre-processing system of casual English. *Procedia-Social and Behavioral Sciences*, 27, 2–11. <https://doi.org/10.1016/j.sbspro.2011.10.577>
- Dekhtyar, A., & Fong, V. (2017). RE Data Challenge: Requirements Identification with Word2Vec and TensorFlow. In *2017 IEEE 25th International Requirements Engineering Conference (RE)* (pp. 484–489). IEEE. <https://doi.org/10.1109/RE.2017.26>
- Dey, A., & Learning, A. S. (2016). Machine Learning Algorithms : A Review. *International Journal of Computer Science and Information Technologies*, 7(3), 1174–1179. Retrieved from www.ijcsit.com
- Di Stefano, M. (2019). *Analyse von Online-Bewertungen mittels Machine Learning am Beispiel von Amazon*. FOM Hochschule für Oekonomie & Management.
- Dodal, S. S., & Kulkarni, P. V. (2018). Multi-Lingual Information Retrieval Using Deep Learning. In *2018 9th International Conference on Computing, Communication and Networking Technologies (ICCCNT)* (pp. 1–6). IEEE. <https://doi.org/10.1109/ICCCNT.2018.8493789>
- Du, X., Zheng, Z., Xiao, G., & Yin, B. (2017). The Automatic Classification of Fault Trigger Based Bug Report. In *2017 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)* (pp. 259–265). IEEE. <https://doi.org/10.1109/ISSREW.2017.28>
- Dube, P., & Sura, Z. (2018). Impact of System Resources on Performance of Deep Neural Network. In *2018 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)* (pp. 125–127). IEEE. <https://doi.org/10.1109/ISPASS.2018.00025>
- Dusserre, E. (2017). Bigger does not mean better! We prefer specificity. In *12th international conference on computer semantics (IWCS)* (pp. 1–6). Retrieved from <http://www.cis.uni->

muenchen.de/~schmid/tools/TreeTagger/

- Erhardt, M. S. (2019). *Entwicklung einer Plattform für Text Mining mittels Machine Learning auf Basis von Python und Tensorflow*. FOM Hochschule für Oekonomie & Management.
- Ertel, W. (2013). *Grundkurs künstliche Intelligenz: Eine praxisorientierte Einführung* (3rd ed.). Wiesbaden: Springer Fachmedien.
- Esuli, A. (2015). Downloader/Parser for Amazon Reviews. Retrieved January 25, 2019, from <https://github.com/aesuli/Amazon-downloader>
- Fahad, S. A., & Yahya, A. E. (2018). Inflectional Review of Deep Learning on Natural Language Processing. In *2018 International Conference on Smart Computing and Electronic Enterprise (ICSCEE)* (pp. 1–4). IEEE. <https://doi.org/10.1109/ICSCEE.2018.8538416>
- Fast, E., Chen, B., & Bernstein, M. S. (2017). Lexicons on Demand: Neural Word Embeddings for Large-Scale Text Analysis. In *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence (IJCAI-17)* (pp. 4836–4840). IJCAI. Retrieved from <https://www.ijcai.org/proceedings/2017/0677.pdf>
- Fry, C., & Manna, S. (2016). Can We Group Similar Amazon Reviews: A Case Study with Different Clustering Algorithms. In *2016 IEEE Tenth International Conference on Semantic Computing (ICSC)* (pp. 374–377). IEEE. <https://doi.org/10.1109/ICSC.2016.71>
- Fu, X., Ch'ng, E., Aickelin, U., & See, S. (2017). CRNN: A Joint Neural Network for Redundancy Detection. In *2017 IEEE International Conference on Smart Computing (SMARTCOMP)* (pp. 1–8). IEEE. <https://doi.org/10.1109/SMARTCOMP.2017.7946996>
- Ge, L., & Moh, T.-S. (2017). Improving text classification with word embedding. In *2017 IEEE International Conference on Big Data (Big Data)* (pp. 1796–1805). IEEE. <https://doi.org/10.1109/BigData.2017.8258123>
- Goldberg, Y. (2016). A Primer on Neural Network Models for Natural Language Processing. *Journal of Artificial Intelligence Research*, 57, 345–420. Retrieved from <http://www.cs.biu>.
- Görz, G., Schneeberger, J., & Schmid, U. (2014). *Handbuch der künstlichen Intelligenz* (5th ed.). München: Oldenbourg.
- Gotti, S., & Mbarki, S. (2016). UML executable: A comparative study of UML compilers and interpreters. In *2016 International Conference on Information Technology for Organizations Development (IT4OD)* (pp. 1–5). IEEE. <https://doi.org/10.1109/IT4OD.2016.7479251>
- Grawe, M. F., Martins, C. A., & Bonfante, A. G. (2017). Automated Patent Classification Using Word Embedding. In *2017 16th IEEE International Conference on Machine Learning and Applications (ICMLA)* (pp. 408–411). IEEE. <https://doi.org/10.1109/ICMLA.2017.0-127>
- Gupta, A., & Tripathy, B. K. (2017). Implementing GloVe for context based k-means++ clustering. In *2017 International Conference on Intelligent Sustainable Systems (ICISS)* (pp. 1041–1046). IEEE. <https://doi.org/10.1109/ISS1.2017.8389339>
- Hellrich, J., & Hahn, U. (2016). Bad Company-Neighborhoods in Neural Embedding Spaces Considered Harmful. In *Proceedings of COLING 2016, the 26th International Conference on Computational Linguistics* (pp. 2785–2796). Association for Computational Linguistics.

Retrieved from <http://www.julielab.de>

- Heyman, G., Vulićvulić, I., & Moens, M.-F. (2017). Bilingual Lexicon Induction by Learning to Combine Word-Level and Character-Level Representations. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics* (Vol. 1, pp. 1085–1095). Association for Computational Linguistics. Retrieved from <http://www.aclweb.org/anthology/E17-1102>
- Hill, J., & Simha, R. (2016). Automatic Generation of Context-Based Fill-in-the-Blank Exercises Using Co-occurrence Likelihoods and Google n-grams. In *Proceedings of the 11th Workshop of Innovative Use of NLP for Building Educational Applications* (pp. 23–30). Association for Computational Linguistics. Retrieved from <http://www.aclweb.org/anthology/W16-0503>
- Hinsen, K. (2009). The Promises of Functional Programming. *Computing in Science & Engineering*, 11(4), 86–90. <https://doi.org/10.1109/MCSE.2009.129>
- Hsu, W.-N., Zhang, Y., & Glass, J. (2017). Unsupervised Learning of Disentangled and Interpretable Representations from Sequential Data. In *31st Conference of Neural Information Processing Systems* (pp. 1–12). Long Beach, CA, USA.
- Hua, J. (2008). Study on the Application of Rough Sets Theory in Machine Learning. In *2008 Second International Symposium on Intelligent Information Technology Application* (pp. 192–196). IEEE. <https://doi.org/10.1109/IITA.2008.154>
- Huffman, D. (1952). A Method for the Construction of Minimum-Redundancy Codes. *Proceedings of the IRE*, 40(9), 1098–1101. <https://doi.org/10.1109/JRPROC.1952.273898>
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90.
- Iacobacci, I., Pilehvar, M. T., & Navigli, R. (2015). SENSEMBED: Learning Sense Embeddings for Word and Relational Similarity. In *Proceedings of the 53rd Annual Meeting of the Association for Computer Linguistics and the 7th International Joint Conference on Natural Language Processing* (pp. 95–105). Association for Computational Linguistics. Retrieved from <http://dumps.wikimedia.org/enwiki/>
- Innes Stefan Karpinski Viral Shah, M., Barber Pontus Stenertorp, D., Besard, T., Bradbury Salesforce Research Valentin Churavy Simon Danisch Alan Edelman Jon Malmaud Jarrett Revels, J., Yuret, D., Innes, M., ... Malmaud, J. (2018). On Machine Learning and Programming Languages. In *Proceedings of SysML conference (SysML 2018)* (pp. 1–3). Association for Computing Machinery. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>
- Islam, M. S., Amin, M. A., & Das Uzzal, S. (2016). Word embedding with hellinger PCA to detect the sentiment of bengali text. In *2016 19th International Conference on Computer and Information Technology (ICCIT)* (pp. 363–366). IEEE. <https://doi.org/10.1109/ICCITECHN.2016.7860224>
- Jackson, T. R., Levine, J. C., Grizzard, J. B., & Owen, H. L. (2004). An investigation of a compromised host on a honeynet being used to increase the security of a large enterprise network. In *Proceedings from the Fifth Annual IEEE SMC Information Assurance Workshop, 2004.* (pp. 9–14). IEEE. <https://doi.org/10.1109/IAW.2004.1437791>
- Jettakul, A., Thamjarat, C., Liaowongphuthorn, K., Udomcharoenchaikit, C., Vateekul, P., &

- Boonkwan, P. (2018). A Comparative Study on Various Deep Learning Techniques for Thai NLP Lexical and Syntactic Tasks on Noisy Data. In *2018 15th International Joint Conference on Computer Science and Software Engineering (JCSSE)* (pp. 1–6). IEEE. <https://doi.org/10.1109/JCSSE.2018.8457368>
- Jia, S., Zhang, X., Wang, X., & Liu, Y. (2018). Fake reviews detection based on LDA. In *2018 4th International Conference on Information Management (ICIM)* (pp. 280–283). IEEE. <https://doi.org/10.1109/INFOMAN.2018.8392850>
- Jin, X., Zhang, S., & Liu, J. (2018). Word Semantic Similarity Calculation Based on Word2vec. In *2018 International Conference on Control, Automation and Information Sciences (ICCAIS)* (pp. 12–16). IEEE. <https://doi.org/10.1109/ICCAIS.2018.8570612>
- Jordan, M. I., & Mitchell, T. M. (2015). Machine learning: Trends, perspectives, and prospects. *Science (New York, N.Y.)*, 349(6245), 255–260. <https://doi.org/10.1126/science.aaa8415>
- Ju, R., Zhou, P., Li, C. H., & Liu, L. (2015). An Efficient Method for Document Categorization Based on Word2vec and Latent Semantic Analysis. In *2015 IEEE International Conference on Computer and Information Technology; Ubiquitous Computing and Communications; Dependable, Autonomic and Secure Computing; Pervasive Intelligence and Computing* (pp. 2276–2283). IEEE. <https://doi.org/10.1109/CIT/IUCC/DASC/PICOM.2015.336>
- Jun-Li, W., Xiao-Min, W., & Ya-Xing, Y. (2017). A word embedding generation method based on flexible length context. In *2017 3rd IEEE International Conference on Computer and Communications (ICCC)* (pp. 2564–2568). IEEE. <https://doi.org/10.1109/CompComm.2017.8322998>
- Katic, T., & Milicevic, N. (2018). Comparing Sentiment Analysis and Document Representation Methods of Amazon Reviews. In *2018 IEEE 16th International Symposium on Intelligent Systems and Informatics (SISY)* (pp. 283–286). IEEE. <https://doi.org/10.1109/SISY.2018.8524814>
- Kaushik, L., Sangwan, A., & Hansen, J. H. L. (2013). Automatic sentiment extraction from YouTube videos. In *2013 IEEE Workshop on Automatic Speech Recognition and Understanding* (pp. 239–244). IEEE. <https://doi.org/10.1109/ASRU.2013.6707736>
- Khagi, B., Lee, C. G., & Kwon, G.-R. (2018). Alzheimer's disease Classification from Brain MRI based on transfer learning from CNN. In *2018 11th Biomedical Engineering International Conference (BMEiCON)* (pp. 1–4). IEEE. <https://doi.org/10.1109/BMEiCON.2018.8609974>
- Kim, K., Hong, Y.-G., & Han, Y.-H. (2018). General labelled data generator framework for network machine learning. In *2018 20th International Conference on Advanced Communication Technology (ICACT)* (pp. 127–131). IEEE. <https://doi.org/10.23919/ICACT.2018.8323670>
- Kruse, R., Borgelt, C., Braune, C., Klawonn, F., Moewes, C., & Steinbrecher, M. (2015). *Computational Intelligence*. Wiesbaden: Springer Fachmedien.
- Kurama, V. (2018). Towards Data Science. Retrieved March 19, 2019, from <https://towardsdatascience.com/unsupervised-learning-with-python-173c51dc7f03>
- Lai, S., Liu, K., He, S., & Zhao, J. (2016). How to Generate a Good Word Embedding. *IEEE Intelligent Systems*, 31(6), 5–14. <https://doi.org/10.1109/MIS.2016.45>
- Lauren, P., Qu, G., Huang, G.-B., Watta, P., & Lendasse, A. (2017). A low-dimensional vector

- representation for words using an extreme learning machine. In *2017 International Joint Conference on Neural Networks (IJCNN)* (pp. 1817–1822). IEEE.
<https://doi.org/10.1109/IJCNN.2017.7966071>
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444.
<https://doi.org/10.1038/nature14539>
- Levy, O., & Goldberg, Y. (2014). Dependency-Based Word Embeddings. In *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics (Short Papers)* (pp. 302–308). Retrieved from <http://www.aclweb.org/anthology/P14-2050>
- Li, L., Liu, S., Qin, M., Wang, Y., & Huang, D. (2016). Extracting Biomedical Event with Dual Decomposition Integrating Word Embeddings. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 13(4), 669–677. <https://doi.org/10.1109/TCBB.2015.2476876>
- Li, M., Yu, C., Nian, F., & Li, X. (2015). A Face Detection Algorithm Based on Deep Learning. *International Journal of Hybrid Information Technology*, 8(11), 285–296.
<https://doi.org/10.14257/ijhit.2015.8.11.24>
- Li, Q., Shah, S., Ghassemi, M., Fang, R., Nourbakhsh, A., & Liu, X. (2016). Using paraphrases to improve tweet classification: Comparing WordNet and word embedding approaches. In *2016 IEEE International Conference on Big Data (Big Data)* (pp. 4014–4016). IEEE.
<https://doi.org/10.1109/BigData.2016.7841094>
- Li, S., Chua, T.-S., Zhu, J., & Miao, C. (2016). Generative Topic Embedding: a Continuous Representation of Documents. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics* (pp. 666–675). Association for Computational Linguistics.
- Lilleberg, J., Zhu, Y., & Zhang, Y. (2015). Support vector machines and Word2vec for text classification with semantic features. In *2015 IEEE 14th International Conference on Cognitive Informatics & Cognitive Computing (ICCI*CC)* (pp. 136–140). IEEE.
<https://doi.org/10.1109/ICCI-CC.2015.7259377>
- Ling, Y., An, Y., Liu, M., Hasan, S. A., Fan, Y., & Hu, X. (2017). Integrating extra knowledge into word embedding models for biomedical NLP tasks. In *2017 International Joint Conference on Neural Networks (IJCNN)* (pp. 968–975). IEEE.
<https://doi.org/10.1109/IJCNN.2017.7965957>
- Liu, S., Bremer, P.-T., Thiagarajan, J. J., Srikumar, V., Wang, B., Livnat, Y., & Pascucci, V. (2018). Visual Exploration of Semantic Relationships in Neural Word Embeddings. *IEEE Transactions on Visualization and Computer Graphics*, 24(1), 553–562.
<https://doi.org/10.1109/TVCG.2017.2745141>
- Lochter, J. V., Pires, P. R., Bossolani, C., Yamakami, A., & Almeida, T. A. (2018). Evaluating the impact of corpora used to train distributed text representation models for noisy and short texts. In *2018 International Joint Conference on Neural Networks (IJCNN)* (pp. 1–8). IEEE. <https://doi.org/10.1109/IJCNN.2018.8489355>
- Ma, L., & Zhang, Y. (2015). Using Word2Vec to process big text data. In *2015 IEEE International Conference on Big Data (Big Data)* (pp. 2895–2897). IEEE.
<https://doi.org/10.1109/BigData.2015.7364114>
- Maas, A. L., Daly, R. E., Pham, P. T., Huang, D., Ng, A. Y., & Potts, C. (2011). Learning Word

- Vectors for Sentiment Analysis. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics* (pp. 142–150). Association for Computational Linguistics. Retrieved from <http://www.aclweb.org/anthology/P11-1015>
- Manaris, B. Z. (1994). Natural language processing tools and environments: the field in perspective. In *Proceedings Sixth International Conference on Tools with Artificial Intelligence. TAI 94* (p. 228). IEEE Comput. Soc. Press.
<https://doi.org/10.1109/TAI.1994.346491>
- Manna, S., & Nakai, H. (2019). Effectiveness of Word Embeddings on Classifiers: A Case Study with Tweets. In *2019 IEEE 13th International Conference on Semantic Computing (ICSC)* (pp. 158–161). IEEE. <https://doi.org/10.1109/ICOSC.2019.8665538>
- McKinney, W. (2010). Data Structures for Statistical Computing in Python. In *Proceedings of the 9th Python in Science Conference (Scipy 2010)* (pp. 51–56). Wes McKinney. Retrieved from <https://pdfs.semanticscholar.org/f6da/c1c52d3b07c993fe52513b8964f86e8fe381.pdf>
- Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient Estimation of Word Representations in Vector Space. In *Proceedings of ICLR Workshop* (pp. 1–11). Retrieved from <http://ronan.collobert.com/senna/>
- Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., & Dean, J. (2013). Distributed Representations of Words and Phrases and their Compositionality. *Advances in Neural Information Processing Systems*, 3111–3119. Retrieved from <http://papers.nips.cc/paper/5021-distributed-representations-of-words-andphrases>
- Mikolov, T., Yih, W.-T., & Zweig, G. (2013). Linguistic Regularities in Continuous Space Word Representations. In *Proceedings of NAACL-HLT* (pp. 746–751). Atlanta, Georgia: Association for Computational Linguistics. Retrieved from <http://research.microsoft.com/en->
- Miller, G. A. (1995). WordNet: A Lexical Database for English. *Communications of the ACM*, 38(11), 39–41. Retrieved from <http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.83.1823&rep=rep1&type=pdf>
- Min, S., Lee, B., & Yoon, S. (2016). Deep learning in bioinformatics. *Briefings in Bioinformatics*, 18(5), 851–869. <https://doi.org/10.1093/bib/bbw068>
- Mingzhen Zhao, Bo Xu, Hongfei Lin, Yang, Z., & Jian Wang. (2015). Discover potential adverse drug reactions using the skip-gram model. In *2015 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)* (pp. 1765–1767). IEEE.
<https://doi.org/10.1109/BIBM.2015.7359955>
- Mitchell, J., & Lapata, M. (2010). Composition in Distributional Models of Semantics. *Cognitive Science*, 34(8), 1388–1429. <https://doi.org/10.1111/j.1551-6709.2010.01106.x>
- Mitchell, T. M. (1997). *Machine Learning*. New York City, USA: McGraw-Hill Education Ltd.
- Mohammed, A. A., & Umaashankar, V. (2018). Effectiveness of Hierarchical Softmax in Large Scale Classification Tasks. In *2018 International Conference on Advances in Computing, Communications and Informatics (ICACCI)* (pp. 1090–1094). IEEE.
<https://doi.org/10.1109/ICACCI.2018.8554637>
- Mori, Y., & Satoh, S. (2018). Co-Cleansing of Image and Text Dataset by Automatic Image

- Annotation and Proper Noun Analysis. In *2018 IEEE 7th Global Conference on Consumer Electronics (GCCE)* (pp. 485–487). IEEE. <https://doi.org/10.1109/GCCE.2018.8574763>
- Morin, F., & Bengio, Y. (2005). Hierarchical Probabilistic Neural Network Language Model. In *Proceedings of the Tenth International Workshop on Artificial Intelligence and Statistics* (pp. 246–252). AISTATS. Retrieved from <https://www.iro.umontreal.ca/~lisa/pointeurs/hierarchical-nnlnm-aistats05.pdf>
- Mu, R., Zeng, X., & Han, L. (2018). A Survey of Recommender Systems Based on Deep Learning. *IEEE Access*, 1–1. <https://doi.org/10.1109/ACCESS.2018.2880197>
- Musumeci, F., Rottondi, C., Nag, A., Macaluso, I., Zibar, D., Ruffini, M., & Tornatore, M. (2018). An Overview on Application of Machine Learning Techniques in Optical Networks. *IEEE Communications Surveys & Tutorials*, 1–1. <https://doi.org/10.1109/COMST.2018.2880039>
- Nishani, E., & Cico, B. (2017). Computer vision approaches based on deep learning and neural networks: Deep neural networks for video analysis of human pose estimation. In *2017 6th Mediterranean Conference on Embedded Computing (MECO)* (pp. 1–4). IEEE. <https://doi.org/10.1109/MECO.2017.7977207>
- Ordentlich, E., Yang, L., Feng, A., Cnudde, P., Grbovic, M., Djuric, N., ... Owens, G. (2016). Network-Efficient Distributed Word2vec Training System for Large Vocabularies. In *Proceedings of the 25th ACM International on Conference on Information and Knowledge Management - CIKM '16* (pp. 1139–1148). New York, New York, USA: ACM Press. <https://doi.org/10.1145/2983323.2983361>
- Parveen, P., & Thuraisingham, B. (2012). Unsupervised incremental sequence learning for insider threat detection. In *2012 IEEE International Conference on Intelligence and Security Informatics* (pp. 141–143). IEEE. <https://doi.org/10.1109/ISI.2012.6284271>
- Paterakis, N. G., Mocanu, E., Gibescu, M., Stappers, B., & van Alst, W. (2017). Deep learning versus traditional machine learning methods for aggregated energy demand prediction. In *2017 IEEE PES Innovative Smart Grid Technologies Conference Europe (ISGT-Europe)* (pp. 1–6). IEEE. <https://doi.org/10.1109/ISGTEurope.2017.8260289>
- Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global Vectors for Word Representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 1532–1543). Association for Computational Linguistics. Retrieved from <http://nlp>.
- Pennington, J., Socher, R., & Manning, C. D. (2019a). GloVe model for distributed word representation. Retrieved February 7, 2019, from <https://github.com/stanfordnlp/GloVe>
- Pennington, J., Socher, R., & Manning, C. D. (2019b). Stanford NLP GloVe. Retrieved February 7, 2019, from <https://github.com/stanfordnlp/GloVe/tree/master/src>
- Pezzotti, N., Lelieveldt, B. P. F., Maaten, L. van der, Holtt, T., Eisemann, E., & Vilanova, A. (2017). Approximated and User Steerable tSNE for Progressive Visual Analytics. *IEEE Transactions on Visualization and Computer Graphics*, 23(7), 1739–1752. <https://doi.org/10.1109/TVCG.2016.2570755>
- Pirinen, T. A., & Linden, K. (2010). Building and using existing hunspell dictionaries and TEX hyphenators as finite-state automata. In *Proceedings of the International Multiconference on Computer Science and Information Technology* (pp. 477–484). IEEE.

<https://doi.org/10.1109/IMCSIT.2010.5679949>

- Pou-Prom, C., & Rudzicz, F. (2018). Learning multiview embeddings for assessing dementia. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing* (pp. 2812–2817). Association for Computational Linguistics. Retrieved from <http://www.aclweb.org/anthology/D18-1304>
- Putra, Y. A., & Khodra, M. L. (2016). Deep learning and distributional semantic model for Indonesian tweet categorization. In *2016 International Conference on Data and Software Engineering (ICoDSE)* (pp. 1–6). IEEE. <https://doi.org/10.1109/ICODSE.2016.7936108>
- Rampasek, L., & Goldenberg, A. (2016). TensorFlow: Biology's Gateway to Deep Learning? *Cell Systems*, 2(1), 12–14. <https://doi.org/10.1016/J.CELS.2016.01.009>
- Rehurek, R. (2019). Corpus from a Wikipedia dump. Retrieved February 14, 2019, from <https://radimrehurek.com/gensim/corpora/wikicorpus.html>
- Rehurek, R., & Sojka, P. (2010). Software Framework for Topic Modelling with Large Corpora. In *Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks* (pp. 46–50). Retrieved from <http://numdam.org>
- Robinson, D. (2017). Stack Overflow. Retrieved February 4, 2019, from <https://stackoverflow.blog/2017/09/06/incredible-growth-python/>
- Rodriguez, D. V., Carver, D. L., & Mahmoud, A. (2018). An efficient wikipedia-based approach for better understanding of natural language text related to user requirements. In *2018 IEEE Aerospace Conference* (pp. 1–16). IEEE. <https://doi.org/10.1109/AERO.2018.8396645>
- Rooy, N. (2018). Word2vec from Scratch with Python and NumPy. Retrieved February 4, 2019, from <https://nathanrooy.github.io/posts/2018-03-22/word2vec-from-scratch-with-python-and-numpy/>
- Saad, F., & Mathiak, B. (2013). Revised mutual information approach for german text sentiment classification. In *Proceedings of the 22nd International Conference on World Wide Web - WWW '13 Companion* (pp. 579–586). New York, New York, USA: ACM Press. <https://doi.org/10.1145/2487788.2487997>
- Sahlgren, M. (2008). The distributional hypothesis. *Italian Journal of Linguistics*, 20(1), 33–54. Retrieved from <http://www.diva-portal.org/smash/get/diva2:1041938/FULLTEXT01.pdf>
- Samiah, A., Aziz, A., & Ikram, N. (2007). An Efficient Software Implementation of AES-CCM for IEEE 802.11i Wireless St. In *31st Annual International Computer Software and Applications Conference - Vol. 2 - (COMPSAC 2007)* (pp. 689–694). IEEE. <https://doi.org/10.1109/COMPSAC.2007.62>
- Samuel, A. L. (1959). Some Studies in Machine Learning Using the Game of Checkers. *IBM Journal of Research and Development*, 3(3), 210–229. <https://doi.org/10.1147/rd.33.0210>
- Sause, W. (2013). Coordinated Reinforcement Learning Agents in a Multi-agent Virtual Environment. In *2013 12th International Conference on Machine Learning and Applications* (pp. 227–230). IEEE. <https://doi.org/10.1109/ICMLA.2013.46>
- Savigny, J., & Purwarianti, A. (2017). Emotion classification on youtube comments using word embedding. In *2017 International Conference on Advanced Informatics, Concepts, Theory, and Applications (ICAICTA)* (pp. 1–5). IEEE.

<https://doi.org/10.1109/ICAICTA.2017.8090986>

Scheffe, H. (1959). *The Analysis of Variance*. New York: John Wiley & Sons, Inc.

Schmidhuber, J. (2015). Deep learning in neural networks: An overview. *Neural Networks*, 61, 85–117. <https://doi.org/10.1016/J.NEUNET.2014.09.003>

Sebastiani, F., & Fabrizio. (2002). Machine learning in automated text categorization. *ACM Computing Surveys*, 34(1), 1–47. <https://doi.org/10.1145/505282.505283>

Semberecki, P., & Maciejewski, H. (2017). Deep Learning methods for Subject Text Classification of Articles. In *2017 Federated Conference on Computer Science and Information Systems (FedCSIS)* (pp. 357–360). IEEE. <https://doi.org/10.15439/2017F414>

Setiowati, S., Zulfanahri, Franita, E. L., & Ardiyanto, I. (2017). A review of optimization method in face recognition: Comparison deep learning and non-deep learning methods. In *2017 9th International Conference on Information Technology and Electrical Engineering (ICITEE)* (pp. 1–6). IEEE. <https://doi.org/10.1109/ICITEED.2017.8250484>

Shancheng, T., Yunyue, B., & Fuyu, M. (2017). A Chinese short text semantic similarity computation model based on stop words and TongyiciCilin. In *2017 6th International Conference on Computer Science and Network Technology (ICCSNT)* (pp. 310–314). IEEE. <https://doi.org/10.1109/ICCSNT.2017.8343708>

Sharma, R., Bhattacharyya, P., Dandapat, S., & Bhatt, H. S. (2018). Identifying Transferable Information Across Domains for Cross-domain Sentiment Classification. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics* (pp. 968–978). Association for Computational Linguistics. Retrieved from <http://stattrek.com/chi-square-test/>

Sharma, Y., Agrawal, G., Jain, P., & Kumar, T. (2017). Vector representation of words for sentiment analysis using GloVe. In *2017 International Conference on Intelligent Communication and Computational Techniques (ICCT)* (pp. 279–284). IEEE. <https://doi.org/10.1109/INTELCCT.2017.8324059>

Shen, C.-H., Sung, J. Y., & Lee, H.-Y. (2018a). Language Transfer of Audio Word2Vec: Learning Audio Segment Representations Without Target Language Data. In *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* (pp. 2231–2235). IEEE. <https://doi.org/10.1109/ICASSP.2018.8461305>

Shen, C.-H., Sung, J. Y., & Lee, H.-Y. (2018b). Language Transfer of Audio Word2Vec: Learning Audio Segment Representations Without Target Language Data. In *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* (pp. 2231–2235). IEEE. <https://doi.org/10.1109/ICASSP.2018.8461305>

Shi, H., Xu, M., & Li, R. (2018). Deep Learning for Household Load Forecasting—A Novel Pooling Deep RNN. *IEEE Transactions on Smart Grid*, 9(5), 5271–5280. <https://doi.org/10.1109/TSG.2017.2686012>

Shiddieqy, H. A., Hariadi, F. I., & Adiono, T. (2018). Effect of Sampling Variation in Accuracy for Fault Transmission Line Classification Application Based On Convolutional Neural Network. In *2018 International Symposium on Electronics and Smart Devices (ISESD)* (pp. 1–3). IEEE. <https://doi.org/10.1109/ISESD.2018.8605469>

Sienčnik, S. K. (2015). Adapting word2vec to Named Entity Recognition. In *Proceedings of the*

- 20th Nordic Conference of Computational Linguistics* (pp. 239–243). NODALIDA. Retrieved from <http://www.ep.liu.se/ecp/109/030/ecp15109030.pdf>
- Simonton, T. M., & Alagband, G. (2017). Efficient and accurate Word2Vec implementations in GPU and shared-memory multicore architectures. In *2017 IEEE High Performance Extreme Computing Conference (HPEC)* (pp. 1–7). IEEE. <https://doi.org/10.1109/HPEC.2017.8091076>
- Smilkov, D., Thorat, N., Nicholson, C., Reif, E., Viégas, F. B., & Wattenberg, M. (2016). Embedding Projector: Interactive Visualization and Interpretation of Embeddings. In *30th Conference on Neural Information Processing Systems (NIPS 2016)* (pp. 1–4). arXiv preprint. Retrieved from <https://arxiv.org/pdf/1611.05469.pdf>
- Solangi, Y. A., Solangi, Z. A., Aarain, S., Abro, A., Mallah, G. A., & Shah, A. (2018). Review on Natural Language Processing (NLP) and Its Toolkits for Opinion Mining and Sentiment Analysis. In *2018 IEEE 5th International Conference on Engineering Technologies and Applied Sciences (ICETAS)* (pp. 1–4). IEEE. <https://doi.org/10.1109/ICETAS.2018.8629198>
- Sravani, L., Reddy, A. S., & Thara, S. (2018). A Comparison Study of Word Embedding for Detecting Named Entities of Code-Mixed Data in Indian Language. In *2018 International Conference on Advances in Computing, Communications and Informatics (ICACCI)* (pp. 2375–2381). IEEE. <https://doi.org/10.1109/ICACCI.2018.8554918>
- Statista. (2018). User-generated internet content per minute 2018 | Statistic. Retrieved March 18, 2019, from <https://www.statista.com/statistics/195140/new-user-generated-content-uploaded-by-users-per-minute/>
- Statista. (2019a). • ecommerceDB - Top Online-Shops weltweit | Statista. Retrieved March 18, 2019, from <https://de.statista.com/statistik/studie/id/55041/dokument/ecommercedb-top-online-shops-weltweit/>
- Statista. (2019b). Biggest reasons for machine learning technology adoption in organizations worldwide as of 2018. Retrieved March 18, 2019, from <https://www.statista.com/statistics/951523/worldwide-top-enterprise-machine-learning-use-cases/>
- Stevens, P., & Pooley, R. (2006). *Using Uml: Software Engineering with Objects and Components* (2nd edition). Boston, MA, USA: Addison-Wesley Longman Publishing Co., Inc. Retrieved from www.pearson.co.uk/stevens
- Stratos, K., Collins, M., & Hsu, D. (2015). Model-based Word Embeddings from Decompositions of Count Matrices. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing* (pp. 1282–1291). Association for Computational Linguistics. Retrieved from <http://www.aclweb.org/anthology/P15-1124>
- Sukanya, M., & Biruntha, S. (2012). Techniques on text mining. In *2012 IEEE International Conference on Advanced Communication Control and Computing Technologies (ICACCCT)* (pp. 269–271). IEEE. <https://doi.org/10.1109/ICACCCT.2012.6320784>
- Sultana Ritu, Z., Nowshin, N., Mahadi Hasan Nahid, M., & Ismail, S. (2018). Performance Analysis of Different Word Embedding Models on Bangla Language. In *2018 International Conference on Bangla Speech and Language Processing (ICBSLP)* (pp. 1–5). IEEE. <https://doi.org/10.1109/ICBSLP.2018.8554681>

- Sumit, S. H., Hossan, M. Z., Al Muntasir, T., & Sourov, T. (2018). Exploring Word Embedding for Bangla Sentiment Analysis. In *2018 International Conference on Bangla Speech and Language Processing (ICBSLP)* (pp. 1–5). IEEE. <https://doi.org/10.1109/ICBSLP.2018.8554443>
- Sun, R.-H., & Hao, J. (2017). Comparisons of word representations for convolutional neural network: An exploratory study on tourism Weibo classification. In *2017 International Conference on Service Systems and Service Management* (pp. 1–5). IEEE. <https://doi.org/10.1109/ICSSSM.2017.7996295>
- Suzuki, J., & Nagata, M. (2015). A Unified Learning Framework of Skip-Grams and Global Vectors. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing* (pp. 186–191). Association for Computational Linguistics. Retrieved from <http://www.aclweb.org/anthology/P15-2031>
- Svozil, D., Kvasnieka, V., & Pospichal, J. (1997). *Chemometrics and intelligent laboratory systems Introduction to multi-layer feed-forward neural networks. Chemometrics and Intelligent Laboratory Systems* (Vol. 39). Retrieved from http://ich.vscht.cz/~svozil/pubs/nn_intro.pdf
- Sze, V., Chen, Y.-H., Emer, J., Suleiman, A., & Zhang, Z. (2017). Hardware for machine learning: Challenges and opportunities. In *2017 IEEE Custom Integrated Circuits Conference (CICC)* (pp. 1–8). IEEE. <https://doi.org/10.1109/CICC.2017.7993626>
- Takhavova, E., & Kosjukhina, G. (2018). Neural Networks for the Moving Objects Control. In *2018 3rd Russian-Pacific Conference on Computer Technology and Applications (RPC)* (pp. 1–4). IEEE. <https://doi.org/10.1109/RPC.2018.8482192>
- Taqi, A. M., Awad, A., Al-Azzo, F., & Milanova, M. (2018). The Impact of Multi-Optimizers and Data Augmentation on TensorFlow Convolutional Neural Network Performance. In *2018 IEEE Conference on Multimedia Information Processing and Retrieval (MIPR)* (pp. 140–145). IEEE. <https://doi.org/10.1109/MIPR.2018.00032>
- TensorFlow. (2019). Retrieved March 20, 2019, from <https://www.tensorflow.org/tutorials/representation/word2vec>
- TH, M., Sahu, S. K., & Anand, A. (2015). Evaluating distributed word representations for capturing semantics of biomedical concepts. In *Proceedings of the 2015 Workshop on Biomedical Natural Language Processing (BioNLP 2015)* (pp. 158–163). Association for Computer Linguistics. Retrieved from <http://nlp.stanford.edu/projects/glove/>
- Tian, W., Li, J., & Li, H. (2018). A Method of Feature Selection Based on Word2Vec in Text Categorization. In *2018 37th Chinese Control Conference (CCC)* (pp. 9452–9455). IEEE. <https://doi.org/10.23919/ChiCC.2018.8483345>
- Urata, T., & Maeda, A. (2017). An Entity Disambiguation Approach Based on Wikipedia for Entity Linking in Microblogs. In *2017 6th IIAI International Congress on Advanced Applied Informatics (IIAI-AAI)* (pp. 334–338). IEEE. <https://doi.org/10.1109/IIAI-AAI.2017.171>
- Van Der Maaten, L., & Hinton, G. (2008). Visualizing Data using t-SNE. *Journal of Machine Learning Research*, 9, 2579–2605. Retrieved from <http://www.jmlr.org/papers/volume9/vandermaaten08a/vandermaaten08a.pdf>

- van der Walt, S., Colbert, S. C., & Varoquaux, G. (2011). The NumPy Array: A Structure for Efficient Numerical Computation. *Computing in Science & Engineering*, 13(2), 22–30. <https://doi.org/10.1109/MCSE.2011.37>
- Van Tuan, D., & Sato, H. (2017). Improving distributed representation by feature selection of Wikipedia. In *2017 Fourth Asian Conference on Defence Technology - Japan (ACDT)* (pp. 1–6). IEEE. <https://doi.org/10.1109/ACDTJ.2017.8259588>
- Vargas, E. (2017). A Comprehensive Introduction to Word Vector Representations. Retrieved February 24, 2019, from <https://medium.com/ai-society/jkljlj-7d6e699895c4>
- Verzani, J. (2014). *Using R for Introductory Statistics* (2nd ed.). New York, USA: Chapman and Hall/CRC.
- Vom Brocke, J., Simons, A., Riemer, K., & Niehaves, B. (2015). Standing on the Shoulders of Giants: Challenges and Recommendations of Literature Search in Information Systems Research. *Communications of the Association for Information Systems*, 37(9), 205–224. <https://doi.org/10.17705/1CAIS.03709>
- Wan, J., Hoi, S. C. H., Wu, P., Zhu, J., Wan, J. ;, Wang, D. ;, ... Zhang, Y. (2014). Deep learning for content-based image retrieval: A comprehensive study Deep learning for content-based image retrieval: A Deep Learning for Content-Based Image Retrieval: A Comprehensive Study. In *Proceedings of the ACM International Conference on Multimedia* (pp. 157–166). Orlando. <https://doi.org/10.1145/2647868.2654948>
- Wang, H. (2009). *Multivariate Dichteschätzung in der explorativen Datenanalyse*. Universität Augsburg. Retrieved from https://opus.bibliothek.uni-augsburg.de/opus4/frontdoor/deliver/index/docId/1383/file/Dissertation_Wang.pdf
- Wang, Y., & Berwick, R. C. (2012). On formal models for cognitive linguistics. In *2012 IEEE 11th International Conference on Cognitive Informatics and Cognitive Computing* (pp. 7–17). IEEE. <https://doi.org/10.1109/ICCI-CC.2012.6311169>
- Wang, Z., Zhang, J., Feng, J., & Chen, Z. (2014). Knowledge Graph and Text Jointly Embedding. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 1591–1601). Doha, Qatar: Association for Computer Linguistics. Retrieved from <http://www.aclweb.org/anthology/D14-1167>
- White, D., & Wilson, R. C. (2007). Spectral Generative Models for Graphs. In *14th International Conference on Image Analysis and Processing (ICIAP 2007)* (pp. 35–42). IEEE. <https://doi.org/10.1109/ICIAP.2007.4362754>
- Wikimedia Foundation. (2018). Wikimedia Statistics - English Wikipedia - Pages to date. Retrieved January 19, 2019, from <https://stats.wikimedia.org/v2/#/en.wikipedia.org/content/pages-to-date/normal%7Cline%7CAll%7C~total>
- Wikimedia Foundation. (2019). Dumps Wikimedia. Retrieved January 19, 2019, from <https://dumps.wikimedia.org/enwiki/latest/>
- Wint, Z. Z., Manabe, Y., & Aritsugi, M. (2018). Deep Learning Based Sentiment Classification in Social Network Services Datasets. In *2018 IEEE International Conference on Big Data, Cloud Computing, Data Science & Engineering (BCD)* (pp. 91–96). IEEE. <https://doi.org/10.1109/BCD2018.2018.00022>

- Wu, C., Shi, X., Chen, Y., Su, J., Wang, B., & Key, F. (2017). Improving Implicit Discourse Relation Recognition with Discourse-specific Word Embeddings. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics* (pp. 269–274). Association for Computational Linguistics. <https://doi.org/10.18653/v1/P17-2042>
- Wu, X., & Li, H. (2017). Topic mover's distance based document classification. In *2017 IEEE 17th International Conference on Communication Technology (ICCT)* (pp. 1998–2002). IEEE. <https://doi.org/10.1109/ICCT.2017.8359979>
- Xiao, Y., & Shi, Q. (2015). Research and Implementation of Hybrid Recommendation Algorithm Based on Collaborative Filtering and Word2Vec. In *2015 8th International Symposium on Computational Intelligence and Design (ISCID)* (pp. 172–175). IEEE. <https://doi.org/10.1109/ISCID.2015.211>
- Xue, M., & Zhu, C. (2009). A Study and Application on Machine Learning of Artificial Intelligence. In *2009 International Joint Conference on Artificial Intelligence* (pp. 272–274). IEEE. <https://doi.org/10.1109/IJCAI.2009.55>
- Yang, S., & Xia, Z. (2016). A convolutional neural network method for Chinese document sentiment analyzing. In *2016 2nd IEEE International Conference on Computer and Communications (ICCC)* (pp. 308–312). IEEE. <https://doi.org/10.1109/CompComm.2016.7924714>
- Yao, H. M., & Jiang, L. J. (2018). Machine Learning Based Neural Network Solving Methods for the FDTD Method. In *2018 IEEE International Symposium on Antennas and Propagation & USNC/URSI National Radio Science Meeting* (pp. 2321–2322). IEEE. <https://doi.org/10.1109/APUSNCURSINRSM.2018.8608745>
- Yi, X., Zheng, R., Wang, A., Qin, H., & Chen, Y. (2017). Design and implementation of Word2Vec parallel algorithm based on HPC. In *2017 Chinese Automation Congress (CAC)* (pp. 585–590). IEEE. <https://doi.org/10.1109/CAC.2017.8242835>
- Yu, L.-C., Wang, J., Lai, K. R., & Zhang, X. (2018). Refining Word Embeddings Using Intensity Scores for Sentiment Analysis. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 26(3), 671–681. <https://doi.org/10.1109/TASLP.2017.2788182>
- Yuan, C., Wu, J., Li, H., & Wang, L. (2018). Personality Recognition Based on User Generated Content. In *2018 15th International Conference on Service Systems and Service Management (ICSSSM)* (pp. 1–6). IEEE. <https://doi.org/10.1109/ICSSSM.2018.8465006>
- Zarghoon, A., Awan, I., Disso, J. P., & Dennis, R. (2017). Evaluation of AV systems against modern malware. In *2017 12th International Conference for Internet Technology and Secured Transactions (ICITST)* (pp. 269–273). IEEE. <https://doi.org/10.23919/ICITST.2017.8356397>
- Zhang, C., Wang, X., Yu, S., & Wang, Y. (2018). Research on Keyword Extraction of Word2vec Model in Chinese Corpus. In *2018 IEEE/ACIS 17th International Conference on Computer and Information Science (ICIS)* (pp. 339–343). IEEE. <https://doi.org/10.1109/ICIS.2018.8466534>
- Zhang, L., & Chen, C. (2016). Sentiment Classification with Convolutional Neural Networks: An Experimental Study on a Large-Scale Chinese Conversation Corpus. In *2016 12th International Conference on Computational Intelligence and Security (CIS)* (pp. 165–169). IEEE. <https://doi.org/10.1109/CIS.2016.0046>

- Zhang, L., Li, J., & Wang, C. (2017). Automatic synonym extraction using Word2Vec and spectral clustering. In *2017 36th Chinese Control Conference (CCC)* (pp. 5629–5632). IEEE. <https://doi.org/10.23919/ChiCC.2017.8028251>
- Zhang, S., Mo, Y., Ghoshal, T., Wilkins, D., Chen, Y., & Zhou, Y. (2017). Novel gene selection method for breast cancer intrinsic subtypes from two large cohort study. In *2017 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)* (pp. 2198–2203). IEEE. <https://doi.org/10.1109/BIBM.2017.8217999>
- Zhang, X., Lin, P., Chen, S., Cen, H., Wang, J., Huang, Q., ... Huang, P. (2016). Valence-arousal prediction of Chinese Words with multi-layer corpora. In *2016 International Conference on Asian Language Processing (IALP)* (pp. 304–307). IEEE. <https://doi.org/10.1109/IALP.2016.7875992>
- Zhang, X., & Yu, Q. (2017). Hotel reviews sentiment analysis based on word vector clustering. In *2017 2nd IEEE International Conference on Computational Intelligence and Applications (ICCIA)* (pp. 260–264). IEEE. <https://doi.org/10.1109/CIAPP.2017.8167219>
- Zhang, Y., Er, M. J., Zhao, R., & Pratama, M. (2017). Multiview Convolutional Neural Networks for Multidocument Extractive Summarization. *IEEE Transactions on Cybernetics*, 47(10), 3230–3242. <https://doi.org/10.1109/TCYB.2016.2628402>
- Zhang, Y., Xiong, R., He, H., & Pecht, M. G. (2018). Long Short-Term Memory Recurrent Neural Network for Remaining Useful Life Prediction of Lithium-Ion Batteries. *IEEE Transactions on Vehicular Technology*, 67(7), 5695–5705. <https://doi.org/10.1109/TVT.2018.2805189>
- Zhao, H., Quach, D., Wang, S., Wang, H., Chen, H., Li, X., & Tan, S. X.-D. (2015). Learning based compact thermal modeling for energy-efficient smart building management. In *2015 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)* (pp. 450–456). IEEE. <https://doi.org/10.1109/ICCAD.2015.7372604>
- Ziak, H., & Kern, R. (2016). Query Splitting for Context-Driven Federated Recommendations. In *2016 27th International Workshop on Database and Expert Systems Applications (DEXA)* (pp. 193–197). IEEE. <https://doi.org/10.1109/DEXA.2016.049>

VI. Anhang

Anhang A: Aufbereitung der Evaluierungsdaten

- RegEx - Reguläre Ausdrücke

<[^<]+?>;

<[^<]+?>;

- Strings

ASCII

";"

&;&

';'

<;<

>;>

HTML

";"

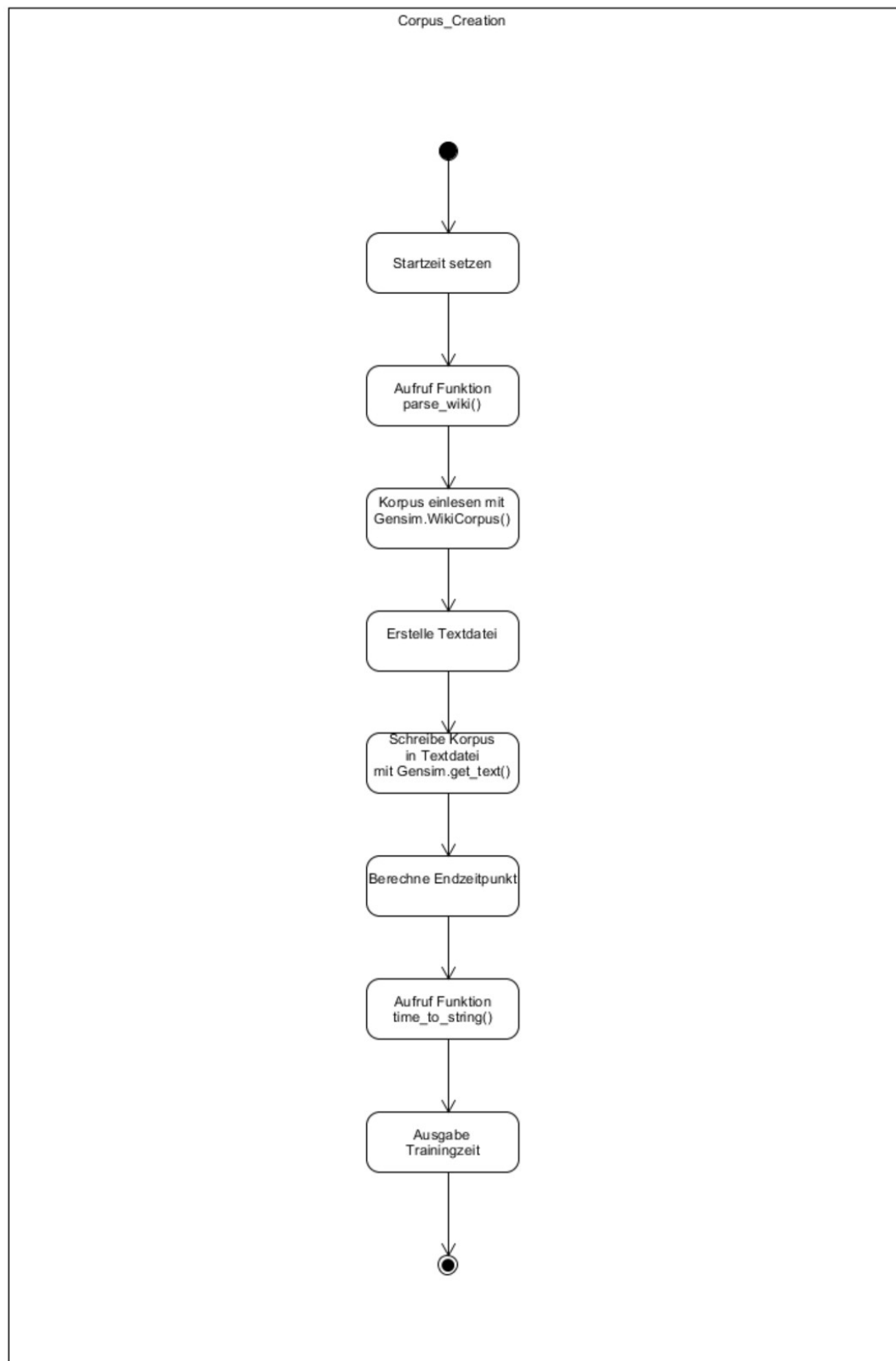
&;&

';'

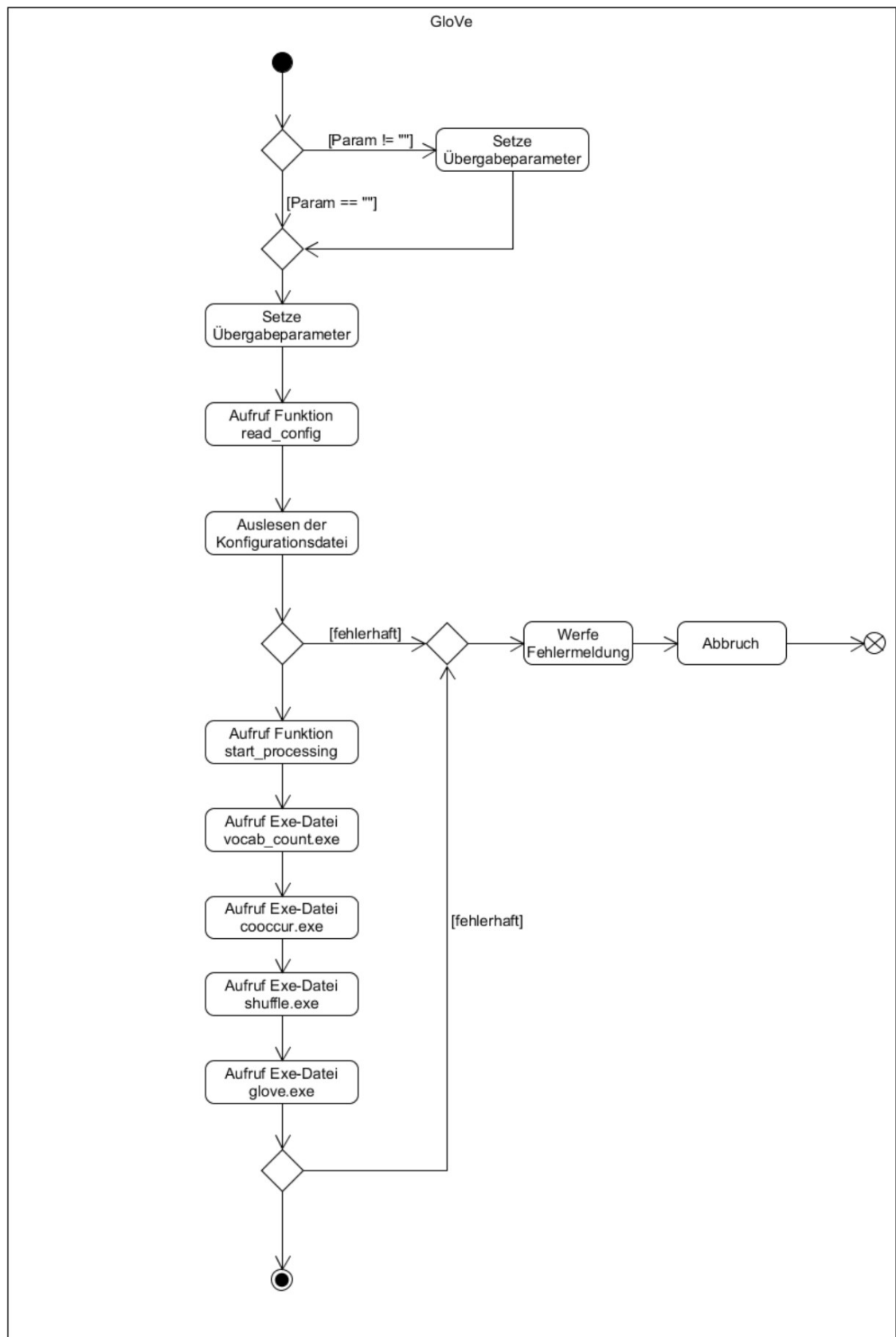
<;<

>;>

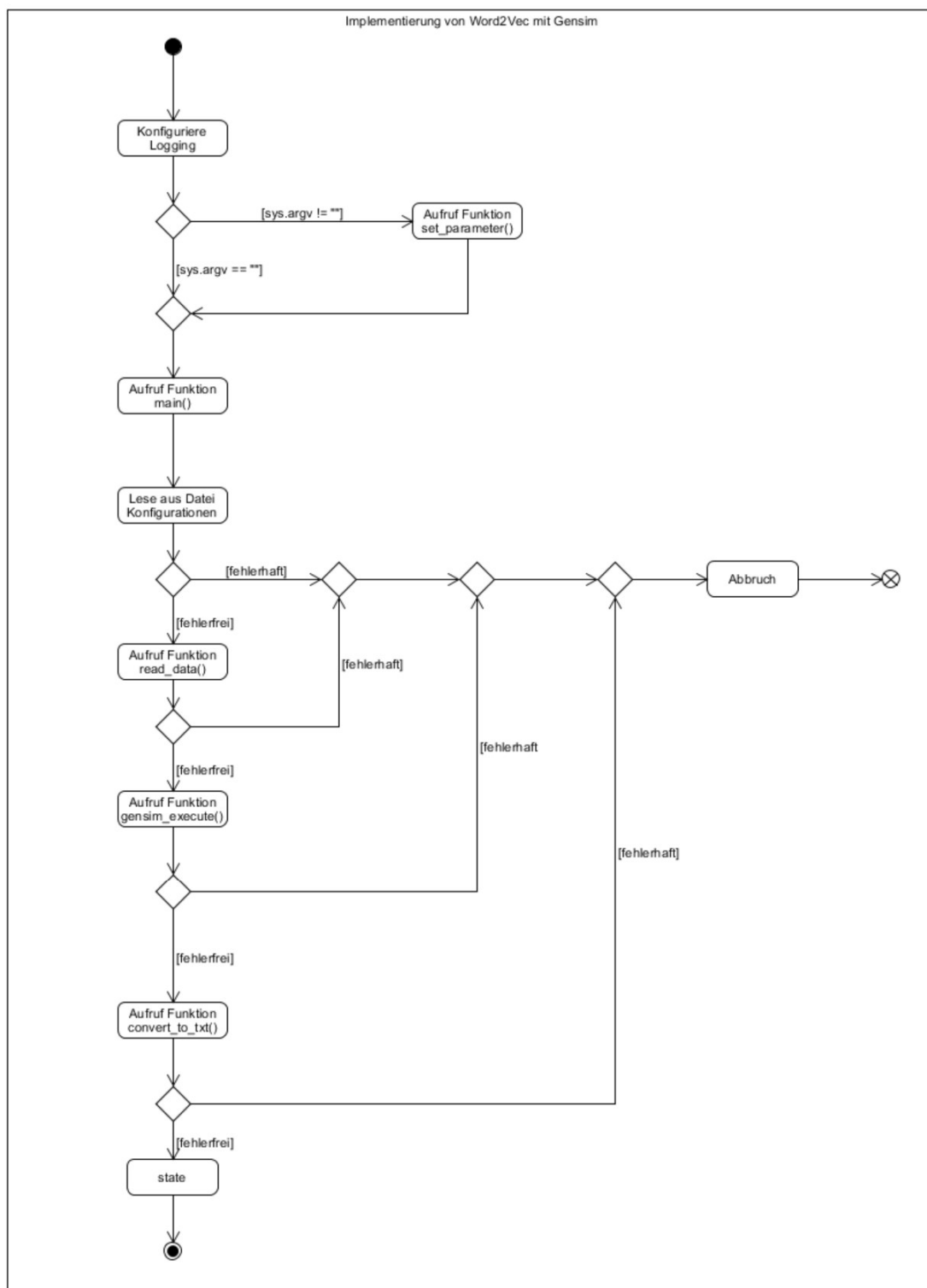
Anhang B: Aufbereitung der Trainingsdaten mit Python und Gensim.WikiCorpus



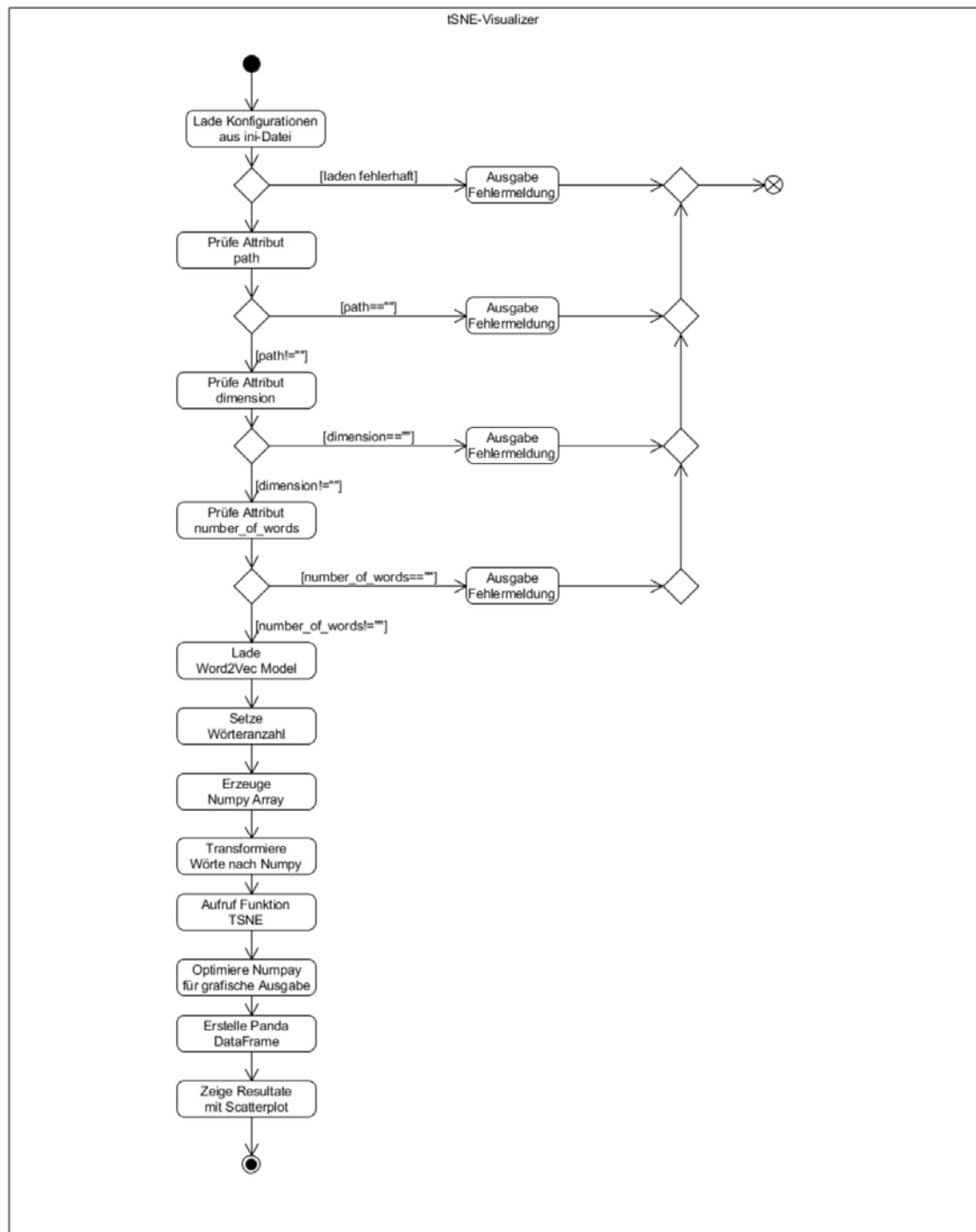
Anhang C: Prototypisierung des Verfahrens GloVe



Anhang D: Prototypisierung des Verfahrens Word2vec mit Gensim



Anhang E: Programm zur Visualisierung von Word2Vec Modellen



Anhang F: Prädiktionsergebnisse der Modellevaluierungen

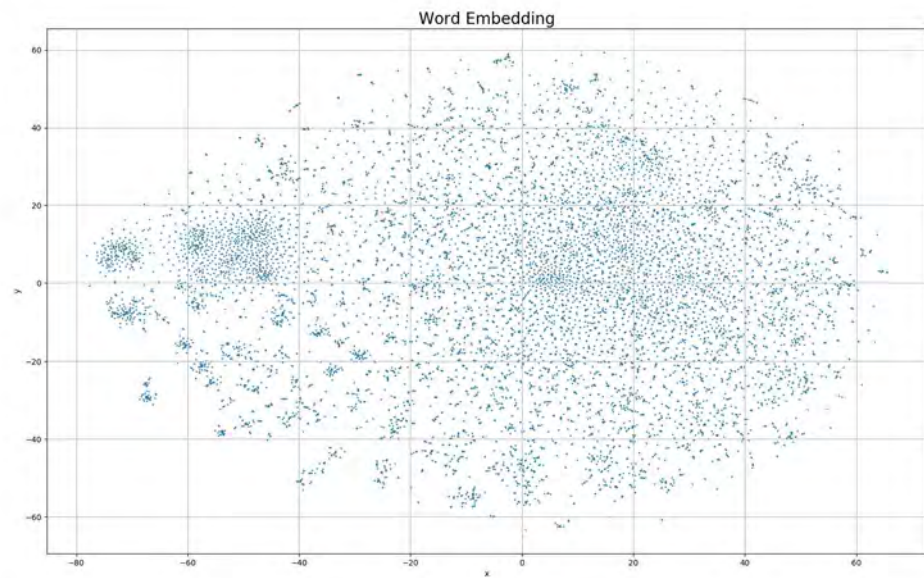
- Verfahren GloVe

Used System	Corpus	Training_ID	Verbose	Memory	Vocab	Min_Count	Vocab	Max_Count	Vector	Size	Max_Iter	Window_Size	Binary_Nbr	Threads	X	Max_ETI	Save_File	Corpus_Size	Accuracy
Glove	wikipedia_150mb_1.txt	2	2	15.0	100	100	100000	300	15	5	2	8	100 0.05	glove_wikipedia_150mb_1.2	150	58.435		150	58.435
Glove	wikipedia_150mb_2.txt	2	2	15.0	100	100	100000	300	15	5	2	8	100 0.05	glove_wikipedia_150mb_2.2	150	59.766		150	59.766
Glove	wikipedia_150mb_3.txt	2	2	15.0	100	100	100000	300	15	5	2	8	100 0.05	glove_wikipedia_150mb_3.2	150	59.292		150	59.292
Glove	wikipedia_300mb_1.txt	2	2	15.0	100	100	100000	300	15	5	2	8	100 0.05	glove_wikipedia_300mb_1.2	300	60.316		300	60.316
Glove	wikipedia_300mb_2.txt	2	2	15.0	100	100	100000	300	15	5	2	8	100 0.05	glove_wikipedia_300mb_2.2	300	33.648		300	33.648
Glove	wikipedia_300mb_3.txt	2	2	15.0	100	100	100000	300	15	5	2	8	100 0.05	glove_wikipedia_300mb_3.2	300	61.584		300	61.584
Glove	wikipedia_500mb_1.txt	2	2	15.0	100	100	100000	300	15	5	2	8	100 0.05	glove_wikipedia_500mb_1.2	500	62.968		500	62.968
Glove	wikipedia_500mb_2.txt	2	2	15.0	100	100	100000	300	15	5	2	8	100 0.05	glove_wikipedia_500mb_2.2	500	63.023		500	63.023
Glove	wikipedia_500mb_3.txt	2	2	15.0	100	100	100000	300	15	5	2	8	100 0.05	glove_wikipedia_500mb_3.2	500	26.344		500	26.344
Glove	wikipedia_150mb_1.txt	3	3	15.0	100	100	100000	300	50	5	2	8	100 0.05	glove_wikipedia_150mb_1.3	150	58.306		150	58.306
Glove	wikipedia_150mb_2.txt	3	3	15.0	100	100	100000	300	50	5	2	8	100 0.05	glove_wikipedia_150mb_2.3	150	61.387		150	61.387
Glove	wikipedia_150mb_3.txt	3	3	15.0	100	100	100000	300	50	5	2	8	100 0.05	glove_wikipedia_150mb_3.3	150	58.304		150	58.304
Glove	wikipedia_300mb_1.txt	3	3	15.0	100	100	100000	300	50	5	2	8	100 0.05	glove_wikipedia_300mb_1.3	300	58.884		300	58.884
Glove	wikipedia_300mb_2.txt	3	3	15.0	100	100	100000	300	50	5	2	8	100 0.05	glove_wikipedia_300mb_2.3	300	61.914		300	61.914
Glove	wikipedia_300mb_3.txt	3	3	15.0	100	100	100000	300	50	5	2	8	100 0.05	glove_wikipedia_300mb_3.3	300	59.851		300	59.851
Glove	wikipedia_500mb_1.txt	3	3	15.0	100	100	100000	300	50	5	2	8	100 0.05	glove_wikipedia_500mb_1.3	500	63.285		500	63.285
Glove	wikipedia_500mb_2.txt	3	3	15.0	100	100	100000	300	50	5	2	8	100 0.05	glove_wikipedia_500mb_2.3	500	62.733		500	62.733
Glove	wikipedia_500mb_3.txt	3	3	15.0	100	100	100000	300	50	5	2	8	100 0.05	glove_wikipedia_500mb_3.3	500	62.886		500	62.886
Glove	wikipedia_150mb_1.txt	1	1	15.0	100	100	100000	300	100	5	2	8	100 0.05	glove_wikipedia_150mb_1.1	150	60.850		150	60.850
Glove	wikipedia_150mb_2.txt	1	1	15.0	100	100	100000	300	100	5	2	8	100 0.05	glove_wikipedia_150mb_2.1	150	63.262		150	63.262
Glove	wikipedia_150mb_3.txt	1	1	15.0	100	100	100000	300	100	5	2	8	100 0.05	glove_wikipedia_150mb_3.1	150	61.420		150	61.420
Glove	wikipedia_300mb_1.txt	1	1	15.0	100	100	100000	300	100	5	2	8	100 0.05	glove_wikipedia_300mb_1.1	300	60.137		300	60.137
Glove	wikipedia_300mb_2.txt	1	1	15.0	100	100	100000	300	100	5	2	8	100 0.05	glove_wikipedia_300mb_2.1	300	60.425		300	60.425
Glove	wikipedia_300mb_3.txt	1	1	15.0	100	100	100000	300	100	5	2	8	100 0.05	glove_wikipedia_300mb_3.1	300	56.155		300	56.155
Glove	wikipedia_500mb_1.txt	1	1	15.0	100	100	100000	300	100	5	2	8	100 0.05	glove_wikipedia_500mb_1.1	500	64.217		500	64.217
Glove	wikipedia_500mb_2.txt	1	1	15.0	100	100	100000	300	100	5	2	8	100 0.05	glove_wikipedia_500mb_2.1	500	62.499		500	62.499
Glove	wikipedia_500mb_3.txt	1	1	15.0	100	100	100000	300	100	5	2	8	100 0.05	glove_wikipedia_500mb_3.1	500	60.816		500	60.816

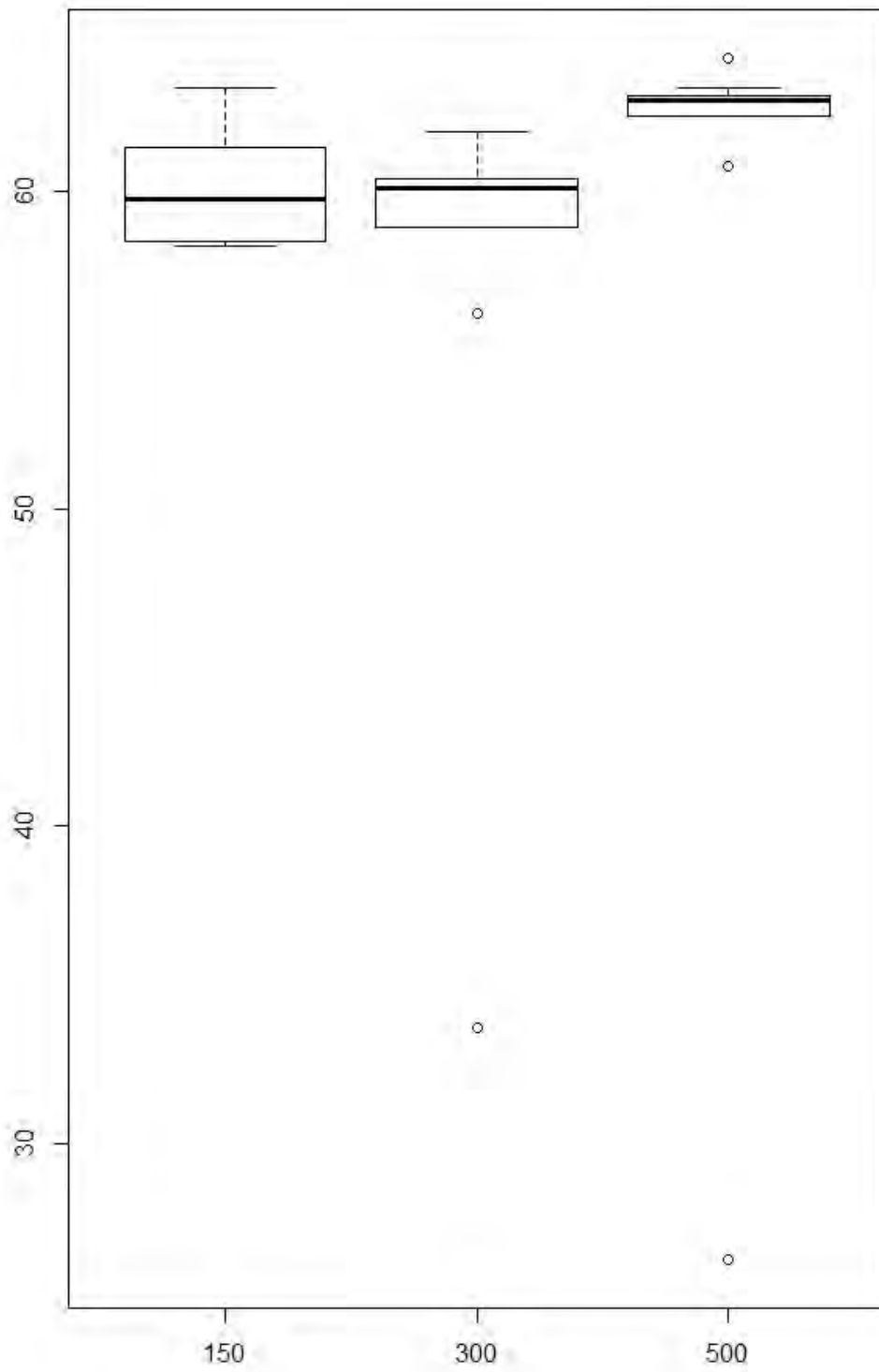
- Verfahren Word2Vec

Corpus	Training_ID	Iterations	Min_Count	Size	Workers	Window	Sg	Hs	Negative	Alpha	Save_File	Corpus_Size	Accuracy
wikipedia_150mb_1.txt	2	15	100	300	8	5	1	1	0.05		vec_wikipedia_150mb_1_2	150	64,245
wikipedia_150mb_2.txt	2	15	100	300	8	5	1	1	0.05		vec_wikipedia_150mb_2_2	150	17,050
wikipedia_150mb_3.txt	2	15	100	300	8	5	1	1	0.05		vec_wikipedia_150mb_3_2	150	37,340
wikipedia_300mb_1.txt	2	15	100	300	8	5	1	1	0.05		vec_wikipedia_300mb_1_2	300	63,793
wikipedia_300mb_2.txt	2	15	100	300	8	5	1	1	0.05		vec_wikipedia_300mb_2_2	300	65,474
wikipedia_300mb_3.txt	2	15	100	300	8	5	1	1	0.05		vec_wikipedia_300mb_3_2	300	62,834
wikipedia_500mb_1.txt	2	15	100	300	8	5	1	1	0.05		vec_wikipedia_500mb_1_2	500	67,513
wikipedia_500mb_2.txt	2	15	100	300	8	5	1	1	0.05		vec_wikipedia_500mb_2_2	500	65,226
wikipedia_500mb_3.txt	2	15	100	300	8	5	1	1	0.05		vec_wikipedia_500mb_3_2	500	65,531
wikipedia_150mb_1.txt	3	50	100	300	8	5	1	1	0.05		vec_wikipedia_150mb_1_3	150	63,805
wikipedia_150mb_2.txt	3	50	100	300	8	5	1	1	0.05		vec_wikipedia_150mb_2_3	150	53,924
wikipedia_150mb_3.txt	3	50	100	300	8	5	1	1	0.05		vec_wikipedia_150mb_3_3	150	64,875
wikipedia_300mb_1.txt	3	50	100	300	8	5	1	1	0.05		vec_wikipedia_300mb_1_3	300	64,188
wikipedia_300mb_2.txt	3	50	100	300	8	5	1	1	0.05		vec_wikipedia_300mb_2_3	300	66,276
wikipedia_300mb_3.txt	3	50	100	300	8	5	1	1	0.05		vec_wikipedia_300mb_3_3	300	51,165
wikipedia_500mb_1.txt	3	50	100	300	8	5	1	1	0.05		vec_wikipedia_500mb_1_3	500	65,069
wikipedia_500mb_2.txt	3	50	100	300	8	5	1	1	0.05		vec_wikipedia_500mb_2_3	500	67,539
wikipedia_500mb_3.txt	3	50	100	300	8	5	1	1	0.05		vec_wikipedia_500mb_3_3	500	66,338
wikipedia_150mb_1.txt	1	100	100	300	8	5	1	1	0.05		vec_wikipedia_150mb_1_1	150	64,071
wikipedia_150mb_2.txt	1	100	100	300	8	5	1	1	0.05		vec_wikipedia_150mb_2_1	150	64,492
wikipedia_150mb_3.txt	1	100	100	300	8	5	1	1	0.05		vec_wikipedia_150mb_3_1	150	60,154
wikipedia_300mb_1.txt	1	100	100	300	8	5	1	1	0.05		vec_wikipedia_300mb_1_1	300	63,209
wikipedia_300mb_2.txt	1	100	100	300	8	5	1	1	0.05		vec_wikipedia_300mb_2_1	300	65,899
wikipedia_300mb_3.txt	1	100	100	300	8	5	1	1	0.05		vec_wikipedia_300mb_3_1	300	63,762
wikipedia_500mb_1.txt	1	100	100	300	8	5	1	1	0.05		vec_wikipedia_500mb_1_1	500	65,749
wikipedia_500mb_2.txt	1	100	100	300	8	5	1	1	0.05		vec_wikipedia_500mb_2_1	500	65,280
wikipedia_500mb_3.txt	1	100	100	300	8	5	1	1	0.05		vec_wikipedia_500mb_3_1	500	64,983

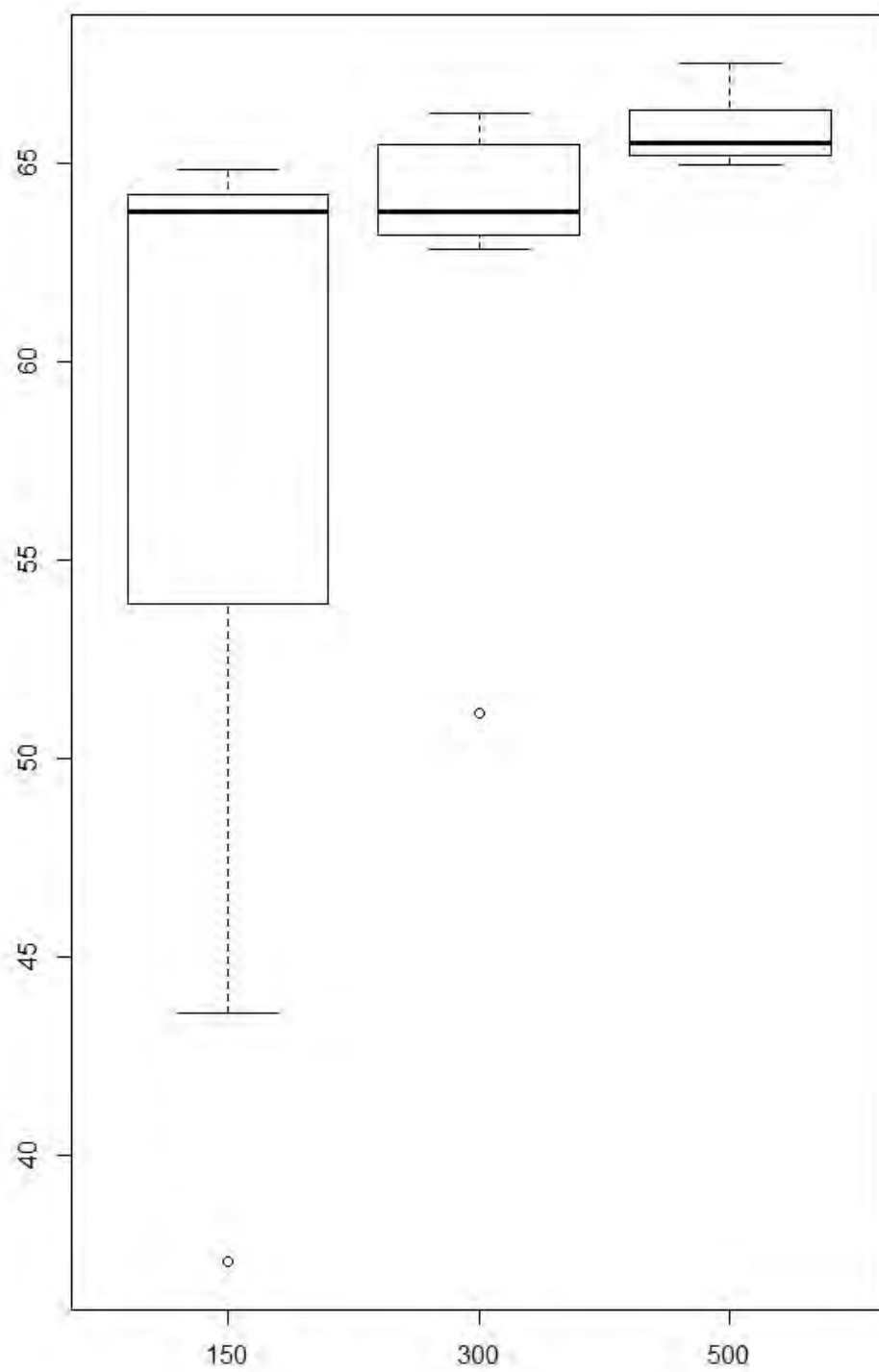
Anhang G: Visualisierung der Word2Vec Modelle



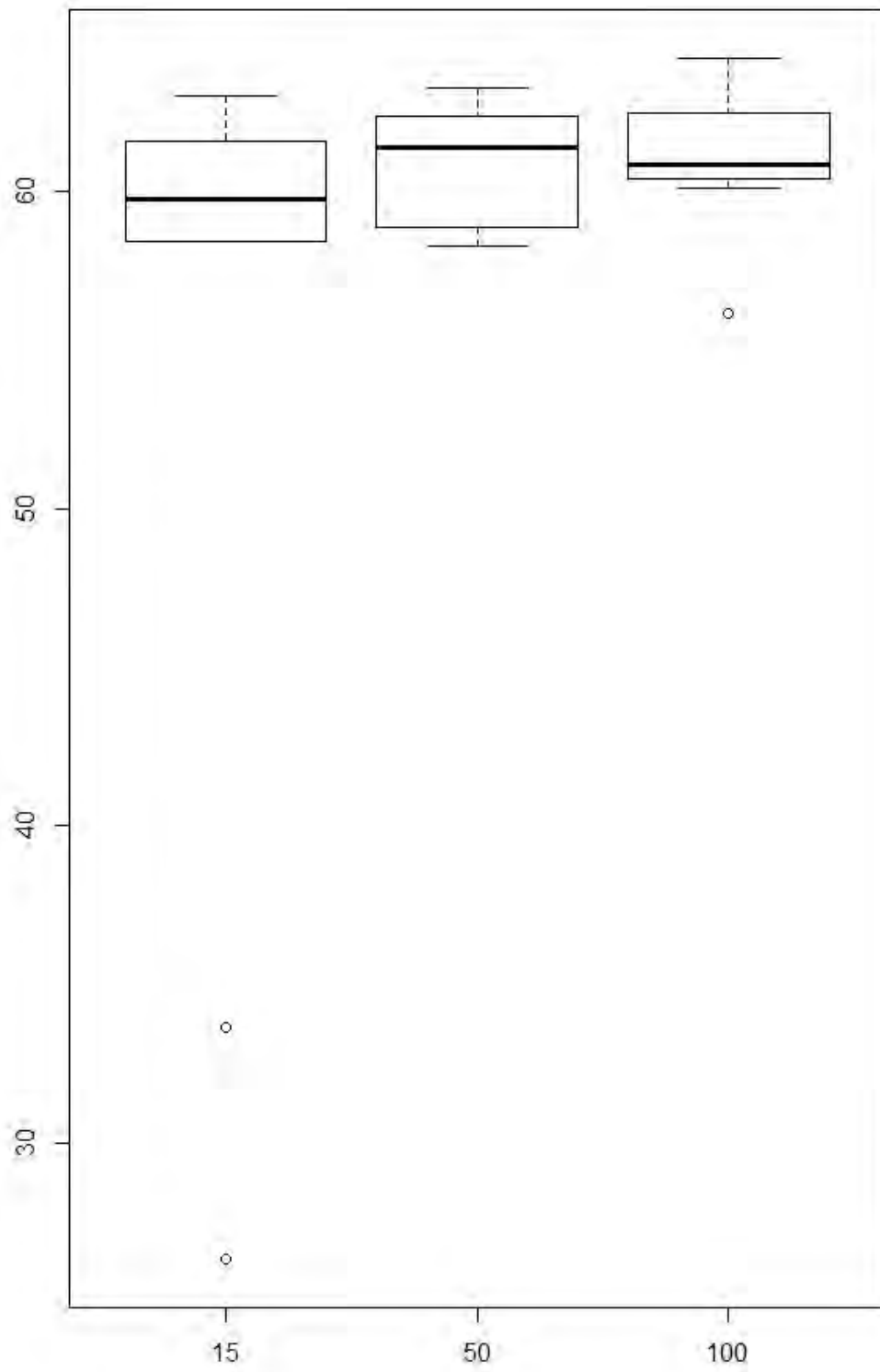
Anhang H: Boxplot der Hypothese H1a



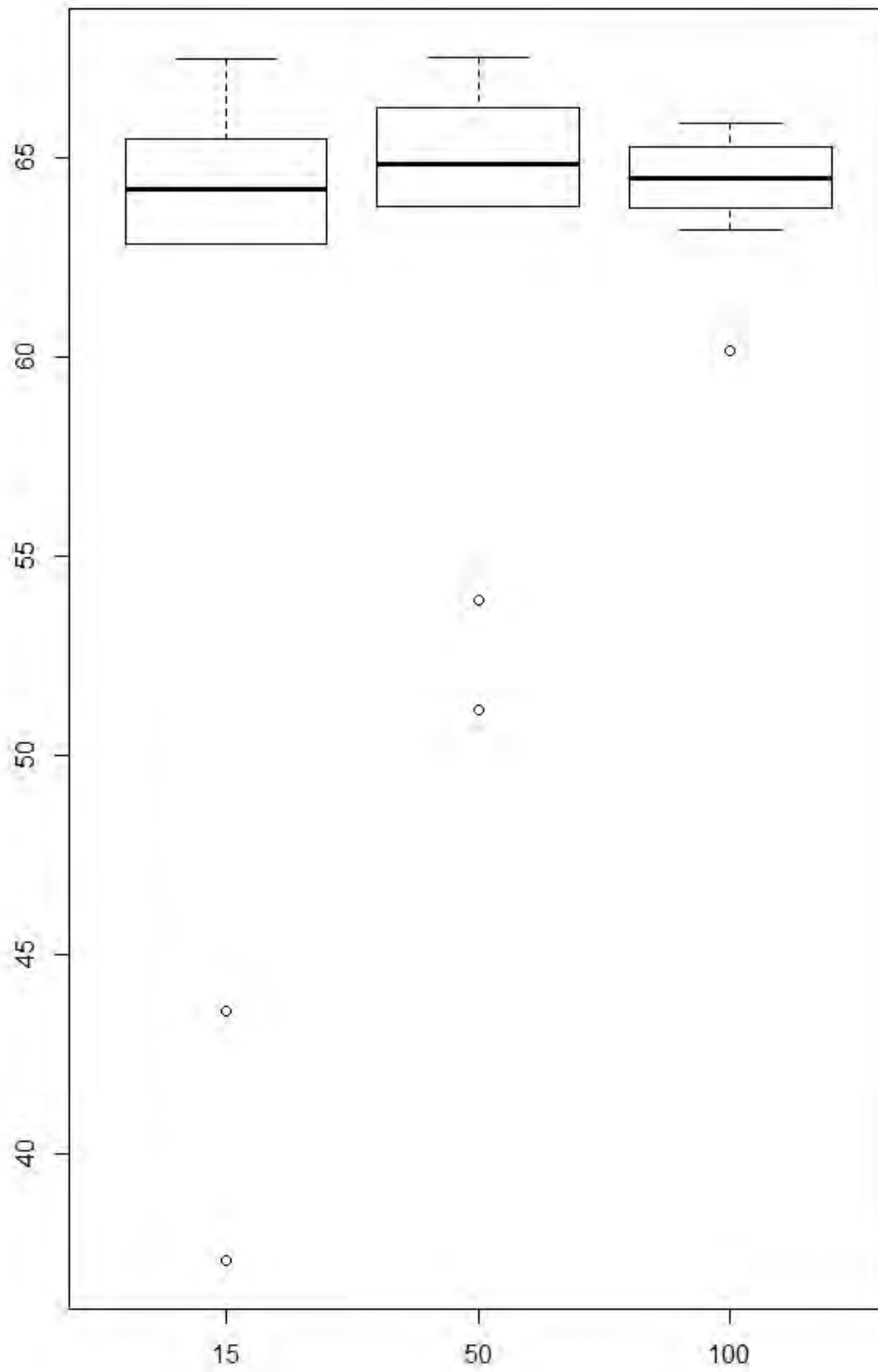
Anhang I: Boxplot der Hypothese H1b



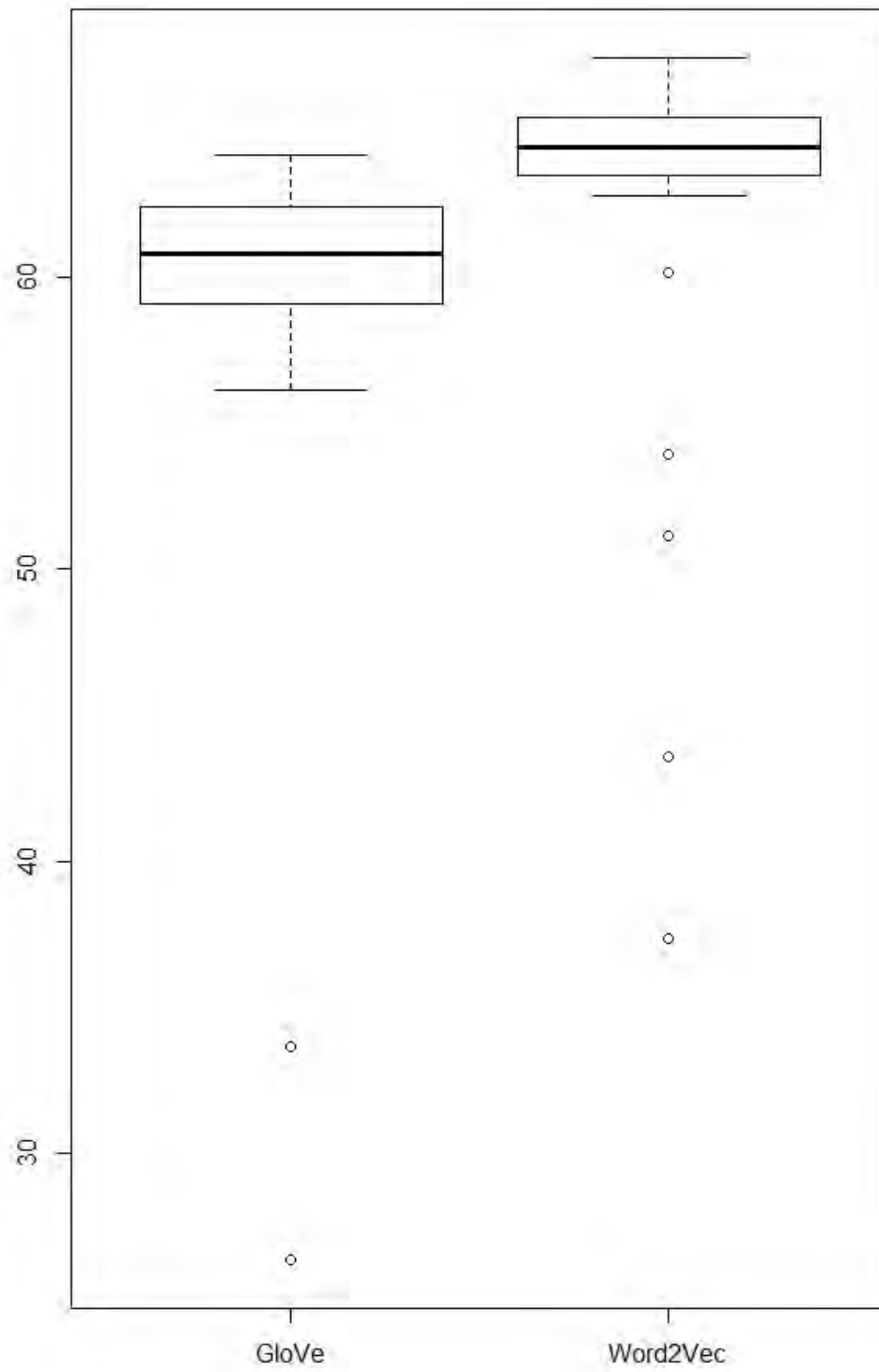
Anhang J: Boxplot der Hypothese H2a



Anhang K: Boxplot der Hypothese H2b



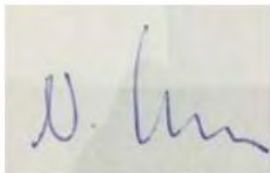
Anhang L: Boxplot der Leitfrage L1



Ehrenwörtliche Erklärung

Hiermit versichere ich, dass die vorliegende Arbeit von mir selbstständig und ohne unerlaubte Hilfe angefertigt worden ist, insbesondere dass ich alle Stellen, die wörtlich oder annähernd wörtlich aus Veröffentlichungen entnommen sind, durch Zitate als solche gekennzeichnet habe. Ich versichere auch, dass die von mir eingereichte schriftliche Version mit der digitalen Version übereinstimmt. Weiterhin erkläre ich, dass die Arbeit in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde/Prüfungsstelle vorgelegen hat. Ich erkläre mich damit einverstanden/nicht einverstanden, dass die Arbeit der Öffentlichkeit zugänglich gemacht wird. Ich erkläre mich damit einverstanden, dass die Digitalversion dieser Arbeit zwecks Plagiatsprüfung auf die Server externer Anbieter hochgeladen werden darf. Die Plagiatsprüfung stellt keine Zurverfügungstellung für die Öffentlichkeit dar.

Ratingen, den 05.04.2019



Nils Horn